

Mastering machine learning with Python practical with tips and technique

Ramandeep Singh

Assistant Professor

Department of Computer Applications

Swami Vivekanand group of institutes

Village Ram Nagar Near Banur / 140401.

Suvidha

Assistant Professor

Department of Computer Applications

Swami Vivekanand Group of Institutes

Village Ram Nagar Near Banur / 140401.

Shivani Guleria

Assistant Professor

Department Of Civil Engineering

Swami Vivekanand Group of Institutes

Village Ram Nagar Near Banur / 140401.

Varinder Kaur

Assistant Professor

Department of Computer Science

Sri Guru Granth Sahib World University

Fatehgarh Sahib, Pin code- 140406.

Title: Mastering machine learning with Python practical with tips and technique

Author's Name: Ramandeep Singh
Suvidha
Shivani Guleria
Varinder Kaur

Published by: Aasan International Publications,
Virudhunagar, 626204, Tamil Nadu, India

Publisher's Address: Aasan International Publications,
Virudhunagar, 626204, Tamil Nadu, India.
26 Main Street, Suite 103, Chatham, NJ 07928, US.

Printer's Details: Aasan International Publications,
Virudhunagar, 626204, Tamil Nadu, India.
26 Main Street, Suite 103, Chatham, NJ7928, US.

Edition Details (I,II,III): I

ISBN: 978-81-982695-7-7 (India)
979-8-9898612-7-9 (United States)

Month & Year: January,2024

Copyright @ Ramandeep Singh
Suvidha
Shivani Guleria
Varinder Kaur

Pages: 223

Price: 650`/-

ACKNOWLEDGEMENT

The field of **Mastering machine learning with Python practical with tips and technique** is transforming the way we approach problem-solving and decision-making in today's data-driven world. From powering personalized recommendations on streaming platforms to enabling self-driving cars and predictive analytics, machine learning is at the heart of many technological advancements. This book, "**Mastering machine learning with Python practical with tips and technique**," is designed to provide a comprehensive understanding of machine learning principles, techniques, and the diverse real-world applications that are reshaping industries.

The main objective of this book is to introduce readers to the fundamental concepts of machine learning and demonstrate how these concepts can be applied across various domains. Machine learning is a rapidly evolving field, blending statistical methods, computer science, and domain knowledge. This book aims to strike a balance between theory and practice, offering a structured approach that starts with foundational principles and progresses to advanced algorithms and practical applications.

I am deeply grateful to all those who have supported the development of this book. Special thanks go to my colleagues, students, and industry experts who provided invaluable feedback, and to the technical reviewers whose insights helped refine the content. I also extend my appreciation to the open-source community for their contributions to the tools and libraries that make machine learning accessible to everyone. Machine learning is a powerful tool that is unlocking new possibilities and reshaping our understanding of what is achievable through data. By exploring the topics presented in this book, I hope you gain the skills, knowledge, and inspiration to apply machine learning effectively and ethically. The journey of learning machine learning is both challenging and rewarding, and I encourage you to experiment, innovate, and stay curious.

Special thanks are due for a number of friends and colleagues in encouraging us to start the work and publish it. We sincerely express our thanks to our family members for their support and love without whom this book wouldn't be possible. We give thanks to Almighty God for his ever-present help and grace in compiling and completing this book on "**Mastering machine learning with Python practical with tips and technique**".

Ramandeep Singh

Suvidha

Shivani Guleria

Varinder Kaur

INDEX

CHAPTER NO.	TOPICS	PAGE NO.
1	Introduction	1 - 56
1.1	Learning Problems	1
1.2	Designing a Learning System	6
1.3	Perspective and Issues	8
1.4	Concept Learning	11
1.5	Version space and candidate Eliminations	19
1.6	Inductive Bias	28
1.7	Decision Tree Learning	33
1.8	Heuristic Search	52
2	Neural Networks and Genetic Algorithms	57 - 99
2.1	Introduction to Artificial Neural Networks (ANN)	57
2.2	Neural Network Representation	61
2.3	Challenges and Suitability of Backpropagation in Neural Networks	73
2.4	Perceptrons	75
2.5	Back Propagation Algorithms	81
2.6	Genetic Algorithm	84
2.7	Hypothesis Space Search	91
2.8	Genetic Programming (GP)	95
2.9	Models of Evolution and Learning	97
3	Bayesian and Computational Learning	100 - 145
3.1	Bayes Theorem	100
3.2	Concept Learning	106
3.3	Maximum Likelihood	108
3.4	Minimum Description Length	112

3.5	Bayes Optimal Classifier	117
3.6	Bayesian Belief Network	123
3.7	EM Algorithm	129
3.8	Probablistic Model	133
3.9	Sample Complexity	137
3.10	Mistake Bound Model	142
4	Instant Based Learning	146 - 160
4.1	K-Nearest Neighbour Learning	146
4.2	Locally Weighted Linear Regression	151
4.3	Radial Basis Function (RBF)	154
4.4	Case based Learning	157
5	Advanced Learning	161 - 208
5.1	Learning Sets of Rules	161
5.2	Sequential Covering Algorithms	163
5.3	First Order Rules	167
5.4	Sets of First-Order Rules: FOIL	170
5.5	Induction on Inverted Deduction	173
5.6	Inverting Resolution	176
5.7	Analytical Learning	179
5.8	Perfect Domain Theories	181
5.9	Explanation-Based Learning (EBL)	184
5.10	FOCL Algorithm	188
5.11	Reinforcement Learning	190
5.12	Markov Decision Process	196
5.13	Q-Learning	200
5.14	Temporal Difference (TD) learning	206
6	Applications	209 - 223
6.1	Image Recognition	209
6.2	Speech Recognition	212

6.3	Traffic Prediction	215
6.4	Product Recommendations	218
6.5	Self-Driving Cars	218
6.6	Email Spam and Malware Filtering	219
6.7	Virtual Personal Assistants	220
6.8	Online Fraud Detection	221
6.9	Stock Market Trading	221
6.10	Medical Diagnosis	222
6.11	Automatic Language Translation	223

Introduction

1.1 Learning Problems

Definition: In the context of machine learning, a program is said to "learn" from experience E when it demonstrates improved performance P on a task T after being exposed to relevant data or experiences. In simpler terms, if a program's performance on task T improves after exposure to specific data, then it is learning from experience E .

For a machine learning problem to be well-defined, it must be possible to determine a unique solution based on the input data or experience, and this solution should be resilient to minor variations in the data.

Key Features of a Machine Learning Problem:

1. **Task:** The specific problem or activity the machine is intended to perform.
2. **Performance Measure:** The metric used to evaluate the effectiveness of the machine in completing the task.
3. **Experience:** The data or past occurrences from which the machine can learn to improve its performance.

How do we define T , P , and E ? Examples of these elements include:

♦ Robot Driving Problem:

- ❖ **Task (T):** Driving a vehicle on a four-lane road using visual input.
- ❖ **Performance Measure (P):** Average distance travelled between errors, where errors are detected by a human supervisor.
- ❖ **Experience (E):** Observing a human driver's steering and visual input as they navigate the road.

♦ Handwriting Recognition Problem:

- ❖ **Task (T):** Identifying and categorizing handwritten words.
- ❖ **Performance Measure (P):** Percentage of words correctly identified and categorized.
- ❖ **Experience (E):** A collection of labelled handwritten words used for training.

♦ Text Categorization Problem:

- ❖ **Task (T):** Assigning text to a suitable category.

- ❖ **Performance Measure (P):** Accuracy in placing text in the correct category.
- ❖ **Experience (E):** A set of pre-labeled documents for training.

Why Study Machine Learning?

What is Machine Learning (ML)? Machine Learning, a subset of Artificial Intelligence (AI), is focused on developing computational methods that allow systems to learn and improve based on experiences or data inputs. The purpose is to develop systems that can autonomously learn from data patterns without explicit human intervention.

Machine Learning can be defined as programming computers to enhance their performance on a specific task T through experience E and evaluation using performance metrics P . In this sense, machine learning resembles "programming by example," where the system generalizes from provided examples rather than following a fixed set of instructions.

Machine learning algorithms are particularly useful for identifying relationships within large datasets (also called "data mining") and for solving problems where predefined algorithms may not exist. For instance, recognizing faces or interpreting natural language is difficult to program manually due to the complexity of human perception and language. By learning from large datasets, machines can begin to make sense of these tasks.

Why Machine Learning is Important:

1. **Generalization from Examples:** Machine learning enables systems to generalize solutions by learning from previous instances rather than relying solely on fixed programming rules.
2. **Business Insights and Intelligence:** Machine learning supports advanced analytics, providing businesses with valuable insights for informed decision-making and competitive advantage.
3. **Handling Complexity in Data:** Machine learning can uncover hidden relationships and correlations within large datasets, enabling insights that traditional programming cannot easily extract.
4. **Adaptability:** Machine learning systems can adapt to new data and changing conditions, making them suitable for dynamic environments.
5. **Solving Complex Problems:** With applications in areas like computer vision, speech recognition, and robotics, machine learning can address problems that are difficult to solve with conventional programming methods.

How Do Machines Learn?

The machine learning process typically involves three main stages:

1. **Training:** A model is trained using a set of examples that represent correct outputs. This stage involves feeding the machine with labeled data that allows it to learn patterns and relationships.
2. **Validation:** During this stage, the rules or patterns learned during training are assessed. A separate dataset, which the model has not seen before, is used to evaluate how well it performs. Additional training may occur if needed to enhance accuracy and robustness.
3. **Testing:** The final phase involves applying the model to new, unseen data to test its ability to generalize effectively beyond the examples it learned from. The performance on this test data helps assess the model's real-world applicability.

Why Machine Learning is Effective:

1. **Complex Pattern Recognition:** Machine learning excels in identifying patterns within complex data, such as images or unstructured text, where traditional algorithms struggle.
2. **Hidden Insights:** By analyzing vast datasets, machine learning can reveal insights that would be challenging for human analysts to discover.
3. **Continuous Learning:** Machine learning models can be updated and refined with new data, allowing them to improve over time.

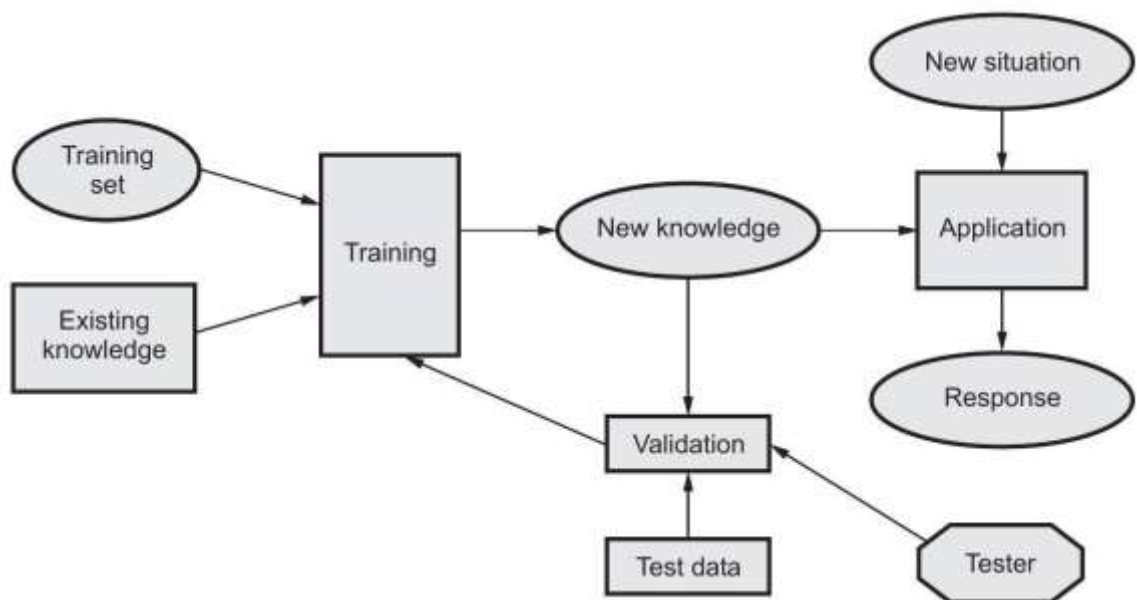


Fig. 1.1. Phases of ML.

1. **Automating Predictions:** Once trained, machine learning models can make predictions and classify data automatically, reducing the need for human intervention in routine tasks.
2. **Dynamism and Scalability:** Machine learning systems are well-suited for applications where the environment is constantly changing, as they can quickly adapt by learning from new experiences.

By leveraging these methods, machine learning has revolutionized fields ranging from medical diagnostics to autonomous vehicles, transforming how data is used to inform and automate complex decision-making processes.

Face Recognition

Effortless Recognition in Daily Life: Human beings can easily recognize faces daily, effortlessly identifying friends, family members, and acquaintances. This recognition capability extends to photographs, where we can identify people despite variations in pose, hairstyle, background lighting, makeup, and even age. This innate skill demonstrates the brain's ability to process complex visual information and distinguish individuals based on subtle facial features.

The Subconscious Nature of Face Recognition: Our ability to recognize faces is largely subconscious. Most people can recognize a familiar face instantly, without being able to describe exactly how they do it. For instance, we might recognize a friend on a crowded street, but if asked to explain the process step-by-step, we may struggle to articulate the exact features or sequence of visual cues that led to recognition. This lack of conscious understanding makes it challenging to create traditional algorithms for face recognition, as such algorithms require explicitly defined steps that computers can follow.

Structural Patterns in Faces: A face is not just a random collection of pixels; it has a specific structure. Every face generally contains distinct features, including the eyes, nose, mouth, and ears, arranged symmetrically in predictable patterns. These facial components vary between individuals but maintain a level of consistency that makes them recognizable as human faces. The unique arrangement of these features—like the distance between the eyes, the shape of the nose, and the contour of the jawline—creates a distinctive pattern for each individual. By analyzing multiple images of a person, a face recognition system can learn the unique structure of that face, enabling it to distinguish that individual from others.

In machine learning, the algorithm learns to detect these unique patterns through training on a dataset of labeled face images. The system identifies specific combinations of features that uniquely characterize each individual and stores these as a “faceprint,” a digital representation of that person's facial structure.

Once the system has been trained, it can use this faceprint to recognize the person in new images, even if they appear in different lighting, angles, or expressions.

Role of Machine Learning in Face Recognition: Machine learning plays a crucial role in face recognition by creating an optimized model that can identify and distinguish faces based on past data and experiences. The learning process involves feeding the model a large number of labeled images so that it can learn to generalize facial patterns across different people and conditions. Through this training, the model becomes more effective at recognizing faces accurately, even in challenging situations like partial occlusion, changes in facial expression, or varying lighting conditions.

The machine learning model undergoes a training process that includes two main components:

- 1. **Feature Extraction:** The model learns to identify key features in faces—such as the distance between the eyes, the contour of the face, and the shape of the mouth. This feature extraction is vital, as it allows the model to break down complex visual data into manageable pieces of information that it can analyze systematically.
- 2. **Pattern Matching:** Once the model has identified the facial features, it compares these patterns to the stored faceprints in its database. This comparison helps determine if the new face matches a known individual or if it is a new face. The model’s ability to optimize its decision-making through experience improves its accuracy in recognizing familiar faces and distinguishing them from unknown individuals.

Differences between Machine Learning and Data Mining

Machine Learning	Data Mining
In machine learning the main goal is to learn a model, which can be used to predict future events.	In data mining, the main goal is to discover new interesting information which describes the current data set.
It considered data as secondary.	It considered data as primary.
Machine learning uses relatively complex and global models.	Data mining uses simple models or local patterns.
Only hundreds or thousands of examples in a training data set.	Huge data sets, even millions of rows.
To learn one or few carefully defined models, this can be used to predict future events.	To find all interesting patterns which describe the data set.

Table.1.1 Machine Learning and Data Mining.

1.2 Designing a Learning System

In machine learning, designing an effective learning system involves addressing fundamental issues and selecting appropriate approaches to achieve the desired outcome. For example, let's consider creating a system that can learn to play checkers and participate in a competitive tournament. The design of this system would require carefully considered steps:

1. **Selecting the Training Experience:** This step involves deciding the type of experiences from which the system will learn. For instance, should the system learn by observing human players, by analyzing past game records, or by playing against itself?
2. **Defining the Target Function:** The target function is the goal or objective the system aims to optimize. In this case, it would be a function that helps the system select the best moves to win the game.
3. **Choosing a Representation for the Target Function:** Representation defines how the target function will be structured mathematically or logically so the machine can use it effectively in decision-making. It could involve defining game scenarios or board states in a way that the machine can process.
4. **Selecting a Function Approximation Algorithm:** The system needs an algorithm to approximate the target function based on the training data, as it is often infeasible to explicitly calculate every possible outcome in a complex game like checkers.

Training Experience

The training experience influences the performance goals and learning quality of the system. Here are key aspects to consider when designing the training experience:

1. **Type of Feedback:** There are two main types of feedback—direct and indirect. Direct feedback provides immediate information about the accuracy of an action (e.g., the system is informed if a move was successful or not). Indirect feedback requires the system to infer the effectiveness of actions over a series of moves, which can be more challenging but may mimic real-world learning scenarios.
2. **Learning Strategy:** This involves deciding whether the system should be taught by an external agent (such as a human player) or should explore moves independently. A balance between exploration (trying new moves) and exploitation (using known successful moves) is essential to enhance learning.
3. **Diversity of Training:** To ensure comprehensive learning, the training data should represent a variety of game situations. For example, should the system play against itself, against other players, or with a range of different tactics? Diverse experiences can help the system handle varied situations in real matches.

In this checkers example, we may decide that the system will learn by playing against itself, allowing it to generate diverse gameplay scenarios without needing external input. This self-play approach ensures that the system encounters a wide range of board configurations and strategies.

Choosing Training Experience Components:

- ❖ **Direct or Indirect Feedback:** Direct feedback simplifies learning but might not reflect complex real-world challenges, while indirect feedback allows for more nuanced decision-making.
- ❖ **Learner's Control:** The system's autonomy in exploring and making decisions can impact its learning depth and adaptability.
- ❖ **Distribution of Examples:** Ensuring the training examples are representative of real gameplay scenarios makes the system's learning more applicable and reliable.

Choosing and Representing the Target Function

The target function defines the type of knowledge the system should acquire and how it will be applied during gameplay. In the context of a checkers-playing system, the target function could evaluate potential board positions and assign them a score that represents the likelihood of winning from that position.

Representing the Target Function:

1. **Expressive Representation:** Using a representation that captures subtle variations in board configurations can allow for more accurate approximations of the target function.
2. **Simple Representation:** In cases where simplicity is key, a straightforward function can provide reasonable guidance without excessive complexity.

For example, the target function for a checkers game could assign values to board states based on factors like:

- ❖ The number of pieces each player has.
- ❖ The number of pieces threatened by the opponent.
- ❖ The spatial control on the board (e.g., controlling the center).

An alternative approach might use a recursive definition to evaluate board states, where the system assigns higher values to better positions. The function $V(b)$ could be defined as follows:

- ❖ $V(b) = 100$: if b represents a winning position.

- ❖ $V(b) = -100$: if b represents a losing position.
- ❖ $V(b) = 0$: if b represents a draw.
- ❖ $V(b) = V(b')$: if b' is the best achievable board state assuming optimal play.

Estimating Training Values

During training, the system must estimate values for intermediate board states to learn the consequences of different moves. The goal is to assign scores to board states that accurately reflect the future outcome of a game. One approach is to estimate the value of a board state $V_{\text{train}}(b)$ based on the system's approximation of the subsequent board state $V(\text{Successor}(b))$. This method helps the system evaluate positions more accurately, especially as the game nears completion, where outcomes become clearer.

- ❖ This process is straightforward yet effective for gradually improving the system's accuracy in predicting board state values, thus enhancing its overall performance.

Adjusting the Weights

In a machine learning system, weights (w_i) represent the importance of different factors in decision making. Adjusting these weights allows the system to fine-tune its learning based on experience, helping it better approximate the target function.

To refine these weights, the system uses a method such as minimizing the squared error E between the actual training values and the predicted values:

$$E = \sum_{(b, V_{\text{train}}(b)) \text{ training examples}} (V_{\text{train}}(b) - \hat{V}(b))^2 \quad (1.1)$$

where $\hat{V}(b)$ is the system's estimate of $V(b)$ for a given board state b .

Incremental algorithms, like Least Mean Squares (LMS), can adjust these weights as new training examples become available, making the system adaptive to new data. LMS helps the system continuously refine its predictions, making it more robust to variations and errors in training examples.

1.3 Perspective and Issues

Perspective in Machine Learning

Machine learning represents a transformative approach to data-driven decision-making, enabling systems to learn from data and improve over time. The primary perspective in ML is to create models that can generalize from examples, perform tasks autonomously, and adapt to new data. Here are some key perspectives shaping the ML field today:

1. **Automation of Decision-Making:** Machine learning is centered on the idea of automating complex decision-making processes. It allows systems to make predictions or take actions without explicit human instructions for every possible scenario. This ability to automate can significantly increase efficiency and reduce human effort in various domains.
2. **Pattern Recognition and Knowledge Discovery:** ML algorithms are adept at recognizing patterns and discovering hidden insights within data. This capability is highly valuable in fields like healthcare, finance, and marketing, where identifying trends and correlations can drive innovation and strategic decisions.
3. **Improvement through Experience:** Machine learning systems improve their performance as they are exposed to more data. This concept of "learning from experience" is fundamental, allowing models to refine their predictions and adapt over time. In contrast to traditional software, ML models can evolve and perform better with each new dataset.
4. **Adaptability to Diverse Domains:** ML is versatile and can be applied across diverse fields. From image and speech recognition to natural language processing and robotics, ML techniques are adaptable to a wide range of tasks. This flexibility has made ML a core technology for applications ranging from autonomous vehicles to personalized recommendations.
5. **Interdisciplinary Influence:** Machine learning intersects with statistics, computer science, neuroscience, and cognitive psychology. This interdisciplinary nature allows it to leverage insights from different fields to improve algorithms, develop new models, and enhance performance in complex tasks.
6. **Ethical and Societal Implications:** The impact of ML on society is profound, with implications for privacy, job displacement, and fairness. Ethical considerations are an integral part of the field, as developers and researchers strive to create systems that are fair, unbiased, and respectful of user privacy.

Issues in Machine Learning

While machine learning holds vast potential, it also faces several critical issues that must be addressed to ensure reliable and responsible deployment. These issues are primarily related to data quality, model design, ethics, and deployment challenges. Here's an overview of the main issues:

1. Data Quality and Bias:

- ❖ **Data Quality:** ML models require accurate, clean, and relevant data. In real-world scenarios, data is often noisy, incomplete, or inconsistent, which can compromise model accuracy.

- ❖ **Bias in Data:** Datasets may carry inherent biases, leading models to make biased predictions. For example, training a hiring algorithm on historical data that reflects gender bias may lead to discrimination in hiring decisions.
- ❖ **Data Privacy:** Collecting and utilizing personal data raises privacy concerns, and regulations like GDPR mandate that personal information be handled with care, impacting data availability for training.

2. Algorithm Selection and Model Complexity:

- ❖ **Algorithm Suitability:** Selecting the right algorithm is crucial, as different tasks require different techniques. The suitability of an algorithm affects model performance, computational efficiency, and interpretability.
- ❖ **Overfitting and Underfitting:** ML models must strike a balance between fitting the training data well (without overfitting) and generalizing to new data. Overly complex models may capture noise rather than patterns, while overly simple models may miss valuable information.
- ❖ **Interpretability:** Complex models like deep neural networks are often difficult to interpret, making it challenging to understand how they arrive at specific predictions, which is a major issue in high-stakes fields like healthcare.

3. Computational Resources and Scalability:

- ❖ **High Computational Costs:** Training large-scale ML models, particularly deep learning models, requires significant computational power, which can be costly and environmentally taxing.
- ❖ **Scalability:** As datasets grow in size and complexity, it becomes challenging to scale models efficiently. Managing computational resources while maintaining performance is an ongoing challenge.

4. Ethics and Fairness:

- ❖ **Bias and Fairness:** Ensuring fairness in ML models is critical, particularly in applications with social implications, such as criminal justice or hiring. Bias in data can lead to unfair treatment of certain groups.
- ❖ **Transparency:** ML models, especially black-box models, lack transparency, making it difficult for stakeholders to trust the model's decisions. Developing interpretable models is essential to build trust.
- ❖ **Accountability:** As ML systems make autonomous decisions, questions arise regarding accountability in case of errors. For instance, if an autonomous vehicle causes an accident, determining responsibility is a complex ethical issue.

5. Deployment and Maintenance:

- ❖ **Model Drift:** ML models can degrade over time if the underlying data distribution changes (e.g., consumer behavior shifts), requiring regular updates and retraining.

- ❖ **Security:** ML models can be vulnerable to adversarial attacks, where slight modifications to the input data can mislead the model. Ensuring security against such attacks is crucial for safe deployment.
- ❖ **Integration with Legacy Systems:** Many organizations have existing infrastructure, and integrating ML models into these legacy systems can be challenging.

6. Regulatory and Legal Compliance:

- ❖ **Compliance with Regulations:** Machine learning applications, particularly those handling personal or sensitive data, must comply with data protection regulations like GDPR, HIPAA, or CCPA.
- ❖ **Liability and Legal Responsibility:** Defining legal responsibilities for ML-driven decisions is an emerging issue, especially in areas where ML outcomes have real-world consequences, such as autonomous vehicles or financial services.

1.4 Concept Learning

In machine learning, a primary challenge is deriving general patterns or rules from specific training examples. This process is known as **concept learning**, where the goal is to form a general definition of a category based on positive (category examples) and negative (non-category examples) instances.

Definition of Concept Learning: Concept learning involves identifying a broad classification rule that can categorize new examples accurately, based on past observations. It is the process of determining the general characteristics of a group from specific examples and non-examples of that group. This is achieved by distinguishing the positive training samples (that belong to the concept) from the negative ones (that do not belong to the concept).

Concept learning can be visualized as a search through a predefined hypothesis space. Here, the task is to identify the hypothesis that best matches the training examples provided. A hypothesis represents a potential explanation or rule for the concept being learned. The process involves navigating a range of hypotheses to find one that accurately reflects the examples in the training data.

The hypothesis space is structured hierarchically, often organized in a "general-to-specific" order. This structure allows for efficient searching within the space, as one can leverage inherent patterns to reach a solution faster.

Formal Definition of Concept Learning: The process of concept learning can also be described as identifying a Boolean-valued function that can correctly label input instances as either "true" or "false," based on their membership in the concept. For instance, concept learning can involve identifying birds from images by distinguishing between "bird" (positive) and "non-bird" (negative) examples.

Through this method, we aim to comprehend and generalize a concept from the examples given. Concept learning is fundamentally about creating a mapping between input variables (attributes of examples) and a Boolean value (whether or not they fit the concept). This approach is often called **inductive learning** as it draws general conclusions from specific instances.

When a function can correctly classify training examples, it has the potential to work on new, unseen data as well, a process known as **generalization**. Generalization is crucial because it enables the model to apply learned knowledge to unfamiliar situations, improving the model's adaptability.

Example: Suppose we want to learn the concept of days on which a friend enjoys a particular water sport. We would need to observe specific days when they participate and when they do not, identifying the common conditions or attributes (such as weather or day of the week) that indicate a "water sport" day. This learned concept would then enable us to predict other days they might engage in the sport.

Example	Sky	Air Temp	Humidity	Wind	Water	Forecast	Enjoy Sport
1	Sunny	Warm	Normal	Strong	Warm	Same	YES
2	Sunny	Warm	High	Strong	Warm	Same	YES
3	Rainy	Cold	High	Strong	Warm	Change	NO
4	Sunny	Warm	High	Strong	Warm	Change	YES
	ATTRIBUTES						↑ CONCEPT

Table 1.2 Example of Concept Learning.

In this example, we examine a set of days, each described by six attributes (Sky, Air Temperature, Humidity, Wind, Water, and Forecast). Our goal is to use these attributes to predict whether someone would enjoy a sport (represented by "EnjoySport") on any given day. This prediction task requires identifying patterns in the relationship between attribute values and the outcome, "EnjoySport," and generalizing this pattern to make predictions about new, unseen instances.

The Inductive Learning Hypothesis

The **inductive learning hypothesis** serves as a fundamental assumption in machine learning. It states that if a hypothesis accurately approximates the target function across a large set of training examples, it will likely perform well on other, unobserved examples. This assumption underpins the concept of generalization in machine learning. By learning from specific examples, we assume the model can apply the learned pattern to new situations.

Objective of the Learning Task

The task is to discover a hypothesis h that aligns as closely as possible with the target concept c , which describes the conditions under which "EnjoySport" would be true or false. Ideally, this hypothesis would apply across all possible instances within the entire space X . However, since we only have access to a subset of instances (the training examples), the learning algorithm must use this data to form a generalized rule that performs well on both seen and unseen instances.

Guarantees and Limitations of Inductive Learning

Inductive learning algorithms aim to produce a hypothesis that best fits the target concept for the given training data. Although these algorithms can ensure a fit to the training data, they cannot guarantee that the hypothesis will work perfectly for all future cases. Nonetheless, the inductive learning hypothesis gives us reason to believe that a hypothesis that performs well on training data will likely perform well on similar new data, enabling effective generalization.

Assumptions in Inductive Learning

In inductive learning, without additional information, the best approach is often to assume that the hypothesis most consistent with the training data will perform well on new data. This assumption forms the core principle of inductive learning, enabling models to infer broader rules from a limited set of examples.

Representation of Hypothesis Constraints

In concept learning, hypotheses are structured by specifying constraints on each attribute. These constraints guide the model in determining when the "EnjoySport" concept applies. Each attribute in the hypothesis can take certain values, represented as follows:

- ❖ "?" – This symbol denotes that any value for the attribute is acceptable.
- ❖ " Φ " – This symbol indicates that no value is acceptable for the attribute.

A hypothesis can then be represented by specifying values for each attribute, such as: <Sky, AirTemp, Humidity, Wind, Water, Forecast>

For example, a hypothesis might specify that the Sky must be "Sunny" and Humidity "Normal," while other attributes are unrestricted or allowed to vary. This combination of constraints enables the hypothesis to represent different levels of generalization, identifying conditions under which "EnjoySport" is true.

Example: Using Hypotheses to Predict "EnjoySport"

In the example table:

- **Example 1:** A "Sunny" day with "Warm" air, "Normal" humidity, "Strong" wind, "Warm" water, and a stable forecast leads to "EnjoySport" being true.
- **Example 2:** Another "Sunny" day with slightly different conditions (higher humidity) also results in "EnjoySport" being true.
- **Example 3:** A "Rainy" day with "Cold" air, high humidity, and changing conditions does not lead to "EnjoySport."
- **Example 4:** A "Sunny" day, warm but with a different forecast, still results in "EnjoySport" being true.

These examples provide the data for determining which attribute combinations are necessary for the concept of "EnjoySport" to be true. By examining these patterns, we can form a hypothesis that can be generalized for future instances.

1.4.1 Concept Learning as Search

In machine learning, **concept learning** can be thought of as a process of searching through a broad hypothesis space, where each hypothesis represents a possible explanation of the concept. The hypothesis space is implicitly defined by the representation of attributes and possible constraints on these attributes.

The primary objective in this search is to identify a hypothesis that best matches the training data, fitting both positive examples (which belong to the concept) and negative examples (which do not). By carefully choosing a hypothesis representation, the designer of a learning algorithm effectively defines the set of all possible hypotheses that the algorithm can explore, enabling the program to learn within this predefined space.

Hypothesis as Constraints on Attributes

A hypothesis can be represented as a vector of constraints for each attribute in the dataset. This vector specifies the acceptable values for each attribute:

1. A "?" symbol indicates that any value for the attribute is acceptable.
2. A specific value requirement means that the attribute must match a particular value.
3. A " Φ " symbol represents that no value for this attribute is acceptable.

When an instance x fulfills all constraints defined by a hypothesis h , it is classified as a positive example by that hypothesis, indicating that $h(x)=1$

Example Hypothesis for "Enjoy-Sport" Concept Learning Task:

In the task of predicting whether someone would enjoy a sport, hypotheses about acceptable weather conditions for "Enjoy-Sport" are represented as constraints on attributes like Sky, Air Temperature, Humidity, Wind, Water, and Forecast. Each hypothesis outlines specific conditions under which "Enjoy-Sport" is true.

Defining Instances and Hypothesis Space

Consider a dataset where each day is characterized by attributes such as:

- ❖ **Sky** (possible values: Sunny, Cloudy, Rainy)
- ❖ **AirTemp** (Warm, Cold)
- ❖ **Humidity** (Normal, High)
- ❖ **Wind** (Strong, Weak)
- ❖ **Water** (Warm, Cool)
- ❖ **Forecast** (Same, Change)

Each hypothesis, denoted H , consists of constraints on these attributes, and these constraints determine whether "EnjoySport" will be classified as true or false.

To measure the scope of this task:

- Distinct Instances: By multiplying the possibilities for each attribute, we calculate the total number of unique combinations of attribute values:

$$3 \times 2 \times 2 \times 2 \times 2 \times 2 = 96$$

- Total Hypotheses: Each attribute can take any value, be unrestricted (?), or be disallowed (Φ), resulting in a substantial hypothesis space:

$$5 \times 4 \times 4 \times 4 \times 4 \times 4 = 5120$$

The hypothesis space also includes combinations of different constraints, creating a large set of possibilities. For instance, the six attributes allow various specific or general conditions, resulting in a significant number of hypotheses, though practical applications often have even larger or infinite hypothesis spaces.

General-to-Specific Ordering of Hypotheses

In concept learning, hypotheses can be organized from **general to specific**. This structure allows algorithms to search more efficiently by focusing first on the broadest hypotheses and narrowing down to more specific ones as needed. This approach avoids the need to explicitly examine every hypothesis, which is particularly beneficial when dealing with infinite hypothesis spaces.

Example of General and Specific Hypotheses

Consider two hypotheses:

- $h_1 = (\text{Sunny} , ? , ? , \text{Strong} , ? , ?)$
- $h_2 = (\text{Sunny} , ? , ? , ? , ? , ?)$

Here, h_2 is more general than h_1 because it imposes fewer constraints. While h_1 requires a "Strong" wind in addition to "Sunny" sky, h_2 is satisfied by any wind condition. Therefore, any instance that meets h_1 's criteria will also meet h_2 's, making h_2 more general.

This generality relationship is crucial for the learning algorithm, as it can determine whether a more specific or general hypothesis is better suited to match the training data. The goal is often to find the most specific hypothesis that still aligns with the examples provided, ensuring accuracy while avoiding unnecessary restrictions.

More-General-Than-Or-Equal Relationship

In concept learning, we can define a more-general-than-or-equal-to relationship between hypotheses. Given two hypotheses, h_1 and h_2 , we say that h_1 is more-general-than-or-equal-to h_2 (written as $h_1 \geq h_2$) if every instance that satisfies h_2 also satisfies h_1 . Formally, this relationship holds if:

$$h_1 \geq h_2 \Leftrightarrow \forall x \in X: h_2(x) = 1 \Rightarrow h_1(x) = 1$$

Additionally, h_1 is strictly more general than h_2 if it includes all instances that satisfy h_2 and possibly more. Conversely, h_2 is more specific than h_1 if it satisfies fewer instances. This ordering is a valuable tool for navigating hypothesis spaces efficiently, especially when determining which hypothesis best captures the target concept.

By using these principles, concept learning algorithms can systematically search through a vast or even infinite hypothesis space to find a suitable hypothesis that generalizes well, balancing the trade-off between specificity and generality by $\downarrow$ ' on the training data.

$h_1 = \langle \text{Sunny}, ?, ?, \text{Strong}, ?, ? \rangle$

$h_2 = \langle \text{Sunny}, ?, ?, ?, ?, ? \rangle$

$h_3 = \langle \text{Sunny}, ?, ?, ?, \text{Cool}, ? \rangle$

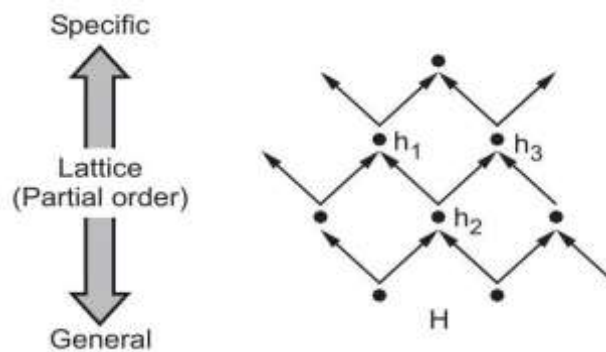


Fig. 1.2 Hypothesis Example.

1.4.2 Find-S Algorithm

- ❖ FIND-S Algorithm starts from the most specific hypothesis and generalize it by considering only positive examples.
- ❖ This algorithm ignores negative examples. As long as the hypothesis space contains a hypothesis that describes the true target concept, and the training data contains no errors, ignoring negative examples does not cause to any problem.
- ❖ FIND-S algorithm finds the most specific hypothesis within H that is consistent with the positive training examples. The final hypothesis will also be consistent with negative examples if the correct target concept is in H, and the training examples are correct.

Example	Sky	Air Temp	Humidity	Wind	Water	Forecast	Enjoy Sport
1	Sunny	Warm	Normal	Strong	Warm	Same	YES
2	Sunny	Warm	High	Strong	Warm	Same	YES
3	Rainy	Cold	High	Strong	Warm	Change	NO
4	Sunny	Warm	High	Strong	Cool	Change	YES

Table. 1.3 Find-S Algorithm.

$h = \langle \phi, \phi, \phi, \phi, \phi, \phi \rangle$
 $h = \langle \text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same} \rangle$
 $h = \langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle$
 $h = \langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ? \rangle$

Algorithm :

- ❖ Initialize h to the most specific hypothesis in H :
- ❖ For each attribute training instance x :

For each attribute constraint a in h :

If the constraint is not satisfied by x .

Then replace a , by the next more general constraint satisfied by x .

- ❖ Output hypothesis h

Example	Sky	Air Temp	Humidity	Wind	Water	Forecast	Enjoy Sport
1	Sunny	Warm	Normal	Strong	Warm	Same	YES
2	Sunny	Warm	High	Strong	Warm	Same	YES
3	Rainy	Cold	High	Strong	Warm	Change	NO
4	Sunny	Warm	High	Strong	Cool	Change	YES

Table. 1.4 Output hypothesis.

$h = \langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ? \rangle$

Prediction :

5	Rainy	Cold	High	Strong	Warm	Change	NO
6	Sunny	Warm	Normal	Strong	Warm	Same	YES
7	Sunny	Warm	Low	Strong	Cool	Same	YES

Table. 1.5 Prediction.

- ❖ The output hypothesis is the most specific one that satisfies all positive training examples.
- ❖ The result is consistent with the positive training examples.

Example	Sky	Air Temp	Humidity	Wind	Water	Forecast	Enjoy Sport
1	Sunny	Warm	Normal	Strong	Warm	Same	YES
2	Sunny	Warm	High	Strong	Warm	Same	YES
3	Rainy	Cold	High	Strong	Warm	Change	NO
4	Sunny	Warm	High	Strong	Cool	Change	YES
5	Sunny	Warm	Normal	Strong	Cool	Change	NO

Table. 1.6 Positive training Examples.

$h = \langle \text{Sunny, Warm, ?, Strong, ?, ?} \rangle$

- ❖ The result is consistent with the negative training examples if the target concept is contained in H (and the training examples are correct).
- ❖ Sizes of the space:
- ❖ Size of the instance space : $|X| = 3 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 96$
- ❖ Size of the concept space $C = 2^{|X|} = 2^{96}$
- ❖ Size of the hypothesis space $H = (4 \cdot 3 \cdot 3 \cdot 3 \cdot 3) + 1 = 973 \ll 2^{96}$.
 $\Rightarrow$ The largest concept (in C) may not be contained in H .

1.5 Version space and candidate Eliminations

1.5.1 Version Space

In machine learning, version space refers to the set of all hypotheses that remain consistent with the training examples provided.

These hypotheses collectively represent possible solutions that align with the observed data. The version space is denoted as $VS_{H,D}$, where H represents the hypothesis space (all possible hypotheses) and D represents the set of training examples. Essentially, $VS_{H,D}$ is the subset of hypotheses from H that do not contradict any examples in D .

Hierarchical Representation of Knowledge with Version Space

Version space is a hierarchical organization of knowledge, which allows the algorithm to retain the essential information from a sequence of training examples without needing to store every individual example. By managing the space of consistent hypotheses instead of specific data points, the version space approach becomes an efficient way of processing and retaining relevant knowledge.

In concept learning, the version space method helps maintain multiple models within a defined space, filtering out inconsistent hypotheses as new training examples are introduced. This approach aids in narrowing down to the correct hypothesis by eliminating candidates that do not align with observed data.

Characteristics of Version Space

The main features of version space include the following:

1. **Representation of Tentative Hypotheses:** Version space encapsulates various plausible hypotheses that are consistent with the observed data, allowing the algorithm to maintain several potential solutions.
2. **Alternative Descriptions of Heuristics:** All plausible descriptions of the target concept are included within the version space, each of which represents a potential understanding of the concept based on current data.
3. **Compatibility with Positive Examples:** A plausible hypothesis in the version space is one that correctly classifies all known positive examples while avoiding any negative examples.
4. **Complementary Generalization and Specialization Trees:**
 - ❖ The **generalization tree** contains nodes representing overly broad hypotheses. These are general models that cover large subsets of instances.
 - ❖ The **specialization tree** consists of nodes representing very specific hypotheses that may apply to only a few or single instances.
5. **Discrete Attribute Values:** Each hypothesis in the version space operates on discrete attribute values, making it easier to define and classify the possible conditions under which the concept applies.

Fundamental Assumptions in Version Space

Certain foundational assumptions support the use of version space in concept learning:

- ❖ **Accuracy of Data:** All training data is assumed to be correct, meaning that there are no mislabeled or erroneous instances.
- ❖ **Conjunctive Description of the Concept:** The target concept is assumed to be representable by a combination of specific attribute values. In other words, the correct hypothesis is a conjunction (AND combination) of certain conditions on attributes.

Diagrammatic Representation of Version Space

In visual representations of version space, two main trees help organize hypotheses:

- ❖ **Generalization Tree:** This tree structures hypotheses from specific to general, with each node covering all hypotheses in its subtree.
- ❖ **Specialization Tree:** This tree organizes hypotheses from general to specific, with each node representing only a limited subset of the hypothesis space.

Links between nodes in these trees represent generalization and specialization relationships. Nodes in the generalization tree connect to models that cover everything in their subtree, whereas nodes in the specialization tree connect to models that focus only on specific instances in their subtree.

Compact Representation of Version Space

Instead of listing all hypotheses that align with the training set, version space can be efficiently represented using **boundary sets**. The boundary sets define the **most general** and **most specific** hypotheses that fit the data, which can serve as a compact representation of the version space.

- ❖ **General Boundary (G):** This boundary includes the maximally general hypotheses that are consistent with the training data. Each hypothesis in the general boundary is as broad as possible without including any negative examples.
- ❖ **Specific Boundary (S):** This boundary contains the most specific hypotheses that still satisfy all positive examples in the data. Hypotheses in the specific boundary are as narrow as possible while covering all positive instances.

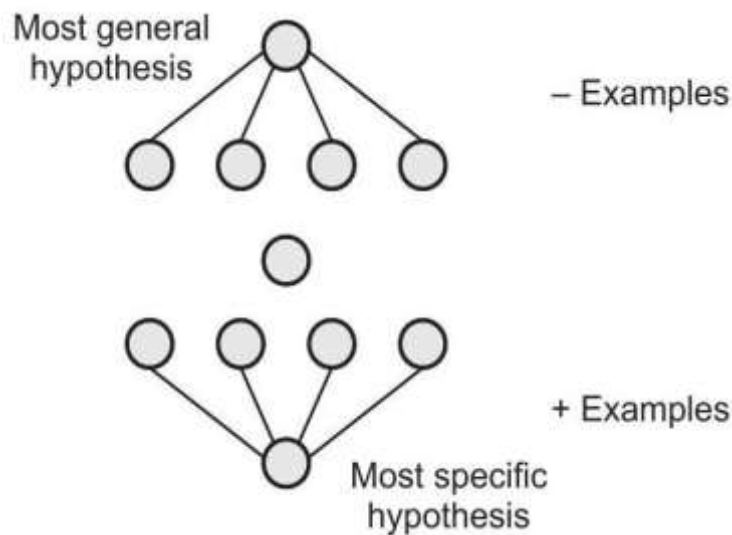


Fig. 1.3 General and specific hypothesis.

- ❖ Each specialization must be a generalization of some specific concept description. No specialization can be a specialization of another general concept description. **Fig. 1.4** shows boundary set with hypothesis

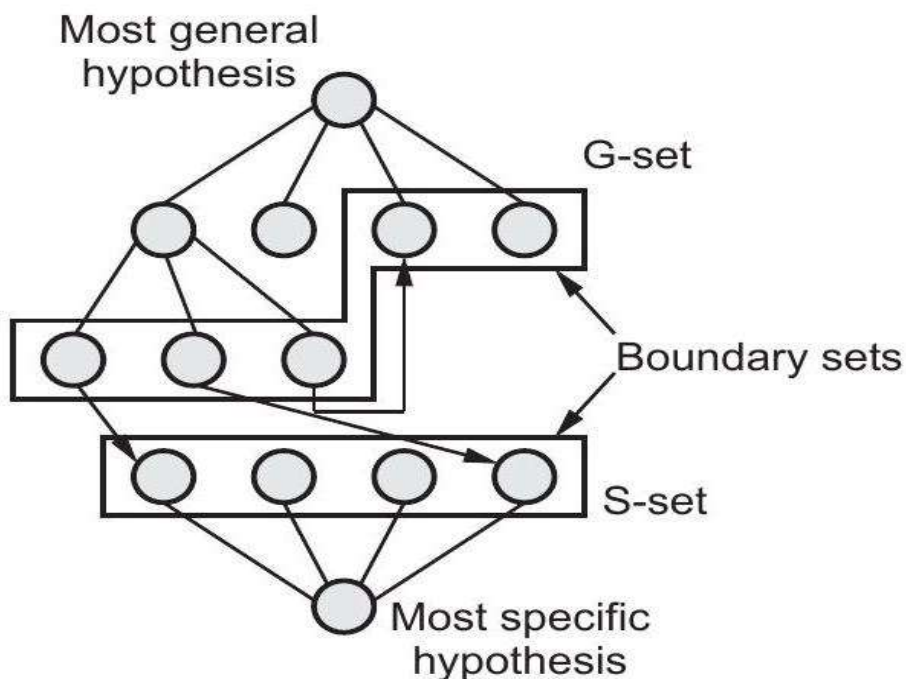


Fig. 1.4 Boundary set with hypothesis.

Advantages of the Version Space Method

1. **Comprehensive Hypothesis Representation:** The version space method is highly effective in representing all possible hypotheses that align with the given data. By systematically narrowing down the hypothesis space, it ensures that every potential explanation within the defined hypothesis language is considered. This approach allows the algorithm to maintain a flexible set of hypotheses that can adapt as more data becomes available, providing a thorough search of the solution space.
2. **Efficiency in Computation:** The process of managing and updating the version space can be relatively efficient, with performance close to linear under ideal conditions. Since the algorithm only needs to adjust the boundaries of the hypothesis space (i.e., the most general and most specific boundaries), it avoids the need for exhaustive searching or recalculating every hypothesis, resulting in faster performance and lower computational cost.

Disadvantages of the Version Space Method

1. **Sensitivity to Noise:** The version space method assumes that all data is accurate and consistent. However, in real-world applications, data often contains noise or errors. When inconsistent data (such as mislabeled examples) is introduced, it can disrupt the learning process, potentially eliminating valid hypotheses that might contain the target concept. This issue can lead to an incorrect or incomplete version space, making it challenging to identify the correct hypothesis.
2. **Difficulty with Disjunctive Concepts:** The version space method is best suited for learning concepts that can be expressed as conjunctive relationships (where attributes combine in an "AND" fashion). When a concept requires a disjunctive representation (where attributes combine in an "OR" fashion), the version space method struggles. Disjunctive concepts are challenging to represent within the strict boundaries of general and specific hypotheses, often requiring alternative approaches or more complex hypothesis spaces to capture the target concept effectively.

List-Then-Eliminate Algorithm

The List-Then-Eliminate algorithm is a method in concept learning used to identify hypotheses consistent with training examples. This algorithm begins by initializing the version space to include all possible hypotheses in the hypothesis space H . Then, it systematically removes any hypothesis that fails to match any training example, effectively "eliminating" hypotheses that are inconsistent with observed data.

How the Algorithm Works

Initialization: At the start, the algorithm assumes that all hypotheses in the hypothesis space. H could potentially represent the target concept. Therefore, the version space initially contains every possible hypothesis.

Elimination Process: As each training example is introduced, the algorithm evaluates each hypothesis in the version space against this example. Any hypothesis that does not correctly classify the example is removed from the version space, narrowing down the set of possible hypotheses.

Progressive Shrinking of the Version Space: With each new example, the version space reduces further, discarding hypotheses that do not align with the observed data. Ideally, this process continues until only one hypothesis remains in the version space, representing a hypothesis that is fully consistent with all provided examples.

Insufficient Data: In cases where the training data is limited, the version space may not narrow down to a single hypothesis. If there are not enough examples to eliminate all but one hypothesis, the algorithm will output the remaining set of hypotheses consistent with the observed data. This output represents the entire range of possible concepts based on the available data.

Advantages of the List-Then-Eliminate Algorithm

The List-Then-Eliminate algorithm is beneficial when the hypothesis space H is finite. Key advantages include:

Guaranteed Consistency: This algorithm ensures that all hypotheses in the final version space are consistent with the training data. This means that any hypothesis left in the version space is guaranteed to correctly classify all observed examples, making the algorithm reliable for identifying accurate concepts.

Comprehensive Filtering: By starting with all hypotheses and removing only those that are inconsistent, the algorithm systematically narrows down to the most plausible hypotheses, providing a complete solution set.

Limitations of the List-Then-Eliminate Algorithm

Despite its advantages, the List-Then-Eliminate algorithm also has notable limitations:

Exhaustive Enumeration: A major drawback of this method is the need to enumerate every hypothesis in the hypothesis space H . For all but the smallest hypothesis spaces, this requirement can be impractical.

In cases with large or infinite hypothesis spaces, exhaustively listing each hypothesis is computationally unfeasible and would demand substantial resources. Limited Applicability to Complex Hypothesis Spaces: Because the algorithm relies on evaluating every hypothesis, it is only realistic for simple or finite hypothesis spaces. For more complex spaces, alternative approaches that do not require exhaustive listing of hypotheses are often preferred.

1.5.2 Candidate Elimination Algorithm

- ❖ The candidate-Elimination algorithm computes the version space containing all (and only those) hypotheses from H that are consistent with an observed sequence of training examples.
- ❖ Example : Learning the concept of "Japanese Economy Car".
- ❖ Features : Country of Origin, Manufacturer, Color, Decade, Type.

Origin	Manufacturer	Color	Decade	Type	Example Type
Japan	Honda	Blue	1980	Economy	Positive
Japan	Toyota	Green	1970	Sports	Negative
Japan	Toyota	Blue	1990	Economy	Positive
USA	Chrysler	Red	1980	Economy	Negative
Japan	Honda	White	1980	Economy	Positive
Japan	Toyota	Green	1980	Economy	Positive
Japan	Honda	Red	1990	Economy	Negative

Table. 1.6 Candidate Elimination Algorithm.

The **Candidate Elimination algorithm** is a concept-learning approach that operates within the version space framework. Its goal is to refine the hypothesis space by gradually eliminating hypotheses that are inconsistent with training examples, ultimately converging on a set of hypotheses that are consistent with all observed data. This process allows the algorithm to identify all possible hypotheses that match the training data, preserving both general and specific boundaries to cover the entire version space.

Initializing the General and Specific Boundaries

1. General Boundary : The algorithm starts by setting G to include the maximally general hypothesis. This is essentially a placeholder hypothesis that includes all possibilities, represented by variables for all attributes (e.g., $= \{(? , ? , ? , ? , ?)\}$). This boundary is flexible enough to match any training example initially.
2. Specific Boundary S : S is initialized to the first positive example encountered, making it the most specific hypothesis possible. By setting S to match the first positive example exactly, it starts with the most restrictive hypothesis and will only be generalized as necessary to accommodate additional positive examples.

Processing Training Examples

As the algorithm processes each training example, it updates G and S based on whether the example is positive or negative:

Positive Example

When the algorithm encounters a positive example, it takes the following steps:

1. Generalize the Specific Boundary : For each hypothesis in S , generalize it slightly to ensure that it can match the new positive example. This generalization involves minimal changes, keeping S as specific as possible while accommodating the example.
2. Prune the General Boundary : Remove any hypotheses from G that fail to match the positive example. This step ensures that only those general hypotheses that are compatible with all positive examples remain.

Negative Example

When a negative example is introduced, the algorithm modifies G and S as follows:

1. Specialize the General Boundary : For each hypothesis in G , specialize it slightly to exclude the negative example. These changes should be minimal, keeping the hypothesis as general as possible while ensuring it does not match the negative example.
2. Prune the Specific Boundary : Remove any hypotheses in S that incorrectly match the negative example. This pruning prevents overly specific hypotheses from remaining in S .

Ensuring Compatibility between S and G

If, at any point, S and G reduce to singleton sets (each containing only one hypothesis), the algorithm checks if they match:

- ❖ If they are identical: The algorithm has converged on a single hypothesis that is consistent with all observed examples. This hypothesis is output as the solution, and the algorithm halts.
- ❖ If they differ: This discrepancy indicates that the training data is inconsistent, meaning the examples cannot be fully represented by a single conjunctive hypothesis. The algorithm outputs this inconsistency and halts.

Example Walkthrough

Consider a set of examples with five attributes. The algorithm follows these steps:

1. **First Positive Example:** Suppose the first positive example is (Japan, Honda, Blue, 1980, Economy). Initialize S to match this example exactly, and G to the maximally general hypothesis:
 - $G = \{(? , ? , ? , ? , ?)\}$
 - $S = \{(\text{Japan}, \text{Honda}, \text{Blue}, 1980, \text{Economy})\}$
2. **Negative Example:** The next example might be negative, such as (Japan, Toyota, Green, 1970, Sports). To exclude this, specialize G by adding constraints:
 - New hypotheses in G might include (Japan, ?, ?, ?, ?), (?, Honda, ?, ?,?), etc., each removing parts that match the negative example.
3. **Subsequent Positive Example:** If another positive example (Japan, Toyota, Blue, 1980, Economy) appears, generalize S to accommodate it by loosening constraints minimally:
 - Update S to $\{(\text{Japan}, ?, \text{Blue}, ?, \text{Economy})\}$ to cover both positive examples.
4. **Final Negative Example:** A final negative example, such as (USA, Chrysler, Red, 1980, Economy), would prompt further specialization of G to ensure it excludes this instance.

When the Version Space Collapses

If at any point, the training examples create conflicts that make it impossible to find a hypothesis in the conjunctive form, the version space may collapse.

This collapse means no consistent conjunctive hypothesis can describe the data, indicating that the concept may be too complex for a simple conjunctive hypothesis representation.

Stopping Criteria for the Algorithm

The Candidate Elimination algorithm stops under the following conditions:

1. **Exhaustion of Training Data:** If all training examples have been processed and S and G have converged (or if no consistent hypothesis can be found).
2. **Unique Solution:** If the algorithm converges to a single hypothesis within the version space.
3. **Version Space Collapse:** If inconsistencies or conflicts cause the version space to collapse, indicating that no hypothesis in the language can match the observed examples consistently.

1.6 Inductive Bias

In machine learning, **inductive bias** refers to the underlying assumptions or preferences that a learning algorithm uses to predict outcomes for new, unseen data based on patterns it has learned from training data. These biases are built into the algorithm's design and provide a framework for making sense of data beyond what has been explicitly observed. Without some form of inductive bias, an algorithm would struggle to generalize from specific examples to broader conclusions, rendering it ineffective for most real-world applications.

Formal Definition of Inductive Bias

In a formal sense, consider a concept learning algorithm L designed to classify instances within a set X . For a target concept c defined over X and a set of training examples $D_c = \{\langle x, c(x) \rangle\}$, the inductive bias of L consists of any minimal set of assumptions B that allows L to classify instances based on D_c .

In other words, the inductive bias B is the minimal collection of assertions that ensures, for any target concept c and any corresponding set of training examples D_c , that the algorithm can apply B in a way that consistently classifies each instance in X . This set of assumptions makes it possible for the algorithm to infer classifications and generalize from its training data.

Role of Inductive Bias in Hypothesis Selection

Inductive bias plays a critical role in guiding the algorithm's choice of hypothesis—the model or explanation it uses to interpret the training data.

Given a hypothesis space (the full set of potential models or explanations), the inductive bias helps the algorithm prioritize certain hypotheses over others, typically aiming for those that balance accuracy on the training data with the potential to perform well on new data. This selection process is essential because the hypothesis space is often vast, and without guidance, the algorithm might select hypotheses that fit the training data too closely or, conversely, are too broad.

Balancing Overfitting and Underfitting

Inductive bias also helps the algorithm manage the trade-off between **overfitting** and **underfitting**:

- ❖ **Overfitting** occurs when an algorithm selects a hypothesis that fits the training data very closely, capturing even minor fluctuations or noise. This leads to poor performance on new data, as the model may be too specific to the training examples.
- ❖ **Underfitting** happens when the algorithm selects a hypothesis that is too general, failing to capture the underlying patterns in the training data. As a result, it performs poorly both on the training data and on new data.

By embedding specific assumptions, the inductive bias helps the algorithm navigate between these extremes, enabling it to identify hypotheses that capture the essential patterns in the data while avoiding irrelevant details.

Examples of Inductive Bias

Inductive bias varies between algorithms and can take different forms. Here are a few examples:

- ❖ **Linear Assumption:** In algorithms like linear regression, the inductive bias assumes that the relationship between input features and the output can be captured by a straight line or plane. This works well for linear relationships but limits the model's ability to capture complex, non-linear patterns.
- ❖ **Smoothness:** Many algorithms, such as k-nearest neighbors, assume that similar instances should produce similar outputs. This smoothness bias works well for datasets where data points that are close in feature space tend to have similar labels.
- ❖ **Preference for Simplicity:** Some algorithms, like decision trees, may incorporate a simplicity bias by preferring shorter, less complex hypotheses. This aligns with **Occam's Razor**, a principle suggesting that simpler explanations are often more accurate.

Importance of Inductive Bias for Generalization

Inductive bias is essential for the algorithm's ability to generalize—its capacity to perform well on data that wasn't part of the training set. Without inductive bias, the algorithm would lack a basis for choosing one hypothesis over another, making it unlikely to generalize effectively. Instead, it would be prone to either memorizing the training data or failing to capture relevant patterns, both of which hinder real-world performance.

Types of Inductive Bias in Machine Learning

Inductive bias is an inherent part of machine learning algorithms, shaping how they interpret training data and generalize to new data. Different algorithms come with unique biases that influence the types of patterns they are more likely to learn and the hypotheses they are more likely to favor. These biases stem from the underlying assumptions of each algorithm and serve as guiding principles for how they approach problem-solving. Here are some common types of inductive bias:

1. Bias towards Simplicity (Occam's Razor)

Many machine learning algorithms, such as decision trees, linear models, and Naive Bayes, exhibit a preference for simpler, more concise hypotheses. This bias towards simplicity, inspired by the principle of **Occam's Razor**, suggests that the simplest explanation that fits the data is often the best one. Simpler models are generally easier to interpret and less likely to overfit, making them more robust for generalizing to new data. For example:

- ❖ **Decision Trees:** Algorithms like CART and C4.5 often prefer trees with fewer splits, which represent simpler rules.
- ❖ **Linear Models:** Linear regression and logistic regression aim to fit data with a straight line or hyperplane, representing the simplest linear relationship.

By favoring parsimonious explanations, these algorithms reduce the risk of capturing noise or irrelevant patterns, which can improve their performance on unseen data.

2. Bias towards Smoothness

Some algorithms assume that similar inputs should yield similar outputs, leading to a **smoothness bias**. This is particularly common in algorithms that focus on local neighborhoods or continuity, such as **kernel methods** (like Support Vector Machines with a Gaussian kernel) and **Gaussian Processes**. These algorithms produce smooth decision boundaries and gradual transitions, meaning that slight changes in input features should not lead to drastic shifts in output predictions.

- ❖ **Kernel Methods:** Algorithms like SVM with a Gaussian (RBF) kernel assume that points close to each other in feature space will have similar labels, resulting in smooth decision boundaries.
- ❖ **Gaussian Processes:** This approach uses a probabilistic framework to make predictions and inherently favors smooth functions, where nearby points have similar output values.

The smoothness bias is particularly beneficial in problems where gradual changes in input are expected to correspond to gradual changes in output, such as in regression tasks or spatial modeling.

3. Bias towards Complex, Nonlinear Functions

Algorithms like **neural networks** have an inherent bias toward learning complex, nonlinear functions. This allows them to capture intricate patterns and dependencies within data, making them suitable for problems that involve high-dimensional and highly complex relationships, such as image recognition, natural language processing, and time-series forecasting.

- ❖ **Neural Networks:** Deep neural networks, especially those with multiple layers (deep learning), are designed to capture hierarchical patterns by combining non-linear transformations. This enables them to represent complex, nonlinear functions and learn subtle patterns in large datasets.

While this ability to learn complex patterns can be powerful, it also increases the risk of **overfitting** if not properly controlled. Regularization techniques, such as dropout or weight decay, are often employed to manage this bias, helping neural networks to generalize better by preventing them from memorizing the training data.

4. Bias towards Sparsity

Some algorithms have a **sparsity bias**, favoring solutions that only use a subset of the available features. This bias leads to models that are simpler and more interpretable, focusing only on the most relevant features. Sparsity can improve generalization by reducing the model's reliance on potentially noisy or redundant features.

- ❖ **Lasso Regression:** Lasso (Least Absolute Shrinkage and Selection Operator) regression introduces a penalty on the absolute value of coefficients, pushing some of them to zero. This effectively selects a subset of features, leading to a sparse model that is easier to interpret.

- ❖ **Feature Selection Methods:** Other algorithms, such as certain tree-based models and elastic net regression, also incorporate sparsity by selecting a limited number of features, which can be useful for high-dimensional data with many irrelevant or redundant attributes.

Sparsity bias is particularly valuable in domains where interpretability is important, such as medical or financial applications, and where focusing on fewer, relevant features can improve the model's performance on new data.

Three Learning Algorithms

Different algorithms come with different types of inductive biases, depending on their structure and approach to learning:

1. ROTE-LEARNER

- ❖ **Learning Approach:** The Rote-Learner algorithm is the simplest form of learning, where the algorithm memorizes each individual training example and retrieves it directly from memory when needed. When a new instance appears, the Rote-Learner checks if this instance is in its memory. If it is, the Rote-Learner returns the stored classification. If the instance is not in memory, the algorithm cannot classify it and simply declines to make a prediction.
- ❖ **Inductive Bias:** The Rote-Learner has no inductive bias because it does not make any assumptions beyond what is explicitly provided in the training data. This means it cannot generalize to unseen instances; it only recognizes instances it has memorized. While this approach avoids errors in known cases, it is ineffective for generalization.

2. CANDIDATE-ELIMINATION

- ❖ **Learning Approach:** The Candidate Elimination algorithm operates by maintaining a version space, which is the set of all hypotheses that remain consistent with the training data. As new instances are introduced, it refines this space by eliminating hypotheses that do not fit the observed examples. The algorithm only classifies a new instance if all remaining hypotheses in the version space agree on its classification. Otherwise, it declines to classify.
- ❖ **Inductive Bias:** The inductive bias in Candidate Elimination is that the target concept can be represented within its hypothesis space as a conjunction of attribute values. This assumption enables the algorithm to systematically exclude inconsistent hypotheses, eventually converging on a version space that contains hypotheses aligning with the target concept. However, if the target concept cannot be expressed in the hypothesis space, the algorithm may fail to find an appropriate classification.

FIND-S Algorithm

- ❖ **Learning Approach:** The FIND-S algorithm identifies the most specific hypothesis that aligns with all positive training examples. Starting with an overly specific hypothesis (that only matches the first positive example exactly), it gradually generalizes this hypothesis to cover additional positive instances while ignoring negative examples.
- ❖ **Inductive Bias:** FIND-S assumes that the target concept is representable within its hypothesis space as a conjunction of attributes and that every instance labeled as positive is indeed an example of the target concept. This bias means that FIND-S is inclined toward specific hypotheses that fit all positive examples precisely while excluding any that match negative examples. However, it is limited by its reliance on the positive examples alone, which can sometimes lead to overgeneralization if the training set is not representative.

Summary

Each type of inductive bias serves a unique purpose, and the choice of bias often depends on the nature of the problem:

- ❖ Simplicity bias helps prevent overfitting by favoring straightforward models that are easier to generalize.
- ❖ Smoothness bias is ideal for problems where slight changes in input should lead to slight changes in output, providing smoother decision boundaries.
- ❖ Complex, nonlinear function bias allows algorithms to model sophisticated relationships but requires careful regularization to avoid overfitting.
- ❖ Sparsity bias enhances model interpretability and generalization by selecting only the most critical features, especially useful in high-dimensional data.

The type of inductive bias embedded in an algorithm directly influences how it learns and generalizes, helping it navigate the balance between fitting the training data well and performing effectively on new data. Selecting the right inductive bias for a given task is crucial to building an effective machine learning model.

1.7 Decision Tree Learning

- ❖ Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.

- ❖ It is a tree-structured classifier, where **internal nodes** represent the features of a dataset, **branches** represent the decision rules and each **leaf node** represents the outcome.
- ❖ In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- ❖ The decisions or the test are performed on the basis of features of the given dataset.
- ❖ It is a **graphical representation for getting all the possible solutions to a problem/decision based on given conditions**.
- ❖ It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- ❖ In order to build a tree, we use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.
- ❖ A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- ❖ Below diagram explains the general structure of a decision tree:

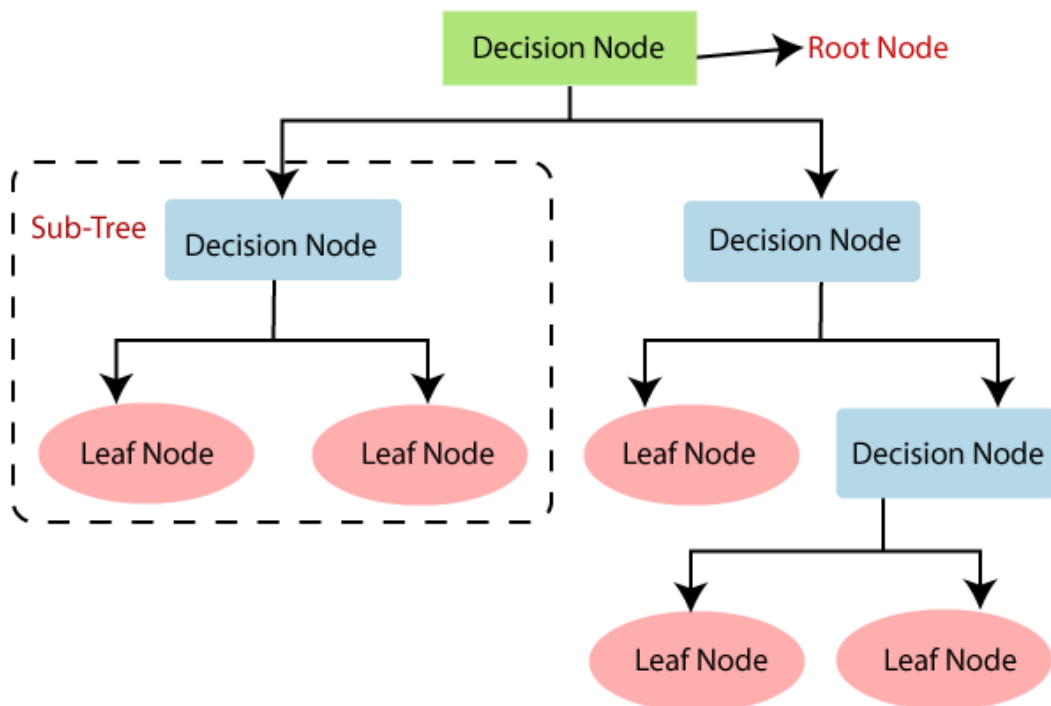


Fig. 1.5 Structure of decision tree.

Why use Decision Trees?

There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

- ❖ Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- ❖ The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

Root Node: Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.

Leaf Node: Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

Splitting: Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

Branch/Sub Tree: A tree formed by splitting the tree.

Pruning: Pruning is the process of removing the unwanted branches from the tree.

Parent/Child node: The root node of the tree is called the parent node, and other nodes are called the child nodes.

How does the Decision Tree algorithm Work?

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- ❖ **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- ❖ **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
- ❖ **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- ❖ **Step-4:** Generate the decision tree node, which contains the best attribute.
- ❖ **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:

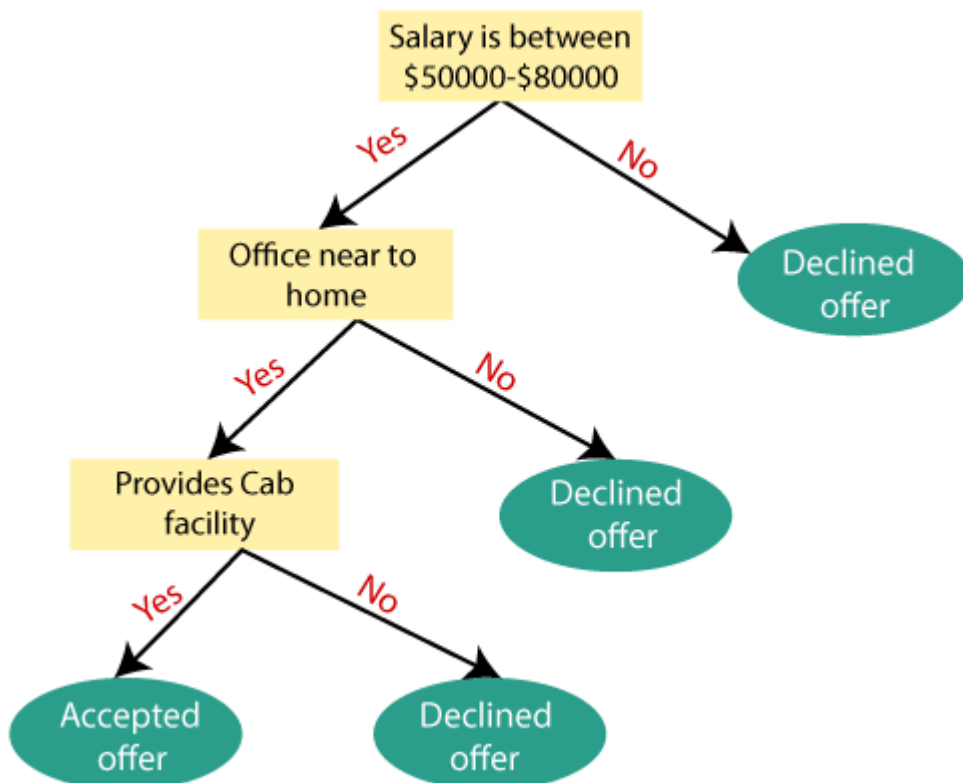


Fig. 1.6 Example of decision tree.

Attribute Selection Measures

While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM**. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:

- ✚ **Information Gain**
- ✚ **Gini Index**

1. Information Gain:

- ❖ Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- ❖ It calculates how much information a feature provides us about a class.
- ❖ According to the value of information gain, we split the node and build the decision tree.
- ❖ A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)]

Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$\text{Entropy}(s) = -P(\text{yes})\log_2 P(\text{yes}) - P(\text{no})\log_2 P(\text{no})$$

Where,

- ❖ **S= Total number of samples**
- ❖ **P(yes)= probability of yes**
- ❖ **P(no)= probability of no**

2. Gini Index:

- ❖ Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- ❖ An attribute with the low Gini index should be preferred as compared to the high Gini index.
- ❖ It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.

- ❖ Gini index can be calculated using the below formula:

Pruning: Getting an Optimal Decision tree

Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.

A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset. Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning. There are mainly two types of tree **pruning** technology used:

- ❖ **Cost Complexity Pruning**
- ❖ **Reduced Error Pruning.**

Advantages of the Decision Tree

- ❖ It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- ❖ It can be very useful for solving decision-related problems.
- ❖ It helps to think about all the possible outcomes for a problem.
- ❖ There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- ❖ The decision tree contains lots of layers, which makes it complex.
- ❖ It may have an overfitting issue, which can be resolved using the **Random Forest algorithm**.
- ❖ For more class labels, the computational complexity of the decision tree may increase.

Python Implementation of Decision Tree

Now we will implement the Decision tree using Python. For this, we will use the dataset "user_data.csv," which we have used in previous classification models. By using the same dataset, we can compare the Decision tree classifier with other classification models such as KNN SVM, LogisticRegression, etc.

Steps will also remain the same, which are given below:

- **Data Pre-processing step**
- **Fitting a Decision-Tree algorithm to the Training set**

- Predicting the test result
- Test accuracy of the result(Creation of Confusion matrix)
- Visualizing the test set result.

1. Data Pre-Processing Step:

Below is the code for the pre-processing step:

```
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd

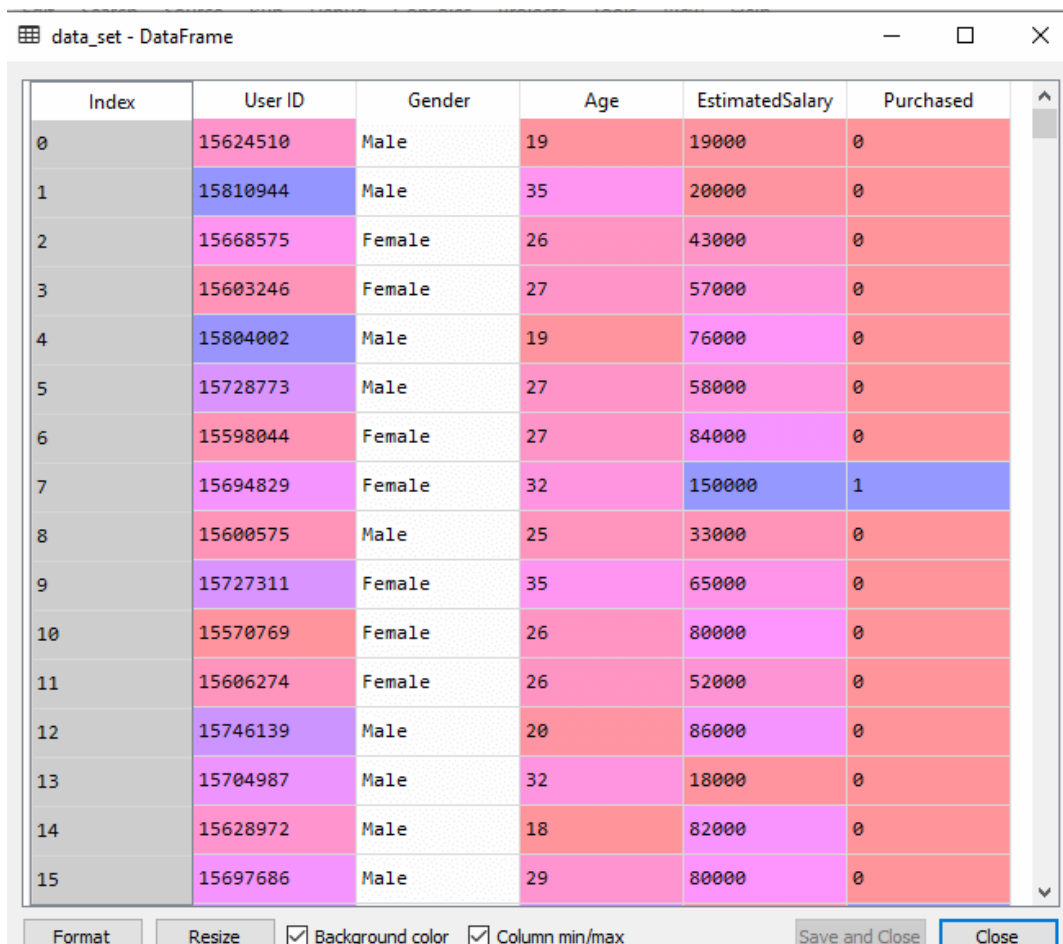
#importing datasets
data_set= pd.read_csv('user_data.csv')

#Extracting Independent and dependent Variable
x= data_set.iloc[:, [2,3]].values
y= data_set.iloc[:, 4].values

# Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)

#feature Scaling
from sklearn.preprocessing import StandardScaler
st_x= StandardScaler()
x_train= st_x.fit_transform(x_train)
x_test= st_x.transform(x_test)
```

In the above code, we have pre-processed the data. Where we have loaded the dataset, which is given as:



Index	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
10	15570769	Female	26	80000	0
11	15606274	Female	26	52000	0
12	15746139	Male	20	86000	0
13	15704987	Male	32	18000	0
14	15628972	Male	18	82000	0
15	15697686	Male	29	80000	0

Fig. 1.7 Data Pre-Processing Step output.

2. Fitting a Decision-Tree algorithm to the Training set

Now we will fit the model to the training set. For this, we will import the **DecisionTreeClassifier** class from **sklearn.tree** library. Below is the code for it:

```
#Fitting Decision Tree classifier to the training set
From sklearn.tree import DecisionTreeClassifier
classifier= DecisionTreeClassifier(criterion='entropy', random_state=0)
classifier.fit(x_train, y_train)
```

In the above code, we have created a classifier object, in which we have passed two main parameters;

- ❖ **"criterion='entropy'":** Criterion is used to measure the quality of split, which is calculated by information gain given by entropy.
- ❖ **random_state=0":** For generating the random states.

Below is the output for this:

Out[8]:

```
DecisionTreeClassifier(class_weight=None, criterion='entropy', max_depth=None,
max_features=None, max_leaf_nodes=None,
min_impurity_decrease=0.0, min_impurity_split=None,
min_samples_leaf=1, min_samples_split=2,
min_weight_fraction_leaf=0.0, presort=False,
random_state=0, splitter='best')
```

3. Predicting the test result

Now we will predict the test set result. We will create a new prediction vector **y_pred**. Below is the code for it:

```
#Predicting the test set result
y_pred= classifier.predict(x_test)
```

Output:

In the below output image, the predicted output and real test output are given. We can clearly see that there are some values in the prediction vector, which are different from the real vector values. These are prediction errors.

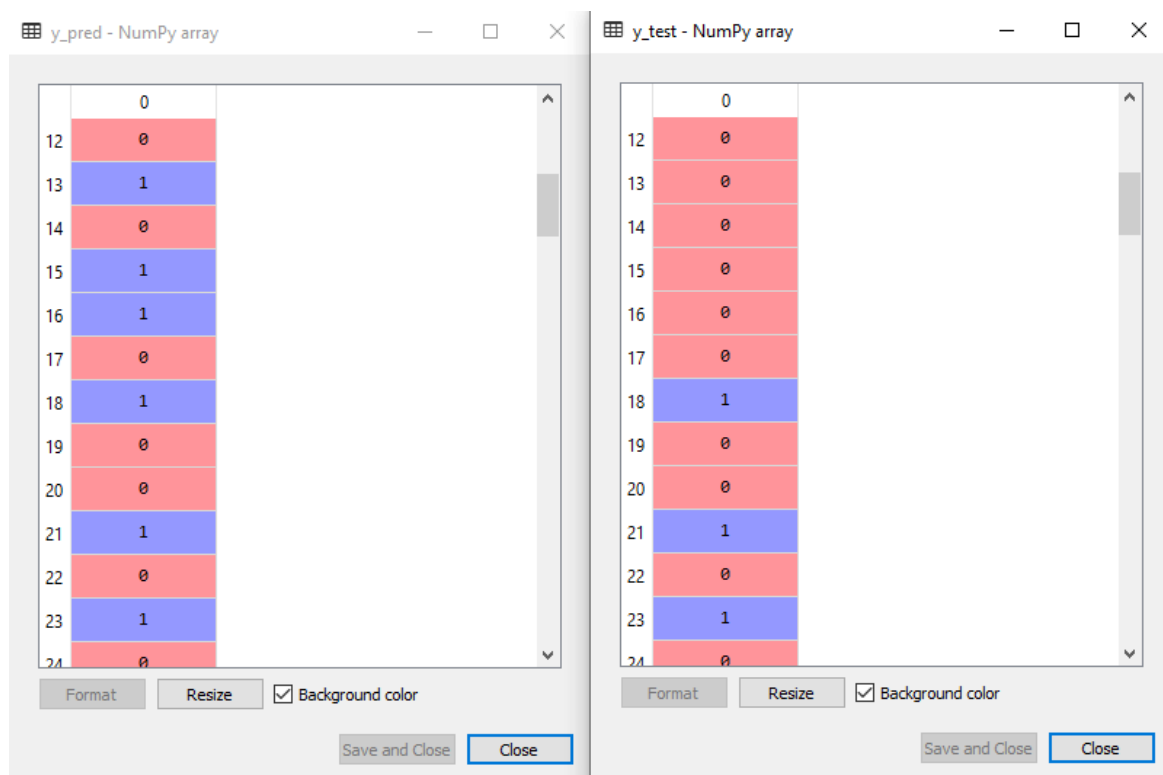


Fig. 1.8 Predicting the test result.

4. Test accuracy of the result (Creation of Confusion matrix)

In the above output, we have seen that there were some incorrect predictions, so if we want to know the number of correct and incorrect predictions, we need to use the confusion matrix. Below is the code for it:

```
#Creating the Confusion matrix
from sklearn.metrics import confusion_matrix
cm= confusion_matrix(y_test, y_pred)
```

Output:

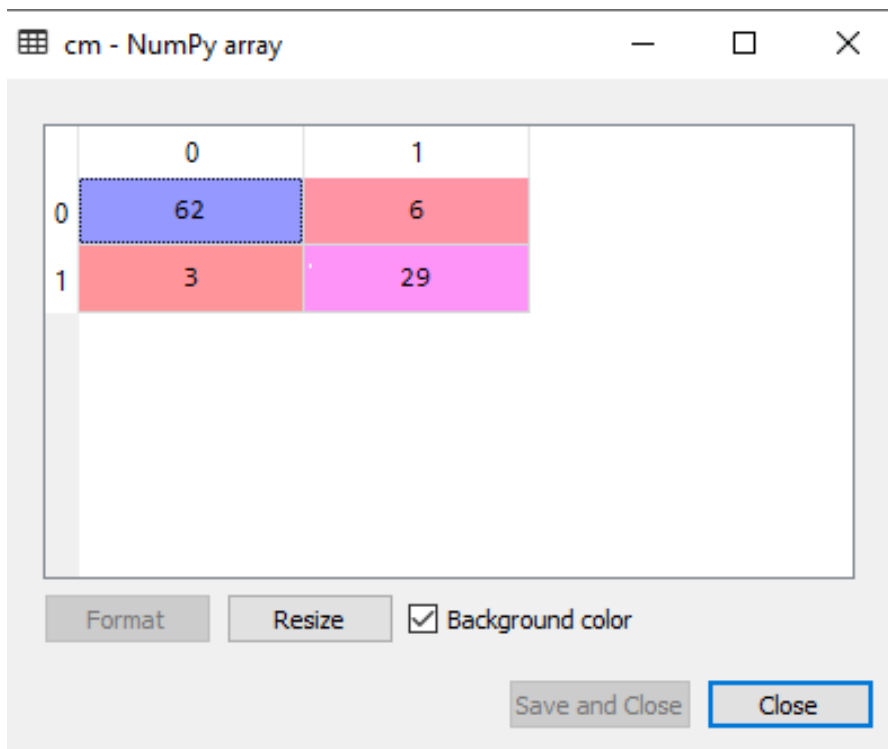


Fig. 1.9 Test accuracy of the result.

In the above output image, we can see the confusion matrix, which has $6+3=9$ incorrect predictions and $62+29=91$ correct predictions. Therefore, we can say that compared to other classification models, the Decision Tree classifier made a good prediction.

5. Visualizing the training set result:

Here we will visualize the training set result. To visualize the training set result we will plot a graph for the decision tree classifier. The classifier will predict yes or No for the users who have either Purchased or Not purchased the SUV car as we did in Logistic Regression. Below is the code for it:

```
#Visulaizing the trianing set result
from matplotlib.colors import ListedColormap
x_set, y_set = x_train, y_train
x1, x2 = nm.meshgrid(nm.arange(start = x_set[:, 0].min() -
1, stop = x_set[:, 0].max() + 1, step =0.01),
nm.arange(start = x_set[:, 1].min() - 1, stop = x_set[:, 1].max() + 1, step = 0.01))
mtp.contourf(x1, x2, classifier.predict(nm.array([x1.ravel(), x2.ravel()]).T).reshape(x1.s
hape),
```

```
alpha = 0.75, cmap = ListedColormap(('purple','green' )))
mtp.xlim(x1.min(), x1.max())
mtp.ylim(x2.min(), x2.max())
fori, j in enumerate(nm.unique(y_set)):
mtp.scatter(x_set[y_set == j, 0], x_set[y_set == j, 1],
            c = ListedColormap(('purple', 'green'))(i), label = j)
mtp.title('Decision Tree Algorithm (Training set)')
mtp.xlabel('Age')
mtp.ylabel('Estimated Salary')
mtp.legend()
mtp.show()
```

Output:

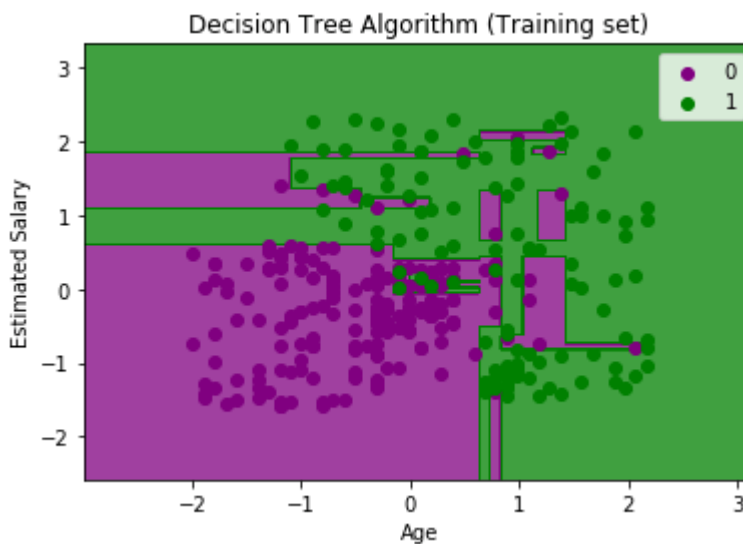


Fig. 1.10 Visualizing the training set result output.

The above output is completely different from the rest classification models. It has both vertical and horizontal lines that are splitting the dataset according to the age and estimated salary variable.

As we can see, the tree is trying to capture each dataset, which is the case of overfitting.

6. Visualizing the test set result:

Visualization of test set result will be similar to the visualization of the training set except that the training set will be replaced with the test set.

```
#Visulaizing the test set result
from matplotlib.colors import ListedColormap
x_set, y_set = x_test, y_test
x1, x2 = nm.meshgrid(nm.arange(start = x_set[:, 0].min() -
1, stop = x_set[:, 0].max() + 1, step = 0.01),
nm.arange(start = x_set[:, 1].min() - 1, stop = x_set[:, 1].max() + 1, step = 0.01))
mtp.contourf(x1, x2, classifier.predict(nm.array([x1.ravel(), x2.ravel()]).T).reshape(x1.s
hape),
alpha = 0.75, cmap = ListedColormap(('purple','green' )))
mtp.xlim(x1.min(), x1.max())
mtp.ylim(x2.min(), x2.max())
fori, j in enumerate(nm.unique(y_set)):
mtp.scatter(x_set[y_set == j, 0], x_set[y_set == j, 1],
c = ListedColormap(('purple', 'green'))(i), label = j)
mtp.title('Decision Tree Algorithm(Test set)')
mtp.xlabel('Age')
mtp.ylabel('Estimated Salary')
mtp.legend()
mtp.show()
```

Output:

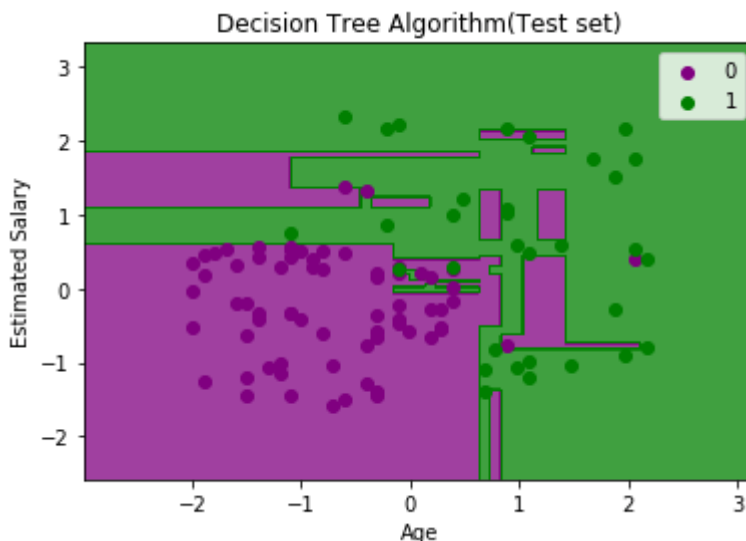
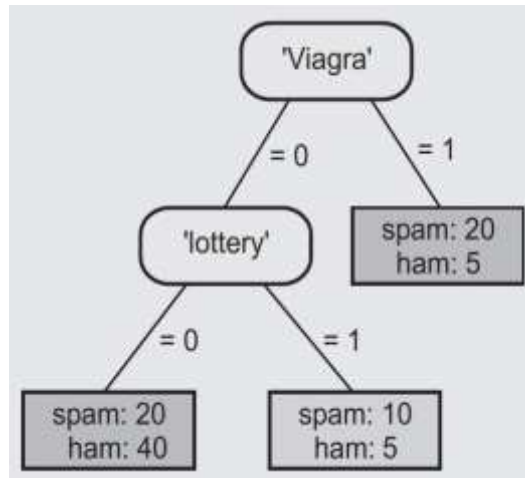


Fig .1.11 Visualizing the test set result output.

Mastering machine learning with Python practical with tips and technique

As we can see in the above image that there are some green data points within the purple region and vice versa. So, these are the incorrect predictions which we have discussed in the confusion matrix.

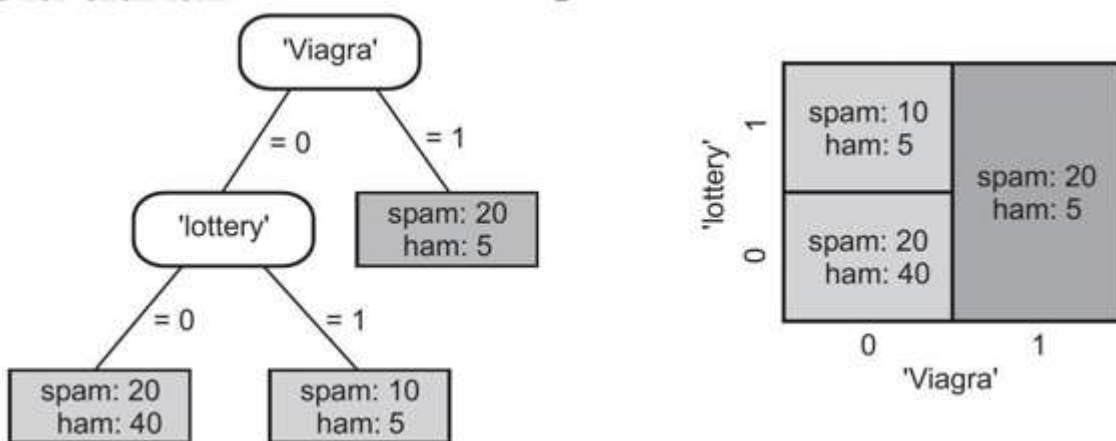
Example 1.7.1 Using following feature tree, write decision rules for majority class.



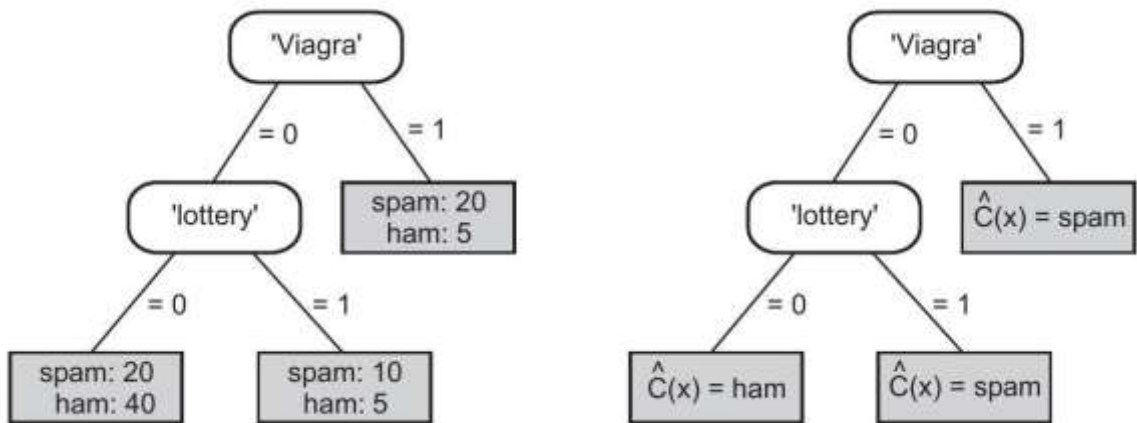
Solution:

Left Side: A feature tree combining two Boolean features. Each internal node or split is labelled with a feature, and each edge emanating from a split is labelled with a feature value. Each leaf therefore corresponds to a unique combination of feature values. Also indicated in each leaf is the class distribution derived from the training set.

Right Side: A feature tree partitions the instance space into rectangular regions, one for each leaf.



✚ The leaves of the tree in the above figure could be labelled, from left to right, as ham - spam - spam, employing a simple decision rule called majority class.



- ✚ Left side : A feature tree with training set class distribution in the leaves.
- ✚ Right side : A decision tree obtained using the majority class decision rule.

Example 1.7.2

Suppose we want to train a decision tree using the following instances:

Weekend (Example)	Weather	Parents	Money	Decision (Category)
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay in
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Solution:

$$\begin{aligned}\text{Entropy}(S) &= -p_{\text{cinema}} \log_2(p_{\text{cinema}}) - p_{\text{tennis}} \log_2(p_{\text{tennis}}) \\ &\quad - p_{\text{shopping}} \log_2(p_{\text{shopping}}) - p_{\text{stay_in}} \log_2(p_{\text{stay_in}}) \\ &= -(6/10) * \log_2(6/10) - (2/10) * \log_2(2/10) - (1/10) * \log_2(1/10) - (1/10) * \log_2(1/10) \\ &= -(6/10) * -0.737 - (2/10) * -2.322 - (1/10) * -3.322 - (1/10) * -3.322 \\ &= 0.4422 + 0.4644 + 0.3322 + 0.3322 = 1.571\end{aligned}$$

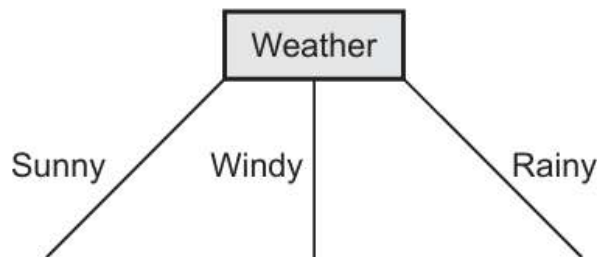
and we need to determine the best of :

$$\begin{aligned}\text{Gain}(S, \text{weather}) &= 1.571 - (|S_{\text{sun}}|/10) * \text{Entropy}(S_{\text{sun}}) - (|S_{\text{wind}}|/10) * \text{Entropy}(S_{\text{wind}}) \\ &\quad - (|S_{\text{rain}}|/10) * \text{Entropy}(S_{\text{rain}}) \\ &= 1.571 - (0.3) * \text{Entropy}(S_{\text{sun}}) - (0.4) * \text{Entropy}(S_{\text{wind}}) - (0.3) * \text{Entropy}(S_{\text{rain}}) \\ &= 1.571 - (0.3) * (0.918) - (0.4) * (0.81125) - (0.3) * (0.918) = 0.70\end{aligned}$$

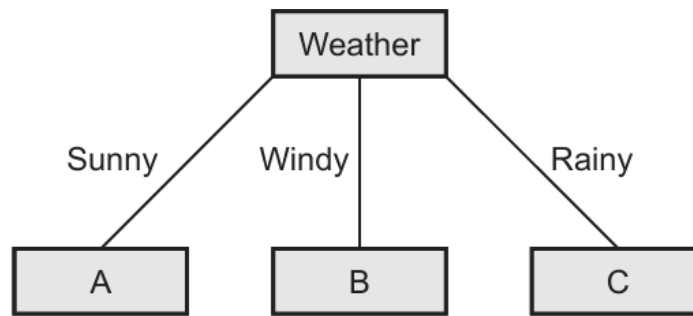
$$\begin{aligned}\text{Gain}(S, \text{parents}) &= 1.571 - (|S_{\text{yes}}|/10) * \text{Entropy}(S_{\text{yes}}) - (|S_{\text{no}}|/10) * \text{Entropy}(S_{\text{no}}) \\ &= 1.571 - (0.5) * 0 - (0.5) * 1.922 = 1.571 - 0.961 = 0.61\end{aligned}$$

$$\begin{aligned}\text{Gain}(S, \text{money}) &= 1.571 - (|S_{\text{rich}}|/10) * \text{Entropy}(S_{\text{rich}}) - (|S_{\text{poor}}|/10) * \text{Entropy}(S_{\text{poor}}) \\ &= 1.571 - (0.7) * (1.842) - (0.3) * 0 = 1.571 - 1.2894 = 0.2816\end{aligned}$$

- ❖ This means that the first node in the decision tree will be the weather attribute. From the weather node, we draw a branch for the values that weather can take: sunny, windy and rainy.



- ❖ Now we look at the first branch.
 $S_{\text{sunny}} = \{W1, W2, W10\}$. This is not empty, so we do not put a default categorization leaf node here.
- ❖ The categorizations of W1, W2 and W10 are Cinema, Tennis respectively. As these are not all the same, we cannot put a categorization leaf node here. Hence we put an attribute node here, which we will leave blank for the time being.



- ❖ Looking at the second branch, $S_{\text{windy}} = \{W3, W7, W8, W9\}$. Again, this is not empty, and they do not all belong to the same class, so we put an attribute node here, left blank for now. The same situation happens with the third branch, hence our amended tree looks like this :
- ❖ In effect, we are interested only in this part of the table :

Weekend (Example)	Weather	Parents	Money	Decision (Category)
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W10	Sunny	No	Rich	Tennis

Hence we can calculate:

$$\begin{aligned} \text{Gain}(S_{\text{sunny}}, \text{parents}) &= 0.918 - (|S_{\text{yes}}|/|S|) * \text{Entropy}(S_{\text{yes}}) - (|S_{\text{no}}|/|S|) * \text{Entropy}(S_{\text{no}}) \\ &= 0.918 - (1/3) * 0 - (2/3) * 0 = 0.918 \end{aligned}$$

$$\begin{aligned} \text{Gain}(S_{\text{sunny}}, \text{money}) &= 0.918 - \left(\frac{|S_{\text{rich}}|}{|S|} \right) * \text{Entropy}(S_{\text{rich}}) - \left(\frac{|S_{\text{poor}}|}{|S|} \right) * \text{Entropy}(S_{\text{poor}}) \\ &= 0.918 - (3/3) * 0.918 - (0/3) * 0 \\ &= 0.918 - (3/3) * 0.918 - (0/3) * 0 = 0.918 - 0.918 = 0 \end{aligned}$$

Challenges in Decision Tree Learning

When constructing decision trees, several practical challenges arise that affect the performance and accuracy of the model. Key considerations include deciding the appropriate depth for the tree, handling both continuous and categorical attributes, choosing suitable attribute selection measures, addressing incomplete data or missing attribute values, managing attributes with

differing costs, and ensuring efficient computation. Each of these factors plays a role in determining how well the decision tree will perform in generalizing to unseen data.

Avoiding Overfitting in Decision Trees

Overfitting is a major issue in decision tree learning and in machine learning generally. It occurs when a model captures not only the true patterns in the training data but also the noise and random fluctuations, making it excessively complex. This results in a model that performs well on training data but poorly on new data, as it fails to generalize effectively. Overfitting is common when the model has too many parameters relative to the number of observations.

Indicators of Overfitting

To assess whether a model is overfitting, we can compare its prediction error on the training data versus a separate evaluation set. A balanced model has a reasonable fit to both sets, while an overfit model shows low error on the training data but high error on evaluation data, indicating that it has memorized the training examples rather than learned the underlying patterns.

Causes of Overfitting

Common causes of overfitting include:

1. **Noisy Data:** Errors or irregularities in data can mislead the model into learning irrelevant patterns.
2. **Small Training Set:** With insufficient data, the model may form complex patterns to explain a limited number of examples.
3. **High Dimensionality:** When there are too many features, the model may find spurious correlations, leading to over-complexity.

Techniques for Preventing Overfitting

To reduce the risk of overfitting, several strategies can be implemented:

1. **Limit Adjustable Parameters:** For instance, in neural networks, the number of hidden units or weights can be restricted to reduce model complexity. This approach applies to decision trees as well, where limiting the depth of the tree or the number of splits can help prevent overfitting.
2. **Early Stopping:** Training can be halted before the model fully fits the training data, leaving it slightly underfit but more likely to generalize well.

3. **Regularization:** Adding a regularization term to the loss function discourages complex models by penalizing high weights or intricate decision boundaries, promoting smoother, simpler functions.
4. **Data Augmentation or Noise Addition:** Adding slight variations or noise to the training data can make the model less sensitive to minor fluctuations and reduce overfitting.

In neural networks, additional heuristics like weight decay (penalizing larger weights) and simplified network structures are often used to control overfitting. In decision trees, pruning techniques are commonly employed.

Methods for Overfitting Control in Decision Trees

There are two main approaches to controlling overfitting in decision tree learning:

1. **Pre-Pruning:** Stop growing the tree before it becomes overly complex. This approach limits the depth of the tree or applies a minimum threshold for information gain to prevent further splits that add complexity without significant predictive improvement.
2. **Post-Pruning:** Allow the tree to grow fully, capturing all patterns in the training data, and then prune it back by removing branches that do not contribute meaningfully to performance on a validation set. This approach allows for a thorough examination of the tree structure and removes only the parts that may lead to overfitting.

Specific Pruning Techniques

Reduced-Error Pruning

1. **Data Splitting:** Divide the dataset into training and validation sets.
2. **Pruning Process:** For each node in the tree, assess whether removing the node improves accuracy on the validation set. Nodes are pruned if this removal does not reduce accuracy.
3. **Result:** This method produces a simpler tree that maintains predictive accuracy by avoiding overfitting to the training data's nuances.

Rule Post-Pruning

1. **Convert Tree to Rules:** Transform each branch of the decision tree into a rule based on the path from the root to a leaf.

2. **Prune Rules:** Simplify each rule by removing conditions that do not significantly impact accuracy, resulting in a set of rules that are easier to interpret and generalize better.
3. **Sort and Apply:** Order the pruned rules by their estimated accuracy and apply them in this sequence when making predictions on new instances.

Advantages of Converting to Rules

- ❖ **Contextual Clarity:** Rules allow better differentiation of contexts, as each rule operates independently and captures specific conditions.
- ❖ **Focus on Key Attributes:** Rule-based pruning clarifies the importance of different attributes by eliminating less relevant conditions.
- ❖ **Improved Interpretability:** Simplified rules are often easier for users to understand and interpret compared to complex tree structures.

Estimating and Improving Accuracy in Decision Trees

To assess and improve the accuracy of decision trees, certain metrics and statistical methods are used:

1. **Calculate Rule Accuracy:** Evaluate the accuracy of each rule on the training data.
2. **Use Statistical Bounds:** Compute the standard deviation assuming a binomial distribution and establish a confidence interval to understand the likely range of the model's performance.
3. **Adjust for Large Datasets:** For extensive datasets, the observed rule accuracy closely approximates the actual performance. However, adjustments for sampling variability can still provide a more robust accuracy estimate.
4. **Practical Value of Estimates:** While not always statistically exact, these estimates often provide practical insights and are widely used in evaluating and fine-tuning decision tree models.

1.8 Heuristic Search

Heuristic search is an algorithmic approach used to find effective solutions to complex problems by employing rules of thumb, or heuristics, rather than exhaustive calculations. In many problem-solving contexts, especially within artificial intelligence, heuristic methods are favored for their speed and efficiency in reaching near-optimal solutions without the need to explore every possible outcome. This approach leverages mathematical principles, insights from computer science, and techniques from artificial intelligence to rapidly identify solutions that satisfy given criteria, making it valuable for tackling a wide range of challenges.

Heuristic Search Working

Heuristics are simple, experience-based rules or strategies that provide a framework for decision-making and problem-solving. Rather than relying solely on systematic calculations, heuristics allow algorithms to make educated guesses, focusing on the most promising paths toward a solution. This can save substantial computational resources by bypassing exhaustive search methods, such as brute force or complete enumeration, which require evaluating every possible option. By allowing users to set specific parameters or preferences, heuristic search methods can be tailored to prioritize certain outcomes, making the process both flexible and adaptable to different types of problems.

Advantages of Heuristic Search

One of the primary benefits of heuristic search is its **ability to navigate large problem spaces efficiently**. This flexibility enables it to handle a wide variety of tasks, including optimization, game strategy, scheduling, routing, planning, and constraint satisfaction problems. Compared to other search techniques, heuristic methods often have lower computational complexity, which means they can operate effectively with limited resources.

Consequently, heuristic search has become an essential tool for researchers and practitioners in fields that require fast, scalable solutions to intricate problems.

Key Concepts in Heuristic Search

Heuristic search methods utilize heuristic functions—mathematical approximations or estimates that help guide the search process. A heuristic function evaluates potential solutions based on specific criteria, such as cost, time efficiency, or proximity to the desired outcome. By focusing on these promising regions of the search space, heuristic search reduces the computational effort needed to arrive at a viable solution. Heuristic search can be implemented using various strategies, each with its advantages depending on the problem's structure. Common search techniques include:

1. **Best-First Search:** This algorithm prioritizes exploring nodes or states that seem closer to the goal, selecting the path that appears to lead most directly to a solution. Best-first search is often used when the search space is vast, as it aims to minimize exploration time by focusing on paths with the least cost.
2. **Depth-First Search (DFS):** DFS dives deep into one path before backtracking, making it useful for problems where solutions are likely to be found at deeper levels of the search tree. While it can be efficient, it risks getting trapped in one path if the search space is large.

3. **Breadth-First Search (BFS):** BFS explores all nodes at one level of the tree before moving to the next, ensuring that it finds the shortest path to a solution in an unweighted graph. However, BFS can be slow and computationally intensive for large search spaces.
4. **Hill Climbing:** This strategy incrementally adjusts a solution to improve it, like climbing up a hill step-by-step, aiming for a local maximum. It's a straightforward approach but can get stuck at local maxima, preventing it from finding the best overall solution.
5. **Simulated Annealing:** Inspired by the process of annealing in metallurgy, this algorithm uses randomness to explore different solutions. It helps avoid local maxima by allowing occasional "downhill" moves, increasing the likelihood of finding a global optimum.

In complex scenarios, multiple heuristic approaches may be combined. For example, if one strategy yields only partial success, another can be applied to continue exploring the search space. Choosing the right mix of heuristics depends on understanding the computational costs and accuracy trade-offs associated with each method.

Example of Heuristic Search in Practice

Heuristic search is commonly applied to problems in computer science, such as the **Travelling Salesman Problem (TSP)**, where the goal is to find the shortest possible route that visits each city once and returns to the starting point. Heuristic algorithms like **Best-First Search** and **Hill Climbing** are frequently used in TSP to identify efficient solutions:

- ❖ **Best-First Search (BFS)** moves step-by-step from an initial city, exploring paths that seem closest to the goal and selecting routes with the lowest associated costs.
- ❖ **Hill Climbing** adjusts routes incrementally to reduce the total travel distance, akin to taking successive steps up a slope toward a solution.

These algorithms allow TSP and similar optimization problems to be solved faster than traditional methods, though they may not guarantee the optimal path.

Heuristic Approaches in Artificial Intelligence (AI)

In AI, heuristic methods help machines make decisions based on approximate, rather than exact, solutions. Heuristic search is crucial in AI applications like game-playing and robotics, where computational efficiency and decision-making speed are essential.

Some popular heuristic search methods in AI include:

1. **Breadth-First Search:** Quick for finding solutions but may not always yield optimal results.
2. **Depth-First Search:** Thorough in exploration but can be slow for large search spaces.
3. **Iterative Deepening Search:** Combines the advantages of both breadth-first and depth-first search.
4. **Memory-Restricted Search:** Focuses on reducing memory usage, which is useful for handling large datasets.
5. **Parallel Search:** Distributes tasks across multiple processors, accelerating the search process.

Each type has its own strengths and weaknesses, and the choice of search method depends on the specific requirements of the task, such as computational limits and the need for accuracy.

Heuristic Techniques in AI Problem-Solving

Heuristic techniques are integral to problem-solving and decision-making in AI. By leveraging evaluation functions, priority queues, and heuristic functions, these techniques guide AI systems through complex tasks. For instance:

- ❖ **Genetic Algorithms:** Used in optimization, genetic algorithms select "good" solutions from a pool and refine them over multiple generations. This iterative process resembles natural selection, where the best solutions are progressively improved.
- ❖ **Shortest Path Algorithms:** When finding the best route, such as in navigation systems, heuristic methods evaluate paths based on criteria like distance and speed. This helps identify the most efficient route without examining every possible path.

Heuristic techniques provide AI systems with the ability to quickly focus on promising solutions, enhancing efficiency and reducing computational demands. By narrowing the scope of the search, these techniques avoid the exhaustive analysis typical of traditional methods, enabling faster and more accurate decision-making.

Three Main Types of Heuristics in Search

Heuristic search methods can be categorized into three primary types:

1. **External Explicit Graph Search:** This approach explores nodes in a sequential manner, evaluating each based on costs or utility values. It is suitable for problems with a manageable number of possible solutions, as it simplifies the search space by breaking it into smaller parts.

2. **Parallel External Search:** In this approach, the core data is divided across multiple processors or computers, which work simultaneously. By processing nodes in parallel, this method significantly reduces search time, making it ideal for large datasets.
3. **Search Value Iteration:** A hybrid of graph and parallel search methods, search value iteration starts with an initial node and iterates through its associated costs and utilities across connected nodes. This approach enables efficient exploration of large spaces, leveraging resources to obtain results faster than sequential evaluations.

Benefits of Heuristic Search

Heuristic search provides a powerful, efficient way to explore large search spaces without exhaustive computations. By focusing only on promising solutions and applying smart decision-making strategies, it cuts down processing time and allows for scalable problem-solving. Heuristic methods have become a valuable asset for researchers and developers looking to tackle complex challenges with limited computational resources.

Neural Networks and Genetic Algorithms

2.1 Introduction to Artificial Neural Networks (ANN)

Artificial Neural Networks (ANN) are computational models inspired by the biological structure and function of the human brain. They are designed to simulate the way the brain processes information, learns, and adapts. Essentially, an ANN is a network composed of interconnected artificial neurons or nodes, structured according to specific network architectures, each tailored for a particular purpose. The primary goal of an ANN is to interpret inputs and produce meaningful outputs, learning to make predictions or classifications based on patterns in the input data.

2.1.1 Key Characteristics of Artificial Neural Networks

1. **Parallel Processing Capability:** Unlike conventional computational systems that follow programmed instructions, ANNs operate in parallel, responding to the entire pattern of inputs they receive. This enables them to process information much faster and handle complex, non-linear relationships.
2. **Memory Representation through Connections:** ANNs do not have separate memory storage units. Instead, the 'knowledge' they acquire is distributed throughout the network in the form of connection weights. These weights are adjusted during training, enabling the network to store and recall patterns effectively.
3. **Non-Deterministic Learning:** ANNs rely on a learning algorithm rather than explicit programming. They learn by adjusting connection weights through methods like supervised learning (learning from labeled examples) or unsupervised learning (discovering patterns in unlabeled data), allowing them to adapt to new data.

2.1.2 Structure of an Artificial Neural Network

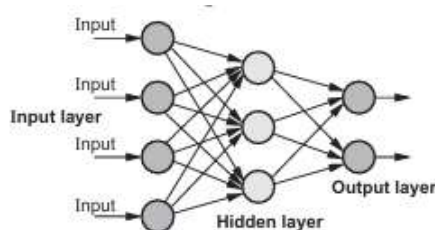


Fig. 2.1 Artificial neural network structure.

As shown in **Fig. 2.1**, an ANN typically consists of three main layers:

- **Input Layer:** Receives the input data and passes it on to the next layer.
- **Hidden Layer(s):** One or more layers where the network processes the input data and extracts patterns or features. These layers help to increase the network's complexity and its ability to capture non-linear relationships.
- **Output Layer:** Produces the final prediction or classification based on the learned patterns.

The elements of an ANN include:

- **Processing Units (Neurons):** Nodes that process inputs and apply activation functions to introduce non-linearity.
- **Topology:** The architecture or arrangement of neurons and layers in the network.
- **Learning Algorithm:** The algorithm that adjusts the weights to minimize errors and improve accuracy.

2.1.3 Tasks Solved by Artificial Neural Networks

ANNs are versatile and can be applied to a variety of tasks, including:

1. **Robot Motion Control:** Controlling robot movements based on sensory inputs and real-time perception of the environment.
2. **Object Categorization:** Classifying various objects, such as food items, in artificial or real-world scenarios.
3. **Visual Recognition:** Identifying objects or individuals in images or video streams.
4. **Prediction and Tracking:** Anticipating the movement trajectory of objects, enabling robots to catch or avoid them.

2.1.4 Additional Applications of ANNs

- **Natural Language Processing (NLP):** Understanding and generating human language in applications like chatbots, translators, and voice assistants.
- **Financial Forecasting:** Predicting stock market trends, currency prices, and other financial metrics.
- **Medical Diagnosis:** Assisting in diagnosing diseases from medical images, like MRIs and X-rays, or data, such as EKGs.
- **Autonomous Vehicles:** Supporting self-driving cars in recognizing objects, interpreting road signs, and making navigation decisions.

2.1.1 Biological Motivation

Artificial neural networks are inspired by the structure and functioning of biological neural systems found in living organisms. The basic unit of biological neural systems is the **neuron**, a specialized cell that transmits information through biochemical and electrical processes. In the human brain alone, there are approximately 10 billion interconnected neurons that work together to facilitate thought, perception, movement, and many other functions. Each neuron acts as a processing unit, receiving, interpreting, and transmitting signals through its network.

2.1.1.1 Structure of a Biological Neuron

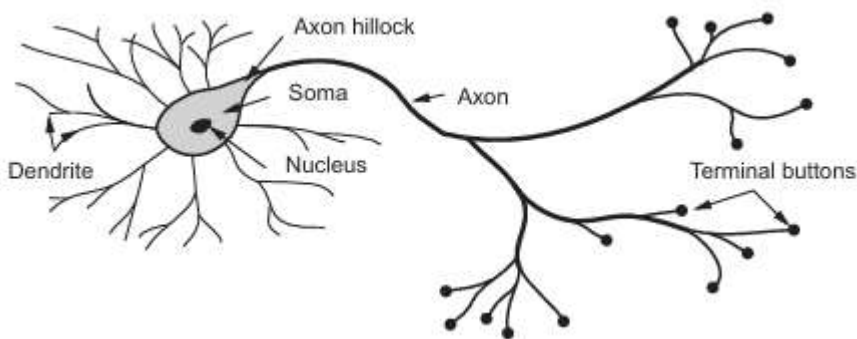


Fig. 2.2 Structure of biological neuron.

As depicted in **Fig. 2.2**, a biological neuron consists of several key components:

1. **Cell Body (Soma):** This is the central part of the neuron, containing the nucleus and other organelles. The soma is responsible for maintaining the cell and processing the information received from dendrites.
2. **Nucleus:** Located within the soma, the nucleus holds the cell's genetic material (DNA) and controls its growth and repair. Although the presence of a nucleus is common in many types of cells, in neurons, it plays a vital role in overall cell function.
3. **Dendrites:** These are branch-like structures that extend from the soma. They receive incoming signals from other neurons. The connections between dendrites of one neuron and the axon of another are called **synapses**. At these junctions, chemical signals (neurotransmitters) facilitate the transmission of information.
4. **Axon:** The axon is a long, thin fiber that transmits signals away from the soma to other neurons, muscles, or glands. The **axon hillock** is a part of the neuron that integrates signals and initiates the electrical impulse if a certain threshold is met.
5. **Terminal Buttons:** Located at the ends of the axon, these structures release neurotransmitters into the synapse, allowing communication between neurons.

6. **Synapses:** Synapses are the junctions where communication between neurons occurs. Neurotransmitters diffuse across these gaps, carrying signals from the axon terminals of one neuron to the dendrites of another.

2.1.1.2 Functional Aspects of Biological Neurons

1. **Signal Encoding:** Most neurons transmit information through electrical pulses, also known as action potentials or spikes. These pulses serve as the primary method of communication within the neural network.
2. **Signal Processing:** The soma or cell body receives inputs from dendrites, integrates them, and determines whether the neuron should fire an action potential. This process is akin to data processing, where information is interpreted and an output is generated.
3. **Synaptic Transmission:** Information between neurons is transmitted at the synapse through the release of neurotransmitters. This chemical diffusion is essential for neural communication and for triggering responses in the receiving neuron.

2.1.1.3 Additional Information on Biological Neurons

- **Plasticity:** Biological neurons are known for their adaptability. Synaptic connections can strengthen or weaken over time, allowing neurons to "learn" from repeated experiences. This adaptability is crucial for memory formation and learning processes.
- **Neural Networks:** Groups of neurons are often organized into networks where they collaborate to carry out complex functions, such as visual processing, language understanding, and motor control. The collective interaction of these networks enables higher cognitive functions.

Comparison of biological NN and artificial NN as shown in Table. 2.1.

Biological NN	Artificial NN
Soma	Unit
Axon, Dendrite	Dendrite
Synapse	Weight
Potential	Weighted Sum
Threshold	Bias Weight
Signal	Activation

Table. 2.1 Comparison of biological and artificial NN.

2.2 Neural Network Representation

Neural Networks are computational models made up of numerous simple processing units, known as neurons, which are interconnected in a complex system. These networks are designed to solve intricate problems and learn from data in a manner that resembles the functioning of the human brain.

1. **Black Box Concept:** For users, a neural network (NN) often operates as a "black box." It takes in an **input vector** (source data) and provides an **output vector** (the resulting prediction or classification). The internal workings may be hidden, but the system's output is optimized to produce useful results.
2. **Network Structure:** Typically, a neural network is composed of three main layers:
 - **Input Layer:** Receives the input data.
 - **Hidden Layer(s):** One or more layers that process the input data to uncover patterns and relationships.
 - **Output Layer:** Produces the final result based on the processed information.
3. **Connections between Neurons:** Neurons in adjacent layers are often fully connected, meaning each neuron in one layer connects to every neuron in the next. The structure and architecture of these connections vary and are defined by the specific topology of the network and the chosen **activation function**.
4. **Weights and Activation Functions:** The strength of each connection between neurons is represented by a **weight**, a numerical value adjusted during training to optimize the network's performance. Activation functions determine whether a neuron's output will be transmitted to the next layer. These functions introduce non-linearity, enabling the network to learn and represent complex patterns.
5. **Learning through Weight Adjustment:** Training a neural network involves adjusting the weights to produce desired outputs for given inputs. This process is achieved through algorithms like back propagation, which minimizes error by fine-tuning weights, allowing the network to learn efficiently from data.
6. **Learning and Generalization:** A neural network learns from examples, similar to how humans learn. For instance, after being trained on numerous images of dogs, the network can generalize and recognize new images of dogs. This capability to generalize from past examples is crucial for a network's adaptability to new data.
7. **Parallel Processing:** Neural networks excel in parallel processing, where each neuron functions independently of others, allowing the network to handle complex computations more efficiently.

8. **Alternative Computing Paradigm:** Unlike traditional computing systems based on the Von Neumann architecture (which separate memory and processing units), neural networks operate on a parallel architecture. This approach is similar to the human brain, where processing and memory are integrated.
9. **Multiprocessor System:** Neural networks can be viewed as a form of multiprocessor system because:
 - Each neuron operates as a simple processing element.
 - There is a high degree of interconnection among neurons.
 - Adaptive interaction occurs as the network learns and adjusts.
 - The system supports flexible and adaptable learning, making it suitable for a variety of complex tasks.

2.2.1 Additional Characteristics and Advantages of Neural Networks

1. **Robustness and Fault Tolerance:** Due to their distributed nature, neural networks exhibit fault tolerance. If some neurons or connections fail, the network can still function effectively, which enhances its robustness.
2. **Dynamic Learning:** Neural networks can adapt to new data without complete retraining. This characteristic is beneficial in environments where data is constantly evolving.
3. **Applications Across Domains:** Neural networks have found applications in various fields, including:
 - **Image and Speech Recognition:** Used in facial recognition, voice assistants, and language translation.
 - **Medical Diagnostics:** Assists in analyzing medical images and identifying patterns that may indicate diseases.
 - **Autonomous Systems:** Vital for self-driving cars and robotics, where real-time data processing is essential.
 - **Financial Forecasting:** Applied in stock market prediction, risk assessment, and other financial analytics.
4. **Limitations:** Despite their advantages, neural networks can be computationally expensive and require a large amount of data to achieve high accuracy. Additionally, their "black box" nature makes interpretability challenging, leading to efforts in developing explainable AI techniques.

2.2.2 NN Architecture: Single Layer Network

- The architecture of the neural network refers to the arrangement of the connection between neurons, processing element, number of layers, and the flow of signal in the neural network.

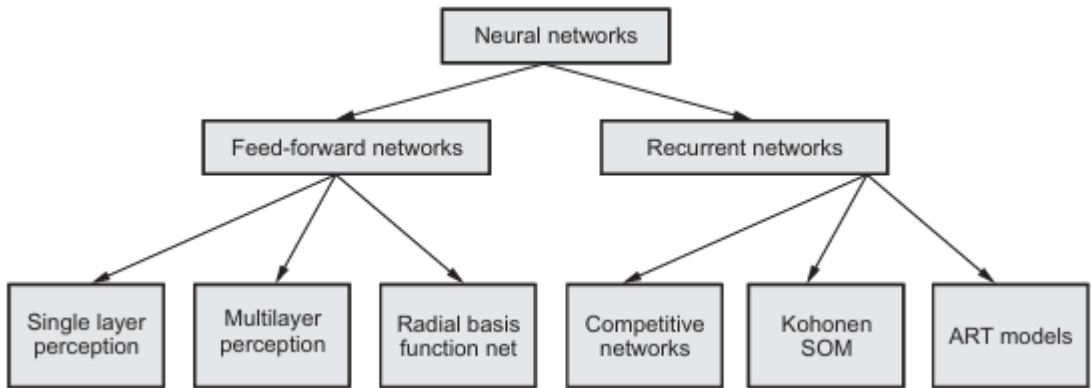


Fig. 2.3 Architecture of neural network.

- There are mainly two category of neural network architecture:
 - a. Feed-forward
 - b. Feedback (recurrent) neural networks.

1. Architecture and Learning Rule

- In late 1950s, Frank Rosenblatt introduced a network composed of the units that were enhanced version of McCulloch-Pitts Threshold Logic Unit (TLU) model.
- Rosenblatt's model of neuron, a perceptron, was the result of merger between two concepts from the 1940s, McCulloch-Pitts model of an artificial neuron and Hebbian learning rule of adjusting weights.
- In addition to the variable weight values, the perceptron model added an extra input that represents bias. Thus, the modified equation is now as follows :

$$\text{Sum} = \sum_{i=1}^N I_i W_i + b,$$

where **b** represents the bias value.

- **Fig. 2.4** shows a typical perception setup for pattern recognition applications, in which visual patterns are represented as matrices of elements between 0 and 1.

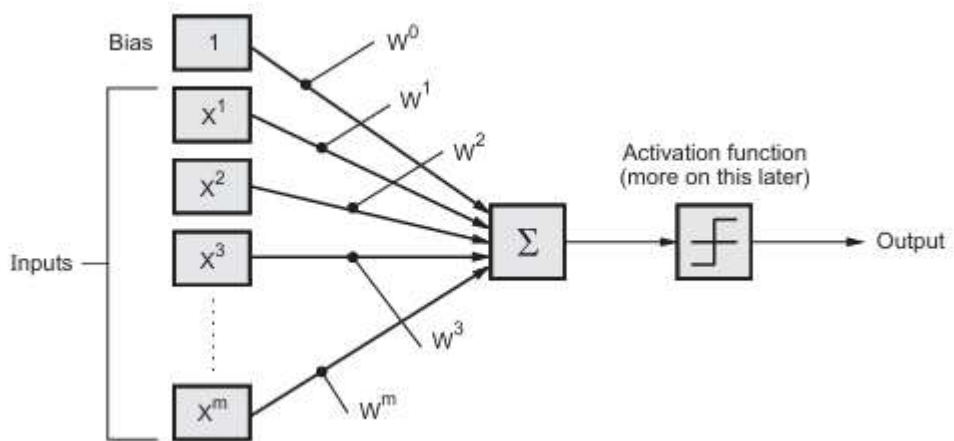


Fig. 2.4 Perceptron setup.

1. First layer act as a set of feature detectors that are hardwired to the input signals to detect specific features.
 2. Second layer i.e. output layer takes the outputs of the feature detectors in the first layer and classifies the given input pattern.
- Learning is initiated by making adjustments to the relevant connection strengths and a threshold value θ .
 - Here we consider only two class problem. Here output layer usually has only a single node. For an n -class problem ($n > 3$), the output layer usually has n -nodes, each corresponding to a class and the output node with the largest value indicates which class the input vector belongs to.
 - In the first stage, the linear combination of inputs is calculated. Each value of input array is associated with its weight value, which is normally between 0 and 1. Also, the summation function often takes an extra input value Theta with weight value of 1 to represent threshold or bias of a neuron.
 - a. The term x_i is referred to as active or excitatory if its value is 1.
 - b. If the value is 0 then it is inactive.
 - c. If the value is -1 then it is inhibitory.
 - The output unit is a linear threshold element with a threshold value θ :

$$\begin{aligned}
 0 &= f\left(\sum_{i=1}^n w_i x_i - \theta\right) \\
 &= f\left(\sum_{i=1}^n w_i x_i + w_0\right), w_0 \equiv -\theta \\
 &= f\left(\sum_{i=1}^n w_i x_i\right), x_0 \equiv 1
 \end{aligned}$$

where w_i is a modifiable weight associated with an incoming signal x_i .

- **Fig. 2.5** shows the bias term w_0 .

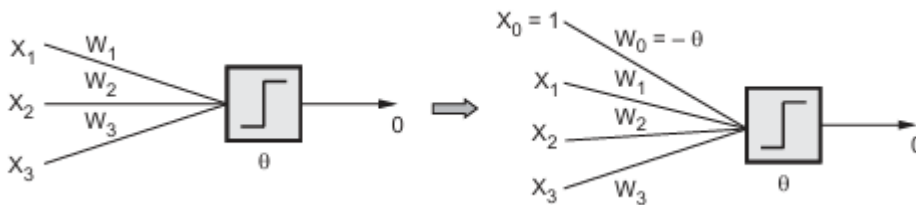


Fig. 2.5 Bias term w_0 .

- The function $y = f(x)$ describes relationship, an input-output mapping from x to v .
- The below equation is the $f(\cdot)$ is the activation function of the perceptron and it is typically either a signum function $\text{sgn}(x)$ or step function $\text{step}(x)$:

$$\begin{aligned}
 \text{sgn}(x) &= \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{otherwise} \end{cases} \\
 \text{step}(x) &= \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}
 \end{aligned}$$

- The sum-of-product value is then passed into the second stage to perform the activation function which generates the output from the neuron. The activation function "squashes" the amplitude of the output in the range of $[0, 1]$ or $[-1, 1]$ alternately. The behavior of the activation function will describe the characteristics of an artificial neuron model.
- The basic learning algorithm for a single layer perceptron repeats the following steps until the weights converge :
 1. Select an input vector x from the training data set.
 2. If the perceptron gives an incorrect response, modify all connection weights w_i according to

$$\Delta w_i = \eta t_i x_i$$

Where t_i is a target output and η is a learning state.

Perceptron Convergence Theorem

Theorem: If there is a set of weights that correctly classify the (linearly separable) training patterns, then the learning algorithm will find one such weight set, w^* in a finite number of iterations.

Assumptions:

1. At least one such set of weights, w^* , exists, and
2. There are a finite number of training patterns.
3. The threshold function is uni-polar (output is 0 or 1).

2. Exclusive OR problem

- XOR problem is a pattern recognition problem in neural network.
- Neural networks can be used to classify boolean functions depending on their desired outputs.
- For a two input binary XOR problem, the desired output is given in the form of truth table.

	X	Y	Class
Desired I/O pair 1	0	0	0
Desired I/O pair 2	0	1	1
Desired I/O pair 3	1	0	1
Desired I/O pair 4	1	1	0

- The XOR problem is not linearly separable. We cannot use a single layer perceptron to construct a straight line to partition the two dimensional input space into two regions, each containing only data points of the same class.

- Let us consider following four equations :

$$0 \times w_1 + 0 \times w_2 + w_0 \leq 0 \Leftrightarrow w_0 \leq 0$$

$$0 \times w_1 + 1 \times w_2 + w_0 \leq 0 \Leftrightarrow w_0 \leq -w_2$$

$$1 \times w_1 + 0 \times w_2 + w_0 \leq 0 \Leftrightarrow w_0 \leq -w_1$$

$$1 \times w_1 + 1 \times w_2 + w_0 \leq 0 \Leftrightarrow w_0 \leq -w_1 - w_2$$

2.2.3 Multilayer Feed Forward NN

- A multilayer feed-forward neural network is a network consisting of multiple layers of units, all of which are adaptive. The network is not allowed to have cycles from later layers back to earlier layers, hence the name "feed-forward". Let us consider a network with a single complete hidden layer. i.e., the network consists of some input nodes, some output nodes and a set of hidden nodes. Every hidden node takes inputs from each of the input nodes and feeds into each of the output nodes.
- In multi-layer feed forward neural networks, the sigmoid activation function, defined by $g(x) = \frac{1}{1+e^{-x}}$ is normally used.
- A multi-layer perceptron (MLP) has the same structure of a single layer perceptron with one or more hidden layers. An MLP is a network of simple neurons called perceptrons.

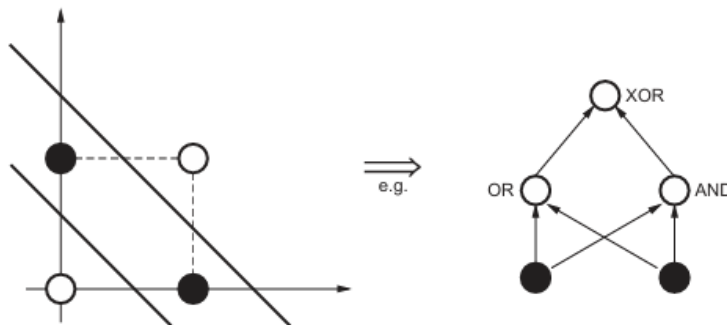


Fig. 2.6 Multilayer feed forward NN.

- A typical multilayer perceptron network consists of a set of source nodes forming the input layer, one or more hidden layers of computation nodes, and an output layer of nodes.
- It is not possible to find weights which enable single layer perceptrons to deal with non-linearly separable problems like XOR :
- Multi-layer perceptrons are able to cope with non-linearly separable problems.
- Each neuron in one layer has direct connections to all the neurons of the subsequent layer. MLP can implement nonlinear discriminants (for classification) and nonlinear regression functions (for regression).

- Historically, the problem was that there were no known learning algorithms for training MLPs. Fortunately; it is now known to be quite straightforward. The procedure for finding a gradient vector in the network structure is generally referred to as backpropagation. Because the gradient vector is calculated in the direction opposite to the flow of the output of each node.

Procedure of back propagation

- The output values are compared with the target to compute the value of some predefined error function.
 - The error is then feedback through the network.
 - Using this information, the algorithm adjusts the weights of each connection in order to reduce the value of the error function.
- Continue this process until the connection weights in the network have been adjusted so that the network output has converged, to an acceptable level, with the desired output.
 - If we use the gradient vector in a simple steepest descent method, the resulting learning paradigm is often referred to as the backpropagation learning rule.

Backpropagation works by approximating the non-linear relationship between the input and the output by adjusting the weight values internally.

- Generally, the backpropagation network has two stages, training and testing. During the training phase, the network is "shown" sample inputs and the correct classifications. For example, the input might be an encoded picture of a face, and the output could be represented by a code that corresponds to the name of the person.
- Fig. 2.7** shows three most commonly used activation functions in backpropagation MLPs.

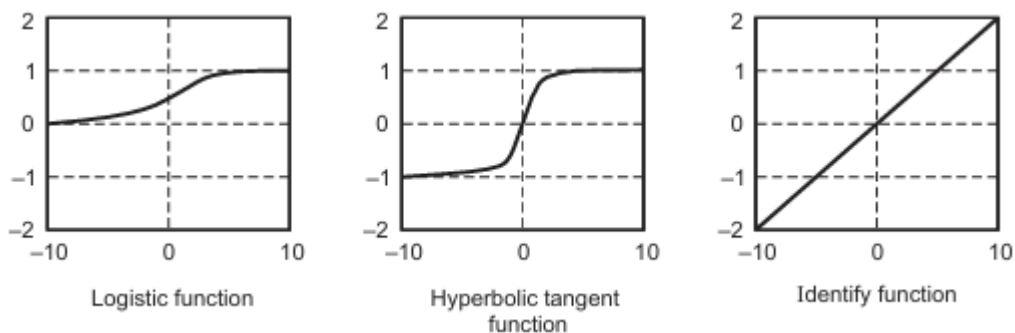


Fig. 2.7 Activation functions.

Logistic function

$$f(x) = \frac{1}{1 + e^{-x}}$$

Hyperbolic tangent function

$$f(x) = \tanh(x/2) = \frac{1 - e^{-x}}{1 + e^{-x}}$$

Identity function

$$f(x) = x$$

- Both the hyperbolic tangent function and logistic function approximate the signum and step function respectively. Sometimes these two function are referred to as squashing functions since the inputs to these functions are squashed to the range $[0,1]$ or $[-1,1]$.
- These functions are also called sigmoidal functions because their S-shaped curves exhibits smoothness and asymptotic properties.

A learning process is organized through a learning algorithm, which is a process of updating the weights in such a way that a machine learning tool implements a given input/output mapping with no errors or with some minimal acceptable error.

- Any learning algorithm is based on a certain learning rule, which determines how the weights shall be updated if the error occurs.

Backpropagation Learning Rule

- The net input of a node is defined as the weighted sum of the incoming signals plus a bias term. **Fig. 2.8** shows the backpropagation MLP for node j .

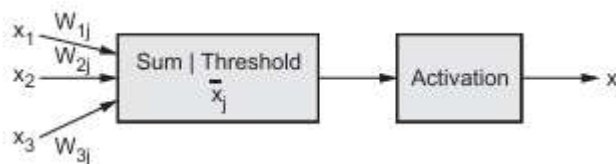


Fig. 2.8 Backpropagation MLP for node j .

- The net input and output of node j is as follows:

$$\bar{X}_j = \sum_i + W_{ij} + W_j$$

$$x_j = f(\bar{X}_j) = \frac{1}{1 + \exp(-\bar{X}_j)}$$

Where x_i is the output of node i located in any one of the pervious layers, W_{ij} is the weight associated with the link connecting nodes i and j , W_j is the bias of node j .

- Internal parameters associated with each node j is the weight W_{ij} . So changing the weights of the node will change the behaviour of the whole backpropagation MLP.
- **Fig. 2.9** shows two layer backpropagation MLP.

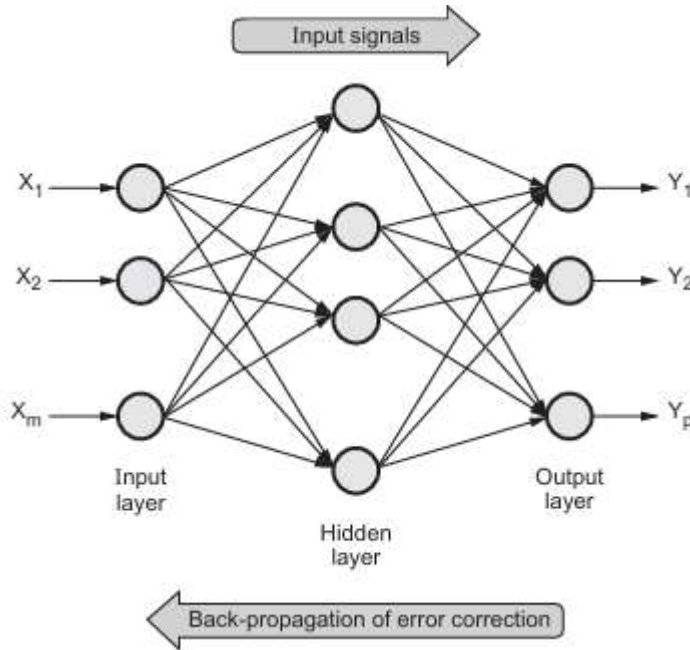


Fig. 2.9 Two layer backpropagation MLP.

- The above backpropagation MLP will refer to as a 3-4-3 network, corresponding to the number of nodes in each layer.
- The backward error propagation also known as the backpropagation (BP) or the Generalized Delta Rule (GDR). A squared error measure for the p^{th} input-output pair is defined as

$$E_p = \sum_k (d_k - x_k)^2$$

Where d_k is the desired output for node k and x_k is the actual output for node k when the input part of the p^{th} data pair is presented.

- To find the gradient vector, an error term $\bar{\epsilon}_i$ for node i is defined as:

$$\bar{\epsilon}_i = \frac{\partial + E_p}{\partial \bar{X}_i}$$

- The partial derivative can be rewritten as product of two terms using chain rule for partial differentiation:

$$\frac{\partial E(t)}{\partial w_{ij}(t)} = \frac{\partial E(t)}{\partial a_i(t)} \cdot \frac{\partial a_i(t)}{\partial w_{ij}(t)}$$

2.2.3.1 Delta Learning Rule for Multiperceptron Layer

- An important generalization of the perceptron training algorithm was presented by Widrow and Hoff as the least mean square learning procedure also known as the delta rule.
- The learning rule was applied to the "adaptive linear element" also named Adaline.
- The perceptron learning rule uses the output of the threshold function for learning. The delta rule uses the net output without further mapping into output values -1 or +1.
- Fig. 2.10** shows adaline.

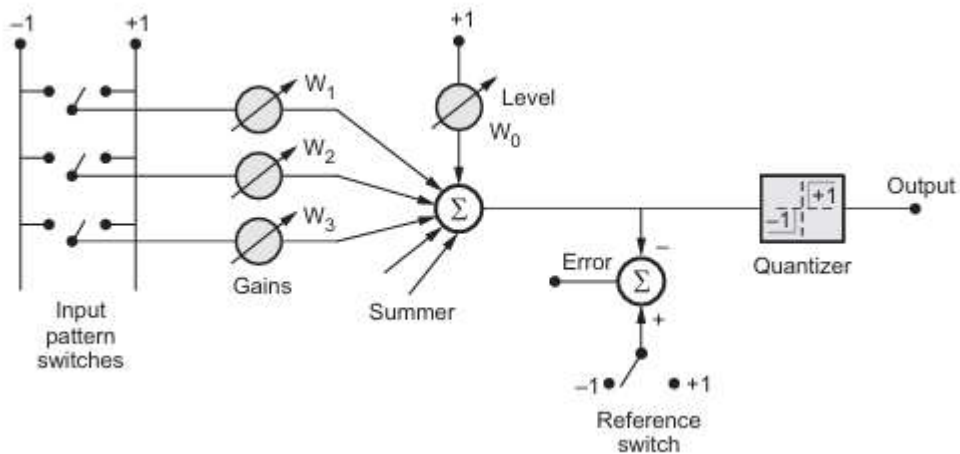


Fig. 2.10 Adaline.

- If the input conductances are denoted by w_i , where $i = 0, 1, 2, \dots, n$, and input and output signals by x_i and y , respectively, then the output of the central block is defined to be :

$$y = \sum_{i=1}^n w_i x_i + \theta$$

where $\theta \equiv w_0$

In a simple physical implementation this device consists of a set of controllable resistors connected to a circuit which can sum up currents caused by the input voltage signals. Usually the central block the summer is also followed by a quantizer which outputs either +1 or -1 , depending on the polarity of the sum.

- The problem is to determine the coefficients w_i where $i = 0, 1, \dots, n$, in such a way that the input output response is correct for a large number of arbitrarily chosen signal sets.
- If an exact mapping is not possible the average error must be minimized, for instance, in the sense of least squares.
- An adaptive operation means that there exists a mechanism by which the w_i can be adjusted, usually iteratively to attain the correct values.
- For the Adaline, Widrow introduced the delta rule to adjust the weights.
- For the p^{th} input-output pattern, the error measure of a single-output Adaline can be expressed as,

$$E_p = (t_p - o_p)^2$$

Where t_p = Target output

o_p = Actual output of the Adaline

- The derivation of E_p with respect to each weight w_i is

$$\frac{\partial E_p}{\partial w_i} = -2(t_p - o_p)x_i$$

- To decrease E_p by gradient descent, the update formula for w_i on the p^{th} input-output pattern is

$$\Delta_p w_i = \eta(t_p - o_p)x_i$$

- The delta rule tries to minimize squared errors, it is also referred to as the least mean square learning procedure or Widrow-Hoff learning rule.

Features of the delta rule are as follows:

1. Simplicity
2. Distributed learning : learning is not reliant on central control of the network.
3. Online learning : weights are updated after presentation of each pattern.

Rules for Feedforward Multilayer Perceptron

- The training algorithm is called Error Back Propagation (EBP) training algorithm. If a submitted pattern provides an output far from desired value, the weights and thresholds are adjusted so that the current mean square classification error is reduced.
- The training is repeated for all patterns until the training set provide an acceptable overall error. Usually the mapping error is computed over the full training set.

Error back propagation algorithm is working in two stages:

1. The trained network operates feed-forward to obtain output of the network
2. The weight adjustment propagate backward from output layer through hidden layer toward input layer.

2.3 Challenges and Suitability of Backpropagation in Neural Networks

The **backpropagation algorithm** is a widely used learning technique in artificial neural networks (ANNs). It enables the network to learn from errors by adjusting weights to minimize the difference between predicted and actual outputs. This algorithm is particularly suitable for problems with the following characteristics:

1. **High Dimensionality:** The data instances consist of numerous attribute-value pairs, which can represent complex relationships and patterns within the dataset. Backpropagation is effective for handling datasets with multiple features and attributes.
2. **Flexible Output Types:** The target outputs can vary in format, allowing for versatility in the types of tasks that can be solved. The output may be:
 - **Discrete-Valued:** Categorical outcomes, such as classifying images into predefined categories.
 - **Real-Valued:** Continuous numerical predictions, such as forecasting stock prices or temperature.

- **Vector Outputs:** Outputs that contain multiple attributes, which may be either real- or discrete-valued, enabling multi-dimensional predictions or classifications.
3. **Tolerance for Imperfect Data:** Backpropagation is capable of learning even when the training data contains noise or errors. This makes it a robust choice for real-world applications where data may not always be perfectly clean or labeled.
 4. **Efficient Evaluation Post-Training:** Although training times may be long, especially with large datasets and deep architectures, the backpropagation algorithm provides fast evaluation once the model has been trained. This is beneficial for applications where real-time predictions or quick responses are necessary.
 5. **Lack of Interpretability Requirement:** Backpropagation often produces complex, non-linear models that are challenging to interpret. In situations where the primary goal is accuracy rather than interpretability, this algorithm is well-suited. For example, backpropagation is ideal for tasks like image and speech recognition, where the model's internal decision-making process is less important than its overall performance.

2.3.1 Additional Aspects of Backpropagation

- **Iterative Learning:** Backpropagation relies on an iterative process of updating weights based on the gradient of the error function. This gradient descent process minimizes the error gradually, making the model more accurate over successive training cycles.
- **Challenges in Training Time:** Since backpropagation can involve numerous iterations to converge on an optimal solution, it may require considerable computational resources and time, particularly for deep neural networks with multiple layers. This is why GPUs and TPUs are often used to accelerate training.
- **Sensitivity to Initial Weights:** The algorithm's performance can be affected by the initial values of the weights. Poor initialization may lead to slower convergence or even cause the algorithm to get stuck in local minima.
- **Overfitting Risks:** When dealing with complex models, there is a risk of overfitting, where the model performs well on training data but fails to generalize to new data. Techniques such as regularization, dropout, and cross-validation are often used to mitigate this issue.
- **Alternative Optimization Algorithms:** While backpropagation with gradient descent is common, other optimization techniques like stochastic gradient descent (SGD), Adam, and RMSprop are also used to improve convergence speed and accuracy.

2.3.2 Practical Applications of Backpropagation

Backpropagation has been successfully applied in various fields, including:

- **Image Recognition:** Enables accurate classification of images, such as identifying objects or recognizing faces.
- **Natural Language Processing:** Used in applications like sentiment analysis, machine translation, and chatbots.
- **Medical Diagnosis:** Assists in analyzing medical images for identifying diseases or conditions.
- **Financial Forecasting:** Predicts stock market trends, credit risks, and other financial metrics.

2.4 Perceptrons

- The perceptron is a feed-forward network with one output neuron that learns a separating hyper-plane in a pattern space.
- The " n " linear F_x neurons feed forward to one threshold output F_y neuron. The perceptron separates linearly separable set of patterns.

2.4.1 Single Layer Perceptron

- The perceptron is a feed-forward network with one output neuron that learns a separating hyper-plane in a pattern space. The " n " linear F_x neurons feed forward to one threshold output F_y neuron. The perceptron separates linearly separable set of patterns.
- SLP is the simplest type of artificial neural networks and can only classify linearly separable cases with a binary target (1,0).
- We can connect any number of McCulloch-Pitts neurons together in any way we like. An arrangement of one input layer of McCulloch-Pitts neurons feeding forward to one output layer of McCulloch-Pitts neurons is known as a Perceptron.
- A single layer feed-forward network consists of one or more output neurons, each of which is connected with a weighting factor W_{ij} to all of the inputs X_i .
- The Perceptron is a kind of a single-layer artificial network with only one neuron. The Perceptron is a network in which the neuron unit calculates the linear combination of its real-valued or boolean inputs and passes it through a threshold activation function. **Fig. 2.11** shows Perceptron.

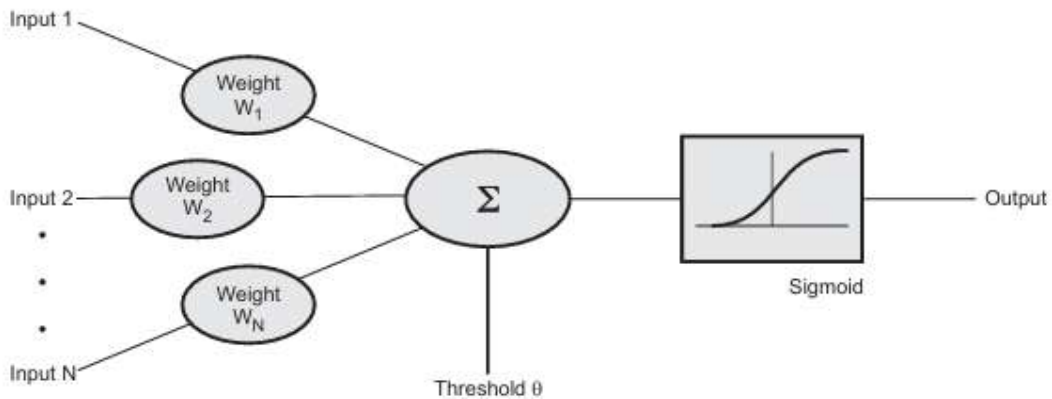


Fig. 2.11 Perceptron.

- The Perceptron is sometimes referred to a Threshold Logic Unit (TLU) since it discriminates the data depending on whether the sum is greater than the threshold value.
- In the simplest case the network has only two inputs and a single output. The output of the neuron is:

$$y = f\left(\sum_{i=1}^2 w_i x_i + b\right)$$

- Suppose that the activation function is a threshold then

$$f = \begin{cases} 1 & \text{if } s > 0 \\ -1 & \text{if } s \leq 0 \end{cases}$$

- The Perceptron can represent most of the primitive boolean functions: AND, OR, NAND and NOR but can't represent XOR.
- In single layer perceptron, initial weight values are assigned randomly because it does not have previous knowledge. It sum all the weighted inputs. If the sum is greater than the threshold value then it is activated i.e. output = 1.

Output

$$\begin{aligned} W_1 X_1 + W_2 X_2 + \dots + W_n X_n > \theta &\Rightarrow 1 \\ W_1 X_1 + W_2 X_2 + \dots + W_n X_n \leq \theta &\Rightarrow 0 \end{aligned}$$

- The input values are presented to the perceptron, and if the predicted output is the same as the desired output, then the performance is considered satisfactory and no changes to the weights are made.

- If the output does not match the desired output, then the weights need to be changed to reduce the error.
- The weight adjustment is done as follows :

$$\Delta W = \eta \times d \times x$$

Where x = Input data

d = Predicted output and desired output.

η = Learning rate

- If the output of the perceptron is correct then we do not take any action. If the output is incorrect then the weight vector is $W \rightarrow W + \Delta W$.
- The process of weight adaptation is called learning.
- **Perceptron Learning Algorithm:**
 1. Select random sample from training set as input.
 2. If classification is correct, do nothing.
 3. If classification is incorrect, modify the weight vector W using

$$W_i = W_i + \eta d(n) X_i(n)$$

Repeat this procedure until the entire training set is classified correctly.

2.4.2 Multilayer Perceptron

- A multi-layer perceptron (MLP) has the same structure of a single layer perceptron with one or more hidden layers. An MLP is a network of simple neurons called perceptrons.
- A typical multilayer perceptron network consists of a set of source nodes forming the input layer, one or more hidden layers of computation nodes, and an output layer of nodes.
- It is not possible to find weights which enable single layer perceptrons to deal with non-linearly separable problems like XOR: See **below figure**.

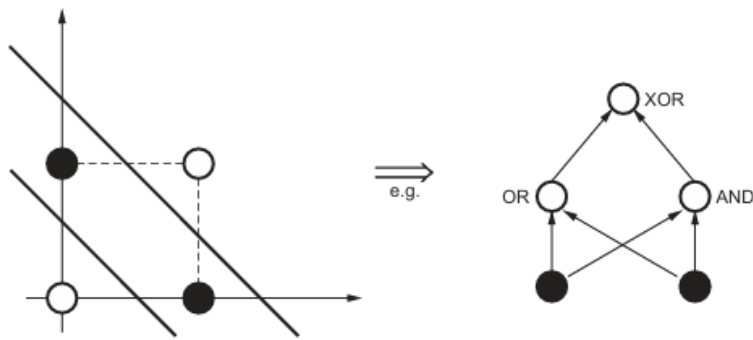


Fig. 2.12 Single Layer Perceptron

2.4.3 Limitation of Learning in Perceptron: Linear Separability

- Consider two-input patterns (X_1, X_2) being classified into two classes as shown in **Fig. 2.12**. Each point with either symbol of x or 0 represents a pattern with a set of values (X_1, X_2) .

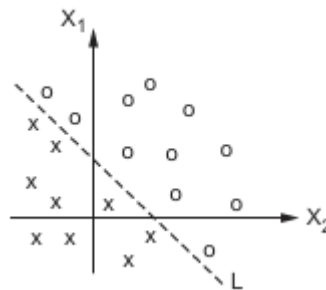


Fig. 2.13 Two class.

- Each pattern is classified into one of two classes. Notice that these classes can be separated with a single line L . They are known as linearly separable patterns.
- Linear separability refers to the fact that classes of patterns with n -dimensional vector $x = (x_1, x_2, \dots, x_n)$ can be separated with a single decision surface. In the case above, the line L represents the decision surface.
- If two classes of patterns can be separated by a decision boundary, represented by the linear equation then they are said to be linearly separable. The simple network can correctly classify any patterns.
- Decision boundary (i.e., W , b or q) of linearly separable classes can be determined either by some learning procedures or by solving linear equation systems based on representative patterns of each classes.
- If such a decision boundary does not exist, then the two classes are said to be linearly inseparable.

- Linearly inseparable problems cannot be solved by the simple network, more sophisticated architecture is needed.
- Examples of linearly separable classes

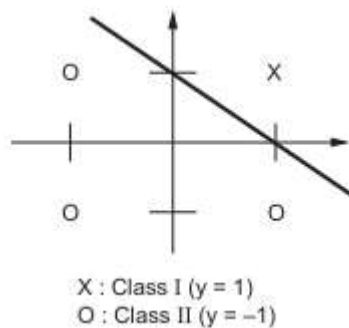
1. Logical AND function

Patterns (bipolar)

x_1	x_2	y
-1	-1	-1
-1	1	1
1	-1	1
1	1	1

Decision boundary

$$\begin{aligned}w_1 &= 1 \\w_2 &= 1 \\b &= -1 \\q &= 0 \\-1 + x_1 + x_2 &= 0\end{aligned}$$



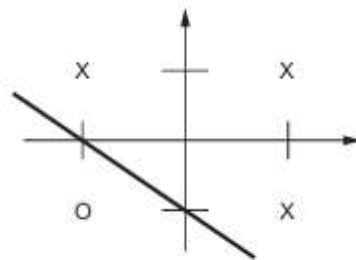
2. Logical OR function

Patterns (bipolar)

x_1	x_2	y
-1	-1	-1
-1	1	1
1	-1	1
1	1	1

Decision boundary

$$\begin{aligned}w_1 &= 1 \\w_2 &= 1 \\b &= 1 \\q &= 0 \\1 + x_1 + x_2 &= 0\end{aligned}$$

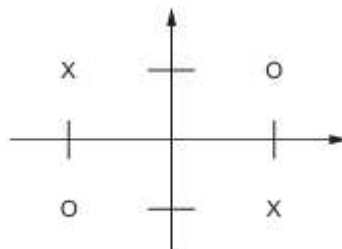


X : Class I ($y = 1$)
O : Class II ($y = -1$)

- Examples of linearly inseparable classes

1. Logical XOR (exclusive OR) function Patterns (bipolar)

x_1	x_2	y
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1



X : Class I ($y = 1$)
O : Class II ($y = -1$)

- No line can separate these two classes, as can be seen from the fact that the following linear inequality system has no solution.

$$\begin{cases} b - w_1 - w_2 < 0 \\ b - w_1 + w_2 \geq 0 \\ b + w_1 - w_2 \geq 0 \\ b + w_1 + w_2 < 0 \end{cases}$$

because we have $b < 0$ from (1) + (4), and $b \geq 0$ from (2) + (3), which is a contradiction.

2.5 Back Propagation Algorithms

- In 1969 a method for learning in multi-layer network, Backpropagation, was invented by Bryson and Ho. This algorithm is a sensible approach for dividing the contribution of each weight. It works basically the same as perceptrons.
 - Backpropagation is an approximate steepest descent algorithm, in which the performance index is mean square error.
 - The backpropagation algorithm looks for the minimum of the error function in weight space using the method of gradient descent. The combination of weights which minimizes the error function is considered to be a solution of the learning problem.
 - The multilayer perceptron, trained by the backpropagation algorithm, is currently the most widely used neural network.
1. Training set = a set of patterns with known labels
 2. Stochastic training = patterns are chosen randomly from the training set
 3. Batch training = all patterns are presented to the network before learning takes place
 4. On-line protocol = each pattern is presented once and only once, no memory in use
 5. Epoch = a single presentation of all patterns in the training set. The number of epochs is an indication of the relative amount of learning.
- When the training set is small one can generate surrogate training patterns. In the absence of problem-specific information, the surrogate patterns should be made by adding d-dimensional Gaussian noise to true training points, the category label should be left unchanged.
 - If we know about the source of variation among patterns we can manufacture training data. The number of hidden units should be less than the total number of training points n , say roughly $n/10$.

2.5.1 Gradient Descent Algorithm

- Goal : Solving minimization nonlinear problems through derivative information
- First and second derivatives of the objective function or the constraints play an important role in optimization. The first order derivatives are called the gradient and the second order derivatives are called the Hessian matrix.
- Derivative based optimization is also called nonlinear. Capable of determining search directions" according to an objective function's derivative information.
- Derivative based optimization methods are used for :
 1. Optimization of nonlinear neuro-fuzzy models
 2. Neural network learning
 3. Regression analysis in nonlinear models
- Basic descent methods are as follows :
 1. Steepest descent
 2. Newton-Raphson method

Gradient Descent:

- Gradient descent is a first-order optimization algorithm. To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point.
- Gradient descent is popular for very large-scale optimization problems because it is easy to implement, can handle black box functions, and each iteration is cheap.
- Given a differentiable scalar field $f(x)$ and an initial guess x_1 , gradient descent iteratively moves the guess toward lower values of " f " by taking steps in the direction of the negative gradient $-\nabla f(x)$.
- Locally, the negated gradient is the steepest descent direction, i.e., the direction that x would need to move in order to decrease " f " the fastest. The algorithm typically converges to a local minimum, but may rarely reach a saddle point, or not move at all if x_1 lies at a local maximum.
- The gradient will give the slope of the curve at that x and its direction will point to an increase in the function. So we change x in the opposite direction to lower the function value :

$$X_{k+1} = x_k - \lambda \nabla f(x_k)$$

The $\lambda > 0$ is a small number that forces the algorithm to make small jumps

Limitations of Gradient Descent

- Gradient descent is relatively slow close to the minimum : technically, its asymptotic rate of convergence is inferior to many other methods.
- For poorly conditioned convex problems, gradient descent increasingly 'zigzags' as the gradients point nearly orthogonally to the shortest direction to a minimum point.

Steepest Descent:

- Steepest descent is also known as gradient method.
- This method is based on first order Taylor series approximation of objective function. This method is also called saddle point method. **Fig. 2.13** shows steepest descent method.

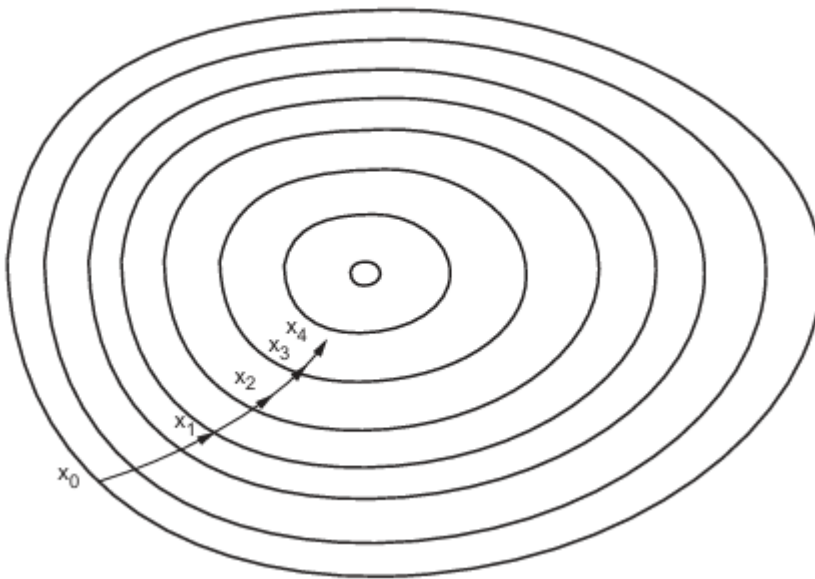


Fig. 2.14 Steepest Descent.

- The Steepest Descent is the simplest of the gradient methods. The choice of direction is where f decreases most quickly, which is in the direction opposite to $\nabla f(x_i)$. The search starts at an arbitrary point x_0 and then go down the gradient, until reach close to the solution.
- The method of steepest descent is the discrete analogue of gradient descent, but the best move is computed using a local minimization rather than computing a gradient. It is typically able to converge in few steps but it is unable to escape local minima or plateaus in the objective function.

- The gradient is everywhere perpendicular to the contour lines. After each line minimization the new gradient is always orthogonal to the previous step direction. Consequently, the iterates tend to zig-zag down the valley in a very inefficient manner.
- The method of Steepest Descent is simple, easy to apply, and each iteration is fast. It also very stable; if the minimum points exist, the method is guaranteed to locate them after at least an infinite number of iterations.

2.5.2 Performance Issue in Error Back Propagation (EBP)

- Computational Efficiency is main aspect of back-propagation. Number of operations to compute derivatives of error function scales with total number W of weights and biases.
- Single evaluation of error function for a single input requires $O(W)$ operations for large W . Compute derivatives using method of finite differences.

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \varepsilon) - E_n(w_{ji})}{\varepsilon} + O(\varepsilon) \text{ where } \varepsilon \ll 1$$

- Accuracy can be improved by making ε smaller until round-off problems arise.
- Accuracy can be improved by using central differences.

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \varepsilon) - E_n(w_{ji} - \varepsilon)}{2\varepsilon} + O(\varepsilon^2)$$

- This is $O(W^2)$.
- Generalization means performance on input patterns, i.e. input patterns which were not among the patterns on which the network was trained.
- If you train for too long, you can often get the sum-squared error very low, by over-fitting the training data you get a network which performs very well on the training data, but not as well as it could on unseen data.

By stopping training earlier, one hopes that the network will have learned the broad rules of the problem, but not bent itself into the shape of some of the more idiosyncratic (perhaps even noisy) training patterns.

2.6 Genetic Algorithm

- First appeared in 1950s and early 1960s while biologists were explicitly seeking to the model of natural evolution. Idea of inheritance and mutation introduced by Ingo Rechenberg which is termed as evolution strategy (1965).

- M. J. Walsh introduced evolutionary programming (1966). Later versions introduced population which leads to the Genetic Algorithms.
- In 1975 John Holland published book Adaptation in Natural and Artificial System. This was the first book to represent concept of adaptive digital systems using mutation, selection and crossover.
- Genetic Algorithm (GA) is a search technique used in computing to find true or approximate solutions to optimization and search problems. Genetic algorithms are categorized as global search heuristics.
- Genetic algorithms are a particular class of evolutionary algorithms that use techniques inspired by evolutionary biology such as inheritance, mutation, selection and recombination.
- A typical genetic algorithm requires two things to be defined:
 1. Genetic representation of the solution domain.
 2. Fitness function to evaluate the solution domain.

Generally speaking, genetic algorithms are simulations of evolution, of what kind ever. In most cases, however, genetic algorithms are nothing else than probabilistic optimization methods which are based on the principles of evolution.

2.6.1 Biological Background

The Cell: Animal or human cell is a complex of many small factories that work together. The center of all this is the cell nucleus. The genetic information is contained in the cell nucleus.

Chromosomes

- All the genetic information gets stored in the chromosomes. Each chromosome is build of Dioxy Ribo Nucleic Acid (DNA).
- In humans, a chromosome exists in the form of pairs (23 pairs). The chromosomes are divided into several parts called genes. Genes code the properties of species i.e., the characteristics of an individual.
- The possibilities values of the genes for one property are called allele and a gene can take different alleles.
- The set of all possible alleles present in a particular population forms a gene pool. This gene pool can determine all the different possible variations for the future generations. The set of all the genes of a specific species is called genome.

- Each gene has a unique position on the genome called locus. Most living organisms store their genome on several chromosomes, but in the Genetic Algorithms (GAs), all the genes are usually stored on the same chromosomes. Thus chromosomes and genomes are synonyms with one other in GAs.
- For a particular individual, the entire combination of genes is called genotype. The phenotype describes the physical aspect of decoding a genotype to produce the phenotype.

The selection process is always done on the phenotype. The reproduction process consists of recombining genotypes. Chromosomes may contain two sets of genes (diploids). In this case, the dominant one will determine the phenotype whereas the other one, called recessive, will still be present and can be passed on to the offspring.

- In haploid representation, only one set of each gene is stored, thus the process of determining which allele should be dominant and which one should be recessive is avoided. Most GA concentrates on haploid chromosomes because they are much simple to construct.
- The following table gives a list of different expressions, which are common in genetics, along with their equivalent in the framework of GAs:

Natural Evolution	Genetic Algorithm
genotype	coded string
phenotype	uncoded point
chromosome	string
gene	string position
allele	value at a certain position
fitness	objective function value

2.6.2 Basic Operators

- A genetic algorithm maintains a population of candidate solutions for the problem at hand and makes it evolve by iteratively applying a set of stochastic operators.

- The simplest form of genetic algorithm involves three types of operators: selection, crossover and mutation.
1. **Selection:** Selection replicates the most successful solutions found in a population at a rate proportional to their relative quality. This operator selects chromosomes in the population for reproduction. The fitter the chromosome, the more times it is likely to be selected to reproduce.
 2. **Crossover:** This operator randomly chooses a locus and exchanges the sub-sequences before and after that locus between two chromosomes to create two offspring. For example, the strings 10000100 and 11111111 could be crossed over after the third locus in each to produce the two offspring 10011111 and 11100100. The crossover operator roughly mimics biological recombination between two single chromosome (haploid) organisms.
 3. **Mutation:** Mutation randomly perturbs a candidate solution. This operator randomly flips some of the bits in a chromosome. For example, the string 00000100 might be mutated in its second position to yield 01000100. Mutation can occur at each bit position in a string with some probability, usually very small (e.g., 0.001).

2.6.3 Working Principle

- To illustrate the working principles of GAs, an unconstrained optimization problem is considered. Let us consider following maximization problem.
Maximize $f(x) = x_i^l \leq x_i \leq x_i^u, i = 1, 2, \dots, N$
where, x_i^l and x_i^u are the lower and upper bound the variable x_i can take. Although a maximization problem is considered here, a maximization problem can also be handled using Gas.

2.6.4 Outline of Genetic Algorithm

1. **[Start]** Generate random population of n chromosomes (suitable solutions for the problem).
2. **[Fitness]** Evaluate the fitness $f(\mathbf{x})$ of each chromosome $\mathbf{x}$ in the population.
3. **[New population]** Create a new population by repeating following steps until the new population is complete.
 - a. **[Selection]** Select two parent chromosomes from a population according to their fitness (the better fitness, the bigger chance to be selected).
 - b. **[Crossover]** with a crossover probability cross over the parents to form a new offspring (children). If no crossover was performed, offspring is an exact copy of parents.

- c. **[Mutation]** with a mutation probability mutate new offspring at each locus (position in chromosome).
- d. **[Accepting]** Place new offspring in a new population.
- 4. **[Replace]** Use new generated population for a further run of algorithm.
- 5. **[Test]** If the end condition is satisfied, stop and return the best solution in current population.
- 6. **[Loop]** Go to step 2.

GA Steps

Step 1: Set population size and probability

Step 2: Define fitness function

Step 3: Generate initial population

Step 4: Calculate fitness for each chromosome

Step 5: Mating of chromosomes

Step 6: Create offspring-crossover and mutation

Step 7: Offspring in new population

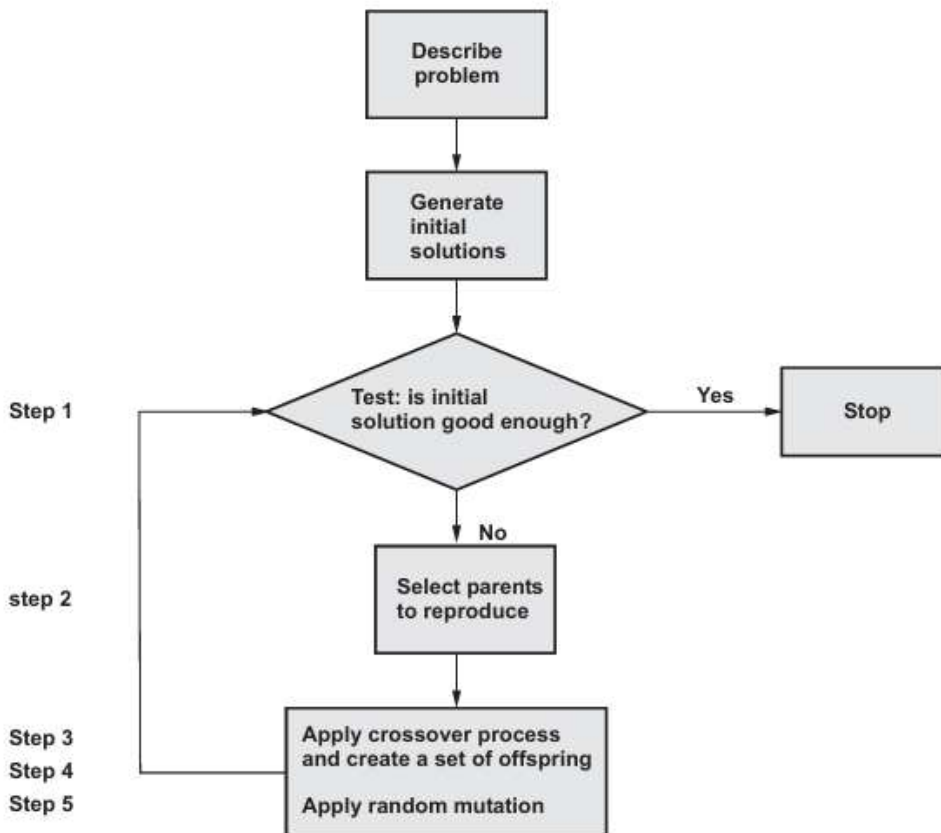


Fig. 2.15 Genetic algorithm process flow chart.

Step 8: Repeat Step 5 until new = initial population

Step 9: Replace initial population with new

Step 10: To Step 4 and repeat until criteria achieved

2.6.5 Advantages of GA

- Genetic Algorithm is a stochastic algorithm.
- Randomness as an essential role in both selection and reproduction phases.
- Genetic algorithms always consider a population of solutions. A population base algorithm is also very amenable for parallelization.
- There is no particular requirement on the problem before using genetic algorithms, so it can be applied to resolve any problem (optimization).
- GAs are a new field and parts of the theory have still to be properly established. We can find almost as many opinions on GAs as there are researchers in this field.

2.6.6 Limitations

- GAs are not guaranteed to find the global optimum solution to a problem.
- GAs are an extremely general tool and they have no specific way for solving particular problems.
- GAs are usually used when everything else is failed or when we don't have enough knowledge of the search space.
- Even when such specialized techniques exist, it often interesting to hybridise them with a GA in order to possibly gain some improvements.

2.6.7 Difference between GA and Traditional Methods

1. Genetic algorithms a coded form of the function values i.e. parameter set, rather than with the actual values themselves. For example, if we want to find the minimum of the function $f(x) = x^3 + x^2 + 5$, the GA would not deal directly with x or y values, but with strings that encode these values. For this case, strings representing the binary x values should be used.
2. Genetic algorithms use a set or population of points to conduct a search, not just a single point on the problem space. This gives GAs the power to search noisy spaces littered with local optimum points. Instead of relying on a single point to search through the space, the GAs looks at many different areas of the problem space at once, and uses all of this information to guide it.

3. Genetic algorithms use only payoff information to guide themselves through the problem space. Many search techniques need a variety of information to guide themselves. Hill climbing methods require derivatives, for example. The only information a GA needs is some measure of fitness about a point in the space. Once the GA knows the current measure of "goodness" about a point, it can use this to continue searching for the optimum.
4. GAs are probabilistic in nature, not deterministic. This is a direct result of the randomization techniques used by GAs.
5. GA is inherently parallel. Here lies one of the most powerful features of genetic algorithms. GA's, by their nature, are very parallel, dealing with a large number of points simultaneously.

Table 2.1 Difference between GA and Traditional Methods

Parameters	Genetic Algorithms	Traditional Methods
Work with	Coding of parameter set	Parameters directly
Use information	Payoff i.e. objective function	Payoff plus derivatives etc.
Rules	Probabilistic	Fully deterministic
Search	a population of points	a population of points a single point

2.6.8 Fitness Function

- Fitness is an important concept in genetic algorithms. The fitness of a chromosome determines how likely it is that it will reproduce. Fitness is usually measured in terms of how well the chromosome solves some goal problem. Fitness can also be subjective (aesthetic). E.g., if the genetic algorithm is to be used to sort numbers, then the fitness of a chromosome will be determined by how close to a correct sorting it produces.
- A fitness function quantifies the optimality of a solution (chromosome) so that particular solution may be ranked against all the other solutions. A fitness value is assigned to each solution depending on how close it actually is to solving the problem.
- Ideal fitness function correlates closely to goal plus quickly computable.

- **Example:** In TSP, $f(x)$ is sum of distances between the cities in solution. The lesser the value, the fitter the solution is.
- The performance of the individual strings is measured by a fitness function. A fitness function is a problem specific user defined heuristic. After each iteration, the members are given a performance measure derived from the fitness function and the "fittest" members of the population will propagate the next iteration.

$$\text{Fitness} = F_i, \text{Hit} = \lambda_i, \text{Survival} = \phi_i, \text{Death} = \delta_i$$
$$F_i = 2\lambda_i - \delta_i + \sum \phi_i$$

- The fitness function is defined over the genetic representation and measures the quality of the represented solution. The fitness function is always problem dependent.
- For instance, in the knapsack problem we want to maximize the total value of objects that we can put in a knapsack of some fixed capacity. A representation of a solution might be an array of bits, where each bit represents a different object and the value of the bit (0 or 1) represents whether or not the object is in the knapsack.
- Not every such representation is valid, as the size of objects may exceed the capacity of the knapsack. The fitness of the solution is the sum of values of all objects in the knapsack if the representation is valid or 0 otherwise. In some problems, it is hard or even impossible to define the fitness expression; in these cases, interactive genetic algorithms are used.

2.7 Hypothesis Space Search

- **Search Space:** The space of all feasible solutions is called search space. Each point in the search space represent one feasible solution. Each feasible solution can be "marked" by its value or fitness for the problem.
- If we are solving some problem, we are usually looking for some solution, which will be the best among others.
- We are looking for our solution, which is one point or more among feasible solutions, that is one point in the search space.
- The looking for a solution is then equal to a looking for some extreme (minimum or maximum) in the search space. The search space can be whole known by the time of solving a problem, but usually we know only a few points from it and we are generating other points as the process of finding solution continues.
- Genetic algorithm employs a randomized beam search method to seek a maximally fit hypothesis.

Motivation

- The solution(s) to machine learning tasks are often called hypotheses, because they can be expressed as a hypothesis that the observed positives and negatives for a categorization is explained by the concept learned for the solution.
- The hypotheses have to be represented in some representation scheme, and, as usual with AI tasks, this will have a big effect on many aspects of the learning methods.
- General definition of a hypothesis: "A hypothesis is a statement of a relationship between two or more variables."
- Sometimes, it is necessary to evaluate the performance of learned hypotheses.

Reason for using hypothesis

- Learning from a limited-size database indicating the effectiveness of different medical treatments, it is important to understand as precisely as possible the accuracy of the learned hypothesis.
 - The evaluating hypotheses are an integral component of many learning methods.
 - It is important to understand the likely errors inherent in estimating the accuracy of the pruned and unpruned tree.
 - Estimating the accuracy of a hypothesis is relatively straightforward when data is plentiful.
 - An estimator is any random variable used to estimate some parameter of the underlying population from which a sample is drawn.
1. The estimation bias of an estimator Y for an arbitrary parameter p is $E[Y] - p$, If the estimation bias is 0 , then Y is an unbiased estimator for p .
 2. The variance of an estimator Y for an arbitrary parameter p is simply the variance of Y

2.7.1 Population Evolution and the Schema Theorem

- It is developed by John Holland.
- If a chromosome is of length n then it contains 3^n schemata (as each position can have the value 0,1 or *)
- In theory, this means that for a population of M individuals we are evaluating up to $M3^n$ schemata.

- But, bear in mind that some schemata will not be represented and others will overlap with other schemata.
- This is exactly what we want. We eventually want to create a population that is full of fitter schemata and we will have lost weaker schemata. It is the fact that we are manipulating M individuals but $M3^n$ schemata that gives genetic algorithms what has been called implicit parallelism.
- **Length:** is defined as the distance between the start of the schema and the end of the schema minus one.
- **Order:** is defined as the number of defined positions.
- **Fitness ratio:** It is defined as the ratio of the fitness of a schema to the average fitness of the population.

*	*	0	*	0	*	1
---	---	---	---	---	---	---

Here length = 6 and order = 3

- In this analysis we assume that the GA is a way of processing genotype features rather than genotypes themselves - a feature being simply a set of values in specific positions. A particular feature is defined in terms of a schema.
- This is a genotype-like string with specific values in some positions and 'don't care' values (asterisks) in others. An example is: `*10**0****`.
- This schema has ten characters in all, including seven "don't care" values. It will match any 10-character genotype with a 1 in the second position, a 0 in the third position and a 0 in the sixth position.

The longer the length of the schema, the more chance there is of the schema being disrupted by a crossover operation. This implies that shorter schemata have a better chance of surviving from one generation to the next. In turn, this implies that if we know that certain attributes of a problem fit well together then these should be placed as close as possible together in the coding.

- This observation is also true for the order of the chromosome. If we are not worried about the number of defined positions (i.e. we allow as many '*' as possible) then a crossover operation has less chance of disrupting good schemata. Intuitively, it would seem better to have short, low-order schema. This is only based on empirical evidence but it is widely believed that these assumptions are true and the following theory makes some sense of this.

- Using a technique where we choose parents relative to their fitness (e.g. roulette wheel selection), fitter schema should find their way from one generation to another.
- Intuitively, if a schema is fitter than average then it should not only survive to the next generation but should also increase its presence in the population. If Φ is the number of instances of any particular schema S within the population at time t , then at $t + 1$ we would expect

$$\Phi(S, t + 1) > \Phi(S)$$

to hold for above average fitness schemata.

- Going one stage further we can estimate the number of schema present at $t + 1$

$$\Phi(S, t + 1) = \Phi(S, t) n \frac{f(S)}{\sum f_i}$$

Where, n is the size of the population

$f(S)$ is the fitness of the schema

$\sum f_i$ is the fitness of the population

$$\Phi(S, t + 1) = \Phi(S, t) \frac{f(s)}{f_{avg}}$$

Where, f_{avg} is the average fitness of the population.

- If a particular schema stays a constant, c , above the average we can say even more about the effects of reproduction

$$\begin{aligned}\Phi(S, t + 1) &= \Phi(S, t) \frac{(f_{avg} + c_{avg})}{f_{avg}} \\ &= \Phi(S, t)(1 + c)\end{aligned}$$

- Setting $t = 0$

$$\Phi(S, t) = \Phi(S, 0)(1 + c)^t$$

- Notice that the number of schema rises exponentially.

2.7.2 Sample Error and True Error

- The sample error ($err_s(h)$) of h with respect to target function (f) and data sample (S) is the proportion of examples h misclassifies. The sample test error is the mean error over the test sample.

$$\text{err}_s(h) = \frac{1}{n} \sum_{i=1}^n L(f(x_i), h(x_i))$$

- The true error of hypothesis h with respect to target function f and distribution D is the probability that h will misclassify an instance drawn at random according to D .

$$\text{Err}(h) = E[L(f(x), h(x))]$$

- An error $_D(h)$ is the true error of hypothesis h with respect to the target function f and data distribution D . It is the probability h will misclassify an instance drawn at random according to D .
- An error (h) is the sample error of hypothesis h with respect to the target function f and data sample set S . It is the proportion of examples in S that h misclassifies.

2.8 Genetic Programming (GP)

Genetic programming (GP) is an evolutionary computation technique pioneered and popularized by John Koza. This domain-independent approach applies genetic principles to evolve a population of computer programs, effectively breeding them to find solutions to complex problems. GP simulates the principles of natural selection by iteratively transforming an initial population of randomly generated programs through operations inspired by biological evolution. Key genetic operations in GP include crossover (recombination), mutation, reproduction, gene duplication, and gene deletion.

The primary advantage of GP is that it automates the design of algorithms by evolving solutions instead of requiring human-engineered solutions for each specific problem. GP abstracts the core principles of Darwinian evolution, enabling it to be applied across various domains without requiring detailed domain knowledge.

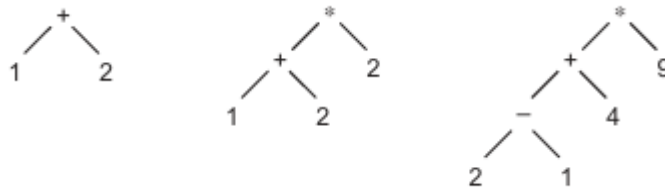
Why Use Genetic Programming?

1. **Automated Design:** GP reduces the need for manual algorithm design by automating the search for optimal or near-optimal solutions.
2. **Flexibility and Adaptability:** GP is adaptable to different types of problems, including optimization, symbolic regression, automatic program generation, and more.
3. **Exploring Unseen Solutions:** GP can yield innovative solutions that may not be obvious to human designers, often leading to creative or unconventional solutions.

2.8.1 Core Characteristics and Applications

Here's a rephrased and expanded explanation of the content on Genetic Programming (GP) from the image, with added advanced details:

- **S-Expressions and Evolution of Solutions:** GP can evolve S-expressions, useful in LISP programming, to solve mathematical and symbolic problems.
- **Prefix Notation in LISP:** GP often uses prefix notation in languages like LISP, where the operator precedes its operands, as in:



This approach simplifies operator precedence, making it easier for compilers to interpret without parentheses.

- **Expression Trees:** GP often represents solutions as expression trees. Each program or solution candidate is structured as a tree where operations form the nodes, and operands are the leaves. This tree structure facilitates easy manipulation of sub-expressions during genetic operations like crossover and mutation.

2.8.2 Genetic Programming Algorithm Steps

1. **Initial Population:** Randomly generate an initial population of candidate solutions, often represented as syntax trees or abstract syntax representations of code.
2. **Fitness Ranking:** Each program is evaluated based on a fitness function, a problem-specific metric that determines how well each program solves the problem.
3. **Selection:** Select programs based on their fitness ranking, with higher-performing solutions more likely to contribute to the next generation.
4. **Genetic Operators (Mating and Mutation):**
 - **Crossover (Mating):** Exchange segments between pairs of selected programs to form offspring, promoting the recombination of beneficial traits.
 - **Mutation:** Introduce random changes to parts of programs to maintain diversity and avoid premature convergence on suboptimal solutions.

5. **Duplication and Pruning:** Duplicate well-performing programs to preserve high-quality solutions and occasionally prune unnecessary parts to optimize the solution.
6. **Iteration until Convergence:** Repeat the process of selection, crossover, mutation, and fitness evaluation until a satisfactory solution is found or a termination criterion is met (such as a maximum number of generations).

2.8.3 Advanced Concepts in Genetic Programming

1. **Dynamic Fitness Adaptation:** Adjusting the fitness function dynamically to focus on different aspects of the problem across generations, promoting a more comprehensive exploration of the solution space.
2. **Multi-objective Genetic Programming:** Optimizing for multiple objectives simultaneously, such as accuracy and computational efficiency, using approaches like Pareto-based selection to evolve balanced solutions.
3. **Co-evolutionary GP:** Introducing multiple populations that evolve together, competing or collaborating to solve complex, interdependent problems, which is especially useful in game theory and competitive environments.
4. **Genetic Programming with Constraints:** Incorporating domain-specific constraints directly into the GP process to ensure solutions adhere to physical, logical, or resource limitations, thereby improving the relevance of generated solutions.
5. **Application of GP in Real-World Problems:** GP has been successfully applied to areas like automated design, symbolic regression, financial modeling, robotic control, bioinformatics, and more, where discovering novel patterns or models is essential.

2.8.4 Benefits and Limitations of Genetic Programming

- **Benefits:** GP can lead to innovative solutions, is highly adaptable, and is capable of handling complex, non-linear, and multi-dimensional problems. Its ability to autonomously evolve programs saves significant time and effort in algorithm design.
- **Limitations:** GP can be computationally intensive, often requiring substantial computational resources and time, particularly for high-dimensional problems. GP solutions may also be challenging to interpret due to their complexity.

2.9 Models of Evolution and Learning

Lamarckian Evolution

Jean-Baptiste Lamarck is most noted for his theory of inheritance of acquired characteristics, first proposed in 1801.

According to Lamarckian evolution, if an organism undergoes changes during its lifetime to adapt to its environment, these changes can be passed down to its offspring.

In 1809, Lamarck further developed this idea in his book *Philosophie Zoologique*, where he outlined a two-part mechanism. He suggested that organisms adapt to their environment, and these adaptations are gradually introduced into the species, passing down acquired traits through generations. This concept, known as Lamarckism or the theory of transformation, contrasts with Darwinian evolution, which is based on natural selection without the inheritance of acquired traits.

Although Lamarck's theory has largely been superseded by Darwinian and neo-Darwinian perspectives, some modern research in epigenetics shows that certain environmental factors can influence gene expression and may be passed to subsequent generations, thus rekindling interest in Lamarckian mechanisms in a limited context.

2.9.1 The Baldwin Effect

The Baldwin Effect explores the relationship between learning and evolution, particularly focusing on how individual learning can impact the evolutionary trajectory of a species. First proposed by James Mark Baldwin in 1896, this effect has significant implications for evolutionary biology, psychology, and artificial intelligence.

Relevance of the Baldwin Effect: The Baldwin Effect is especially relevant for:

1. Researchers working on hybrid systems involving Genetic Algorithms (GAs), Neural Networks (NNs), or Genetic Algorithm/Neural Network (GANN) systems, where evolutionary and learning processes intersect.
2. Evolutionary psychologists studying the relationships among learning, instinct, and evolution.
3. Researchers in machine learning, genetic algorithms, neural networks, evolutionary theory, and cognitive science.

Concept of the Baldwin Effect: Baldwin proposed that individual learning could drive evolution without requiring Lamarckian inheritance. By enabling individuals to adapt through learning, the species can smooth the fitness landscape, essentially facilitating evolutionary progress.

- **Guiding Evolution through Learning:** When individuals learn advantageous behaviors, it eases the adaptive requirements for survival, creating a smoother fitness landscape. This gradual facilitation allows genetic evolution to 'catch up' to learned behaviors.

- **Evolution of Instinctive Behaviors:** Baldwin suggested that over generations, behaviors initially learned might be genetically encoded, turning what once required learning into instinctive actions.
- **Non-Lamarckian Inheritance:** Unlike Lamarckian theories, the Baldwin Effect does not suggest that learned traits are passed down directly. Instead, it proposes that individuals with beneficial learned behaviors are more likely to survive and reproduce, indirectly influencing gene frequencies in the population.

Mechanism of the Baldwin Effect: The Baldwin Effect operates based on certain key observations:

1. **Environmental Adaptation Pressure:** In a dynamic environment, evolutionary pressure favors individuals who can learn and adapt within their lifetimes, enhancing survival chances.
2. **Transition from Learned to Genetically Encoded Traits:** Over many generations, behaviors that initially require learning may be genetically encoded, reducing dependence on individual learning for these traits.

Advanced Concepts Related to the Baldwin Effect:

1. **Learning as an Evolutionary Driver:** The Baldwin Effect illustrates how learning can act as an accelerant in evolutionary processes. For example, in artificial neural networks, this concept underpins hybrid approaches that combine evolution and learning, improving convergence rates and solution quality in complex problems.
2. **Epigenetic Considerations:** While the Baldwin Effect traditionally does not involve Lamarckian inheritance, recent epigenetic research suggests that environmental factors can influence gene expression in ways that mimic Baldwinian dynamics. These heritable epigenetic changes provide a potential mechanism by which environmentally-induced adaptations may affect evolution.
3. **Evolutionary Robotics and Artificial Life:** The Baldwin Effect has inspired research in fields like evolutionary robotics, where robots are designed to learn and adapt to their environments. By combining learning algorithms with evolutionary principles, robots can autonomously develop behaviors that improve their fitness, often leading to innovations in autonomous system design.
4. **Implications for Cognitive Science and Evolutionary Psychology:** The Baldwin Effect bridges the gap between learned and instinctual behaviors, suggesting a mechanism through which complex cognitive abilities may evolve. This has implications for understanding human cognition, including language acquisition, social behavior, and cultural evolution.

Bayesian and Computational Learning

3.1 Bayes Theorem

Introduction to Bayes Theorem

Bayes theorem is given by an English statistician, philosopher, and Presbyterian minister named **Mr. Thomas Bayes** in 17th century. Bayes provides their thoughts in decision theory which is extensively used in important mathematics concepts as Probability. Bayes theorem is also widely used in Machine Learning where we need to predict classes precisely and accurately. An important concept of Bayes theorem named **Bayesian method** is used to calculate conditional probability in Machine Learning application that includes classification tasks. Further, a simplified version of Bayes theorem (Naïve Bayes classification) is also used to reduce computation time and average cost of the projects.

Bayes theorem is also known with some other name such as **Bayes rule or Bayes Law**. **Bayes theorem helps to determine the probability of an event with random knowledge**. It is used to calculate the probability of occurring one event while other one already occurred. It is a best method to relate the condition probability and marginal probability.

In simple words, we can say that Bayes theorem helps to contribute more accurate results.

Bayes Theorem is used to estimate the precision of values and provides a method for calculating the conditional probability. However, it is hypocritically a simple calculation but it is used to easily calculate the conditional probability of events where intuition often fails. Some of the data scientist assumes that Bayes theorem is most widely used in financial industries but it is not like that. Other than financial, Bayes theorem is also extensively applied in health and medical, research and survey industry, aeronautical sector, etc.

What is Bayes Theorem?

Bayes theorem is one of the most popular machine learning concepts that helps to calculate the probability of occurring one event with uncertain knowledge while other one has already occurred.

Bayes' theorem can be derived using product rule and conditional probability of event X with known event Y:

- ❖ According to the product rule we can express as the probability of event X with known event Y as follows;

$$P(X \mid Y) = P(X|Y) P(Y) \quad (3.1)$$

- ❖ Further, the probability of event Y with known event X:

$$P(X \mid Y) = P(Y|X) P(X) \quad (3.2)$$

Mathematically, Bayes theorem can be expressed by combining both equations on right hand side. We will get:

$$P(X \mid Y) = \frac{P(Y|X) \cdot P(X)}{P(Y)} \quad (3.3)$$

Here, both events X and Y are independent events which means probability of outcome of both events does not depends one another.

The above equation is called as Bayes Rule or Bayes Theorem.

- ❖ $P(X|Y)$ is called as **posterior**, which we need to calculate. It is defined as updated probability after considering the evidence.
- ❖ $P(Y|X)$ is called the likelihood. It is the probability of evidence when hypothesis is true.
- ❖ $P(X)$ is called the **prior probability**, probability of hypothesis before considering the evidence
- ❖ $P(Y)$ is called marginal probability. It is defined as the probability of evidence under any consideration.

Hence, Bayes Theorem can be written as:

$$\text{posterior} = \text{likelihood} * \text{prior} / \text{evidence}$$

Prerequisites for Bayes Theorem

While studying the Bayes theorem, we need to understand few important concepts. These are as follows:

1. Experiment

An experiment is defined as the planned operation carried out under controlled condition such as tossing a coin, drawing a card and rolling a dice, etc.

2. Sample Space

During an experiment what we get as a result is called as possible outcomes and the set of all possible outcome of an event is known as sample space. For example, if we are rolling a dice, sample space will be:

$$S_1 = \{1, 2, 3, 4, 5, 6\}$$

Similarly, if our experiment is related to toss a coin and recording its outcomes, then sample space will be:

$$S_2 = \{\text{Head}, \text{Tail}\}$$

3. Event

Event is defined as subset of sample space in an experiment. Further, it is also called as set of outcomes.

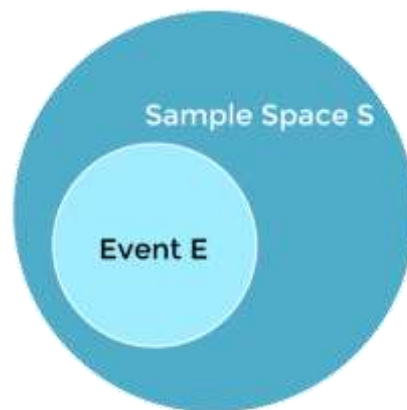


Fig. 3.1 Event.

Assume in our experiment of rolling a dice, there are two event A and B such that;

$$A = \text{Event when an even number is obtained} = \{2, 4, 6\}$$

$$B = \text{Event when a number is greater than 4} = \{5, 6\}$$

- ❖ **Probability of the event A "P(A)"** = Number of favourable outcomes / Total number of possible outcomes
 $P(E) = 3/6 = 1/2 = 0.5$
- ❖ Similarly, **Probability of the event B "P(B)"** = Number of favourable outcomes / Total number of possible outcomes
 $= 2/6$
 $= 1/3$
 $= 0.333$
- ❖ **Union of event A and B:**
 $A \cup B = \{2, 4, 5, 6\}$

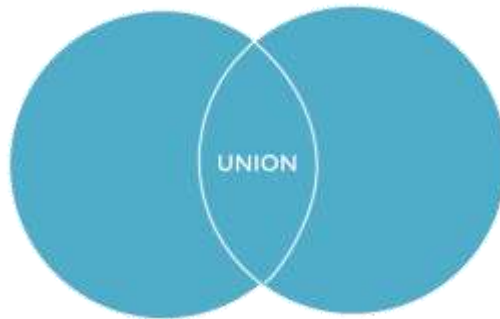


Fig. 3.2 Union of event A and B

- ❖ **Intersection of event A and B:**
 $A \cap B = \{6\}$

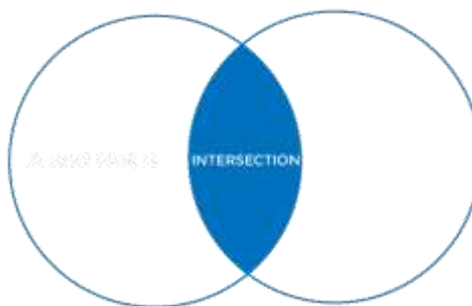


Fig. 3.3 Intersection of event A and B

- ❖ **Disjoint Event:** If the intersection of the event A and B is an empty set or null then such events are known as **disjoint event** or **mutually exclusive events** also.

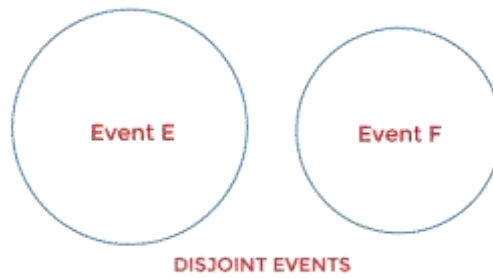


Fig. 3.4 Disjoint of event A and B

4. Random Variable:

It is a real value function which helps mapping between sample space and a real line of an experiment. A random variable is taken on some random values and each value having some probability. However, it is neither random nor a variable but it behaves as a function which can either be discrete, continuous or combination of both.

5. Exhaustive Event:

As per the name suggests, a set of events where at least one event occurs at a time, called exhaustive event of an experiment.

Thus, two events A and B are said to be exhaustive if either A or B definitely occur at a time and both are mutually exclusive for e.g., while tossing a coin, either it will be a Head or may be a Tail.

6. Independent Event:

Two events are said to be independent when occurrence of one event does not affect the occurrence of another event. In simple words we can say that the probability of outcome of both events does not depends one another.

Mathematically, two events A and B are said to be independent if:

$$P(A \cap B) = P(AB) = P(A) \cdot P(B) \quad (3.4)$$

7. Conditional Probability:

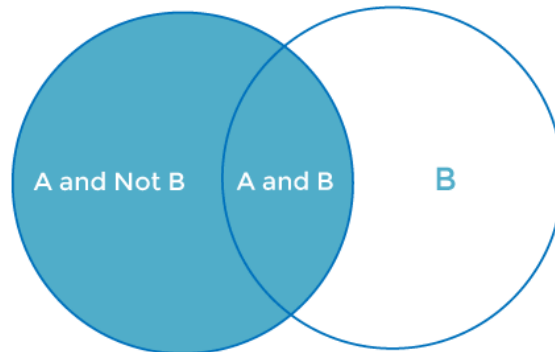
Conditional probability is defined as the probability of an event A, given that another event B has already occurred (i.e. A conditional B). This is represented by $P(A|B)$ and we can define it as:

$$P(A|B) = P(A \cap B) / P(B) \quad (3.5)$$

8. Marginal Probability:

Marginal probability is defined as the probability of an event A occurring independent of any other event B. Further, it is considered as the probability of evidence under any consideration.

$$P(A) = P(A|B) * P(B) + P(A|\sim B) * P(\sim B) \quad (3.6)$$



Here $\sim B$ represents the event that B does not occur.

How to apply Bayes Theorem or Bayes rule in Machine Learning?

Bayes theorem helps us to calculate the single term $P(B|A)$ in terms of $P(A|B)$, $P(B)$, and $P(A)$. This rule is very helpful in such scenarios where we have a good probability of $P(A|B)$, $P(B)$, and $P(A)$ and need to determine the fourth term.

Naïve Bayes classifier is one of the simplest applications of Bayes theorem which is used in classification algorithms to isolate data as per accuracy, speed and classes.

Let's understand the use of Bayes theorem in machine learning with below example.

Suppose, we have a vector A with I attributes. It means

$$A = A_1, A_2, A_3, A_4, \dots, A_i \quad (3.7)$$

Further, we have n classes represented as $C_1, C_2, C_3, C_4, \dots, C_n$.

These are two conditions given to us, and our classifier that works on Machine Language has to predict A and the first thing that our classifier has to choose will be the best possible class. So, with the help of Bayes theorem, we can write it as:

$$P(C_i/A) = [P(A/C_i) * P(C_i)] / P(A) \quad (3.8)$$

Here;

$P(A)$ is the condition-independent entity.

$P(A)$ will remain constant throughout the class means it does not change its value with respect to change in class. To maximize the $P(C_i/A)$, we have to maximize the value of term $P(A/C_i) * P(C_i)$.

With n number classes on the probability list let's assume that the possibility of any class being the right answer is equally likely. Considering this factor, we can say that:

$$P(C_1) = P(C_2) = P(C_3) = P(C_4) = \dots = P(C_n). \quad (3.9)$$

This process helps us to reduce the computation cost as well as time. This is how Bayes theorem plays a significant role in Machine Learning and Naïve Bayes theorem has simplified the conditional probability tasks without affecting the precision. Hence, we can conclude that:

$$P(A_i/C) = P(A_1/C) * P(A_2/C) * P(A_3/C) * \dots * P(A_n/C) \quad (3.10)$$

Hence, by using Bayes theorem in Machine Learning we can easily describe the possibilities of smaller events.

3.2 Concept Learning

❖ Bayes theorem is a principled way to calculate posterior probability of each hypothesis

Formula for Probabilities :

Product rule : Probability $P(A \wedge B)$ of conjunction of two events A and B

$$P(A \wedge B) = P(A/B)P(B) = P(B/A)P(A) \quad , \quad (3.11)$$

Sum rule : Probability of a disjunction of two events A and B

$$P(A \vee B) = P(A) + P(B) - P(A \wedge B) \quad (3.12)$$

Theorem of total probability : If events $A_1, \dots, A_n$ are mutually exclusive with $\sum_{i=1}^n P(A_i) = 1$, then

$$P(B) = \sum_{i=1}^n P(B/A_i)P(A_i) \quad (3.13)$$

3.2.1 Brute-Force Bayes Concept Learning

- ❖ A Concept-Learning algorithm considers a finite hypothesis space H defined over an instance space X .
- ❖ The task is to learn the target function : $c: X \rightarrow \{0,1\}$.
- ❖ The learner gets a set of training examples $\langle x_1, d_1 \rangle \dots \langle x_m, d_m \rangle$ where x_i is an instance from X and d_i is its target value.
- ❖ Brute - Force Bayes Concept Learning Algorithm finds the maximum a posteriori hypothesis (h_{MAP}), based on Bayes theorem.

Brute - Force MAP Learning Algorithm

1. For each hypothesis h in H , calculate the posterior probability

$$P(h/I) = \frac{P(I/h)P(h)}{P(I)} \quad (3.14)$$

2. Output the hypothesis h_{MAP} with the highest posterior probability.

$$h_{ML} = \underset{h \in H}{\operatorname{argmax}} P(h/I) \quad (3.15)$$

- ❖ This algorithm may require significant computation, because it applies Bayes theorem to each hypothesis in H to calculate $P(I | D)$. But this is impractical for large hypothesis spaces. The algorithm is still of interest because it provides a standard against which we may judge the performance of other concept learning algorithms.
- ❖ BF MAP learning algorithm must specify values for $P(h)$ and $P(I | h)$.
- ❖ $P(h)$ and $P(I | h)$ must be chosen to be consistent with the assumptions:
 1. The training data (I) is noise free.
 2. The target concept c is contained in the hypothesis space H
 3. We have no a priori reason to believe that any hypothesis is more probable than any other

- ❖ With these assumptions:

$$\begin{aligned} p(h) &= \frac{1}{|H|} \text{ for all } h \text{ in } H \\ P(I/H) &= \begin{cases} 1 & \text{if } d_1 = h(x_i) \text{ for all } d_1 \text{ in } I \\ 0 & \text{otherwise} \end{cases} \end{aligned} \quad (3.16)$$

- ❖ In other words, the probability of data (I) given hypothesis h is 1 if I is consistent with h and 0 otherwise.
- ❖ So, the values of $P(h/I)$ will be :

$$\begin{aligned}P(h/I) &= \frac{0 \cdot P(h)}{P(D)} \\&= 0 \text{ if } h \text{ inconsistent with } D. \\P(h/I) &= \frac{1 \cdot 1/|H|}{P(I)} = \frac{1 \cdot 1/|H|}{|VS_{H,I}|/|H|} \\&= \frac{1}{|VS_{H,I}|} \text{ if } h \text{ is consistent with } I\end{aligned}\tag{3.17}$$

- ❖ $P(I) = |VS_{H,I}|/|H|$ because
 1. the sum over all hypotheses of $P(h/I)$ must be one and the number of hypotheses from H consistent with I is $|VS_{H,I}|$ or
 2. we can derive $P(I)$ from the theorem of total probability and the fact that the hypotheses are mutually exclusive.

3.2.2 MAP Hypotheses and Consistent Learners

- ❖ A learning algorithm is a consistent learner if it commits zero errors over the training examples
- ❖ Every consistent learner outputs a MAP hypothesis if
 1. we assume a uniform prior probability distribution over H and if
 2. we assume a deterministic noise free training data.
- ❖ Find-S and Candidate-Elimination output a MAP hypotheses
- ❖ Bayesian perspective can be used to characterize learning algorithms even if they do not explicitly manipulate probabilities

3.3 Maximum Likelihood

Maximum likelihood is an approach commonly used for such density estimation problems, in which a likelihood function is defined to get the probabilities of the distributed data.

In machine learning, the likelihood is a measure of the data observations up to which it can tell us the results or the target variables value for particular data points. In simple words, as the name suggests, the likelihood is a function that tells us how likely the specific data point suits the existing data distribution.

For example. Suppose there are two data points in the dataset. The likelihood of the first data point is greater than the second. In that case, it is assumed that the first data point provides accurate information to the final model, hence being likable for the model being informative and precise.

Difference between Probability and Likelihood

Although the working and intuition of both probability and likelihood appear to be the same, there is a slight difference, here the possibility is a function that defines or tells us how accurate the particular data point is valuable and contributes to the final algorithm in data distribution and how likely is to the machine learning algorithm.

Whereas probability, in simple words is a term that describes the chance of some event or thing happening concerning other circumstances or conditions, mostly known as conditional probability.

Also, the sum of all the probabilities associated with a particular problem is one and can not exceed it, whereas the likelihood can be greater than one.

Maximum Likelihood: The Base

The maximum likelihood estimation is a base of some machine learning and deep learning approaches used for classification problems. One example is logistic regression, where the algorithm is used to classify the data point using the best-fit line on the graph. The same approach is known as the perceptron trick regarding deep learning algorithms.

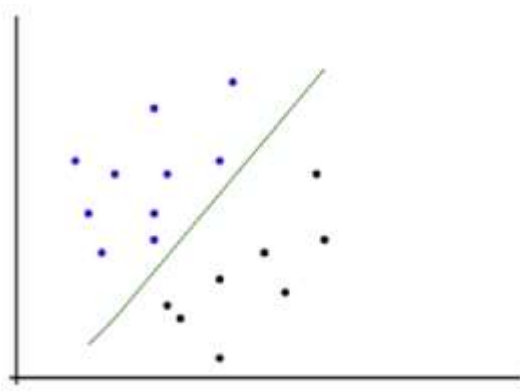


Fig. 3.5 Maximum Likelihood: The Base

As shown in the above image, all the data observations are plotted in a two-dimensional diagram where the X-axis represents the independent column or the training data, and the y-axis represents the target variable. The line is drawn to separate both data observations, positives and negatives. According to the algorithm, the observations that fall above the line are considered positive, and data points below the line are regarded as negative data points.

Maximum Likelihood Estimation: Code Example

We can quickly implement the maximum likelihood estimation technique using logistic regression on any classification dataset. Let us try to implement the same.

```
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()
lr.fit(X_train,y_train)
lr_pred=lr.predict(X_test)
sns.regplot(x="X",y='lr_pred',data=df_pred ,logistic=True, ci=None)
```

The above code will fit the logistic regression for the given dataset and generate the line plot for the data representing the distribution of the data and the best fit according to the algorithm.

Key Takeaways

- ❖ Maximum Likelihood is a function that describes the data points and their likeliness to the model for best fitting.
- ❖ Maximum likelihood is different from the probabilistic methods, where probabilistic methods work on the principle of calculation probabilities. In contrast, the likelihood method tries to maximize the likelihood of data observations according to the data distribution.
- ❖ Maximum likelihood is an approach used for solving the problems like density distribution and is a base for some algorithms like logistic regression.
- ❖ The approach is very similar and is predominantly known as the perceptron trick in terms of deep learning methods.

The Maximum Likelihood Principle

This has a Bayesian interpretation which can be helpful to think about. Suppose that we have a model with parameters θ and a collection of data examples X .

For concreteness, we can imagine that θ is a single value representing the probability that a coin comes up heads when flipped, and X is a sequence of independent coin flips. We will look at this example in depth later.

If we want to find the most likely value for the parameters of our model, that means we want to find

$$\operatorname{argmax}_{\theta} P(\theta | X) \quad (3.18)$$

By Bayes' rule, this is the same thing as

$$\operatorname{argmax}_{\theta} \frac{P(X|\theta)P(\theta)}{P(X)}. \quad (3.19)$$

The expression $P(X)$, a parameter agnostic probability of generating the data, does not depend on θ at all, and so can be dropped without changing the best choice of θ . Similarly, we may now posit that we have no prior assumption on which set of parameters are better than any others, so we may declare that $P(\theta)$ does not depend on θ either! This, for instance, makes sense in our coin flipping example where the probability it comes up heads could be any value in $[0,1]$ without any prior belief it is fair or not (often referred to as an uninformative prior). Thus we see that our application of Bayes' rule shows that our best choice of θ is the maximum likelihood estimate for:

$$\hat{\theta} = \operatorname{argmax}_{\theta} P(X | \theta). \quad (3.20)$$

As a matter of common terminology, the probability of the data given the parameters ($P(X | \theta)$) is referred to as the likelihood.

Maximum Likelihood for Continuous Sample

Everything that we have done so far assumes we are working with discrete random variables, but what if we want to work with continuous ones?

The short summary is that nothing at all changes, except we replace all the instances of the probability with the probability density. Recalling that we write densities with lower case, this means that for example we now say

$$-\log(p(X | \theta)) = -\log(p(x_1 | \theta)) - \log(p(x_2 | \theta)) \cdots - \log(p(x_n | \theta)) = -\sum_i \log(p(x_i | \theta)). \quad (3.21)$$

The question becomes, "Why is this OK?" After all, the reason we introduced densities was because probabilities of getting specific outcomes themselves was zero, and thus is not the probability of generating our data for any set of parameters zero?

Indeed, this is the case, and understanding why we can shift to densities is an exercise in tracing what happens to the epsilons.

Let's first re-define our goal. Suppose that for continuous random variables we no longer want to compute the probability of getting exactly the right value, but instead matching to within some range ϵ . For simplicity, we assume our data is repeated observations $x_1, \dots, x_N$ of identically distributed random variables $X_1, \dots, X_N$. As we have seen previously, this can be written as

$$P(X_1 \in [x_1, x_1 + \epsilon], X_2 \in [x_2, x_2 + \epsilon], \dots, X_N \in [x_N, x_N + \epsilon] \mid \theta) \approx \epsilon^N p(x_1 \mid \theta) \cdot p(x_2 \mid \theta) \cdots p(x_n \mid \theta). \quad (3.22)$$

Thus, if we take negative logarithms of this we obtain

$$-\log(P(X_1 \in [x_1, x_1 + \epsilon], X_2 \in [x_2, x_2 + \epsilon], \dots, X_N \in [x_N, x_N + \epsilon] \mid \theta)) \approx -N \log(\epsilon) - \sum_i \log(p(x_i \mid \theta)) \quad (3.23)$$

3.4 Minimum Description Length

The minimum description length (MDL) criteria in machine learning says that the best description of the data is given by the model which compresses it the best. Put another way, learning a model for the data or predicting it is about capturing the regularities in the data and any regularity in the data can be used to compress it. Thus, the more we can compress a data, the more we have learnt about it and the better we can predict it.

MDL is also connected to Occam's Razor used in machine learning which states that "other things being equal, a simpler explanation is better than a more complex one." In MDL, the simplicity (or rather complexity) of a model is interpreted as the length of the code obtained when that model is used to compress the data.

The ideal version of MDL is given by the Kolmogorov Complexity, which is defined as the length of the shortest computer program that prints the sequence of observed data and halts. However, the Komogorov Complexity is uncomputable i.e. it can be shown that there exists no computer program that, for every set of data D , when given D as input, returns the shortest program that prints D). Moreover, for finite length sequences, the best program may depend on the sequence itself.

Practical versions of MDL are either based on one stage universal codes (known as refined MDL) or two-stage codes (known as crude MDL). The refined MDL suggest that, for a given class of models, pick the model which minimizes the worst case redundancy on the data. This leads to precisely the universal models such as mixture models and normalized maximum likelihood (NML) model, which we discussed in last few lectures:

$$\begin{aligned} \text{Mixture model: } q_\gamma(x^n) &= \sum_{q \in Q_\gamma} \pi(q) q(x^n) \quad \text{where } \pi \text{ is chosen to be (nearly)} \\ &\quad \text{minimax optimal prior for class } Q_\gamma \\ \text{NML model: } q_\gamma(x^n) &= \arg \max_{q \in Q_\gamma} \frac{q(x^n)}{\sum_{x^n} \max_{q \in Q_\gamma} q(x^n)} \end{aligned} \quad (3.24)$$

The crude MDL or two-stage code approach is based on the notion that we can specify the descriptive properties of a model for data in two stages - (i) encode the model with some codelength $L(q)$, encode the data using the model with codelength $L_q(x^n)$. Now pick the model which minimizes the total codelength of the two-stage code:

$$q_\gamma(x^n) = \arg \min_{q \in Q_\gamma} \{L(q) + L_q(x^n)\} \quad (3.25)$$

MDL for Model Class Selection

Suppose we know the universal model q_γ (say mixture or NML) within a given class Q_γ . The best model class γ^* however may be unknown.

We can use the two-stage MDL coding approach as follows. Encode the data using the universal model for each class $\{Q_\gamma\}_{\gamma \in \Gamma}$, i.e. $L_{q_\gamma}(x^n) = \log 1/q_\gamma(x^n)$. Encode the class using a prefix code for the class index γ . Now pick the model class that minimizes the length of concatenated prefix code:

$$\hat{\gamma} = \arg \min_{\gamma \in \Gamma} \{L(\gamma) + L_{q_\gamma}(x^n)\} \quad (3.26)$$

The model selected by this procedure is $q_{\hat{\gamma}}$, the universal model for class $\hat{\gamma}$.

If the true class γ^* was known, the model with the smallest description would be q_{γ^*} and its encoding would have length $L_{q_{\gamma^*}}(x^n)$. Lets look at the length of the encoding using the selected model $q_{\hat{\gamma}}$:

$$L_{q_{\hat{\gamma}}}(x^n) \leq L(\hat{\gamma}) + L_{q_{\hat{\gamma}}}(x^n) = \min_{\gamma \in \Gamma} \{L(\gamma) + L_{q_\gamma}(x^n)\} \quad (3.27)$$

Since $\gamma^* \in \Gamma$, this implies that

$$L_{q_{\gamma^*}}(x^n) \leq L(\gamma^*) + L_{q_{\gamma^*}}(x^n) \quad (3.28)$$

From homework, we know that a prefix code for integers can be designed so that $L(\gamma^*) = O(\log \gamma^*)$. Thus, the overhead for choosing the model class using the MDL procedure is small - $O\left(\frac{\log \gamma^*}{n}\right)$ bits/symbol.

This approach is similar to the notion of universal models for hierarchical classes as discussed in last lecture. Here the nested hierarchical models are $\cup_{\gamma \in \Gamma} Q_{\gamma}$.

Comparison of 2 -stage MDL with mixture model

What if we knew the optimal weights to design an optimal mixture code for the entire union of classes $\cup_{\gamma \in \Gamma} Q_{\gamma}$? Could we have gained much over the two-stage MDL approach? The answer is no. In fact, the length of the two-stage code is sandwiched between the lengths of two mixture models for the union class:

$$-\log Q^L(x^n) \leq L_{2\text{-stage}}(x^n) \leq -\log Q^U(x^n) + \log c_2 \quad (3.29)$$

where $Q^L(x^n) = \sum_{\gamma \in \Gamma} \pi^L(\gamma) q_{\gamma}(x^n)$ with $\pi^L(\gamma) = 2^{-L(\gamma)} / \sum_{\gamma \in \Gamma} 2^{-L(\gamma)}$, and $Q^U(x^n) = \sum_{\gamma \in \Gamma} \pi^U(\gamma) q_{\gamma}(x^n)$ with $\pi^U(\gamma) = 2^{-2L(\gamma)} / \sum_{\gamma \in \Gamma} 2^{-2L(\gamma)}$.

Proof:

$$L_{2\text{-stage}}(x^n) = \min_{\gamma \in \Gamma} \{L(\gamma) - \log q_{\gamma}(x^n)\} = -\log \max_{\gamma \in \Gamma} [2^{-L(\gamma)} q_{\gamma}(x^n)] \quad (3.30)$$

Now, since $\sum_{\gamma \in \Gamma} 2^{-L(\gamma)} \leq 1$ by Kraft's inequality, we have:

$$Q^L(x^n) \geq \sum_{\gamma \in \Gamma} 2^{-L(\gamma)} q_{\gamma}(x^n) \geq \max_{\gamma \in \Gamma} [2^{-L(\gamma)} q_{\gamma}(x^n)] \geq \sum_{\gamma \in \Gamma} 2^{-2L(\gamma)} q_{\gamma}(x^n) = Q^U(x^n) / c_2 \quad (3.31)$$

The result follows by taking the $-\log$ of the above expression.

MDL for model selection

Now let's consider model selection within a class using two-stage MDL as an alternative to mixture model with optimal weights and NML:

$$q_{\gamma}(x^n) = \arg \min_{q \in Q_{\gamma}} \{L(q) + L_q(x^n)\} \quad (3.32)$$

where $L_q(x^n) = \log 1/q(x^n)$ and $L(q)$ is the length of a prefix code for the models. $L(q)$ is a measure of the description length of the model q and hence measures the complexity of the model. Thus, the above approach is also called complexity penalized or regularized maximum likelihood estimation.

A Bayesian interpretation of the above method is that it is akin to selecting the maximum aposterior model since $L(q)$ is equivalent to a $-\log$ prior. Thus, the method is seeking the model which minimizes $-\log \text{prior} + \log \text{likelihood}$, or equivalently, which maximizes the $\text{prior} \times \text{likelihood}$ (which is $\propto$ posterior).

The length of the two-stage code for the selected model is $L_{2\text{-stage}}(x^n) = \min_{q \in Q_Y} \{L(q) + L_q(x^n)\}$. Barron and Cover [BC91] showed a bound on the expected redundancy per symbol of such a code as follows. Consider the expected value of the length of the two-stage code with respect to the true data generating distribution p : $L(q) + E_p[\log 1/q(X^n)] = L(q) + D_n(p||q) + H_n(p)$, and define the index of resolvability as

$$R_n(p) = \min_{q \in Q_Y} \{L(q) + D_n(p||q)\} \quad (3.33)$$

Then, the expected redundancy of the two-stage code

$$\frac{1}{n} E_p [L_{2\text{-stage}}(x^n) - \log 1/p(x^n)] \leq \frac{1}{n} R_n(p) \quad (3.34)$$

Proof:

$$\begin{aligned} E_p [L_{2\text{-stage}}(x^n) - \log 1/p(x^n)] &= E_p \left[\min_{q \in Q_Y} \{L(q) + \log p(x^n)/q(x^n)\} \right] \\ &\leq \min_{q \in Q_Y} E_p [L(q) + \log p(x^n)/q(x^n)] \\ &= \min_{q \in Q_Y} \{L(q) + D_n(p||q)\} = R_n(p) \end{aligned}$$

The inequality holds using Jensen's inequality since minimum is a concave function.

This bound reflects the approximation error vs estimation error (or bias vs variance) tradeoff in machine learning problems, since the more complex the model class, the larger the number of bits needed to describe the model $L(q)$, but the smaller the divergence with respect to the true distribution $D_n(p||q)$.

We will investigate this in next class for some parametric and nonparametric classes Q_Y such as Markov models of order γ and histograms density estimates with γ bins. For parametric classes, this will lead to a per symbol expected redundancy bound of $O((\# \text{ parameters } \log n)/$

n), which is precisely the same bound obtained using an optimal mixture model as we saw in last few lectures.

Classification/Regression as Maximum Likelihood estimation

The MDL principle can also be used for classification or regression. In these problems, the goal is essentially equivalent to modeling $p_{Y|X}$ or p_{XY} . We will show this for one concrete setup next.

A typical model for regression is $Y_i = f(X_i) + \epsilon_i$ for $i = 1, \dots, n$ where $\epsilon_i \stackrel{iid}{\sim} \mathcal{N}(0, \sigma^2)$. The regression problem is equivalent to modeling $p_{Y|X}$ which in this case is $\mathcal{N}(f(X), \sigma^2)$, the Gaussian model indexed by parameter $f(X)$. The length of codeword for encoding the data using this model is

$$\log 1/p_{Y|X}(y^n | x^n) = \log \left(\sqrt{2\pi\sigma^2} e^{\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - f(x_i))^2} \right) \propto \sum_{i=1}^n (y_i - f(x_i))^2 \quad (3.35)$$

Thus, the negative log loss under additive Gaussian noise model is simply the least squares error.

A model class now correspond to assuming a class that the parameter $f(X)$ belongs to, e.g. $F_\gamma = \{f: f(x) = a_0 + a_1x + a_2x^2 + \dots + a_\gamma x^\gamma\}$ is the class of polynomials of degree γ , or $F_\gamma = \{f: f(x) = \sum_{i=1}^\gamma a_i \phi_i(x)\}$ where $\phi_i(x)$ are nonlinear features or basis functions such as fourier basis, wavelet basis, splines etc.

The two-stage MDL procedure for selecting the best model in a given class would be:

$$f_\gamma = \arg \min_{f \in F_\gamma} \left[L(f) + \frac{1}{n} \sum_{i=1}^n (Y_i - f(X_i))^2 \right] \quad (3.36)$$

This is precisely the regularized least squares framework.

We can also select the model class (e.g. order of the polynomial) in addition to selecting the best model within the class, adaptively using a three-step MDL approach:

$$\hat{\gamma} = \arg \min_{\gamma \in \Gamma} \left\{ L(\gamma) + \min_{f \in F_\gamma} \left[L(f) + \frac{1}{n} \sum_{i=1}^n (Y_i - f(X_i))^2 \right] \right\} \quad (3.37)$$

3.5 Bayes Optimal Classifier

The Bayes Optimal Classifier is a probabilistic model that predicts the most likely outcome for a new situation. In this blog, we'll have a look at Bayes optimal classifier and Naive Bayes Classifier.

The Bayes theorem is a method for calculating a hypothesis's probability based on its prior probability, the probabilities of observing specific data given the hypothesis, and the seen data itself.

The Bayes Theorem, which provides a systematic means of computing a conditional probability, is used to describe it. It's also related to Maximum a Posteriori (MAP), a probabilistic framework for determining the most likely hypothesis for a training dataset.

Take a hypothesis space that has 3 hypotheses h_1 , h_2 , and h_3 .

The posterior probabilities of the hypotheses are as follows:

$h_1 \rightarrow 0.4$

$h_2 \rightarrow 0.3$

$h_3 \rightarrow 0.3$

Hence, h_1 is the MAP hypothesis. ($\text{MAP} \Rightarrow \text{max posterior}$)

Suppose a new instance x is encountered, which is classified negative by h_2 and h_3 but positive by h_1 .

Taking all hypotheses into account, the probability that x is positive is .4 and the probability that it is negative is therefore .6.

The classification generated by the MAP hypothesis is different from the most probable classification in this case which is negative.

The most probable classification of the new instance is obtained by combining the predictions of all hypotheses, weighted by their posterior probabilities.

If the new example's probable classification can be any value v_j from a set V , the probability $P(v_j/D)$ that the right classification for the new instance is v_j is merely

$$P(v_j | D) = \sum_{h_i \in H} P(v_j | h_i) P(h_i | D) \quad (3.38)$$

The denominator is omitted since we're only using this for comparison and all the values of $P(v_j/D)$ will have the same denominator.

The value v_j , for which $P(v_j/D)$ is maximum, is the best classification for the new instance.

Bayes optimal classification:

$$\operatorname{argmax}_{v_j \in V} \sum_{h_i \in H} P(v_j | h_i) P(h_i | D) \quad (3.39)$$

To illustrate in terms of the above example, the set of possible classifications of the new instance is $V = \{\oplus, \ominus\}$, and

$$\begin{aligned} P(h_1 | D) &= .4, P(\ominus | h_1) = 0, P(\oplus | h_1) = 1 \\ P(h_2 | D) &= .3, P(\ominus | h_2) = 1, P(\oplus | h_2) = 0 \\ P(h_3 | D) &= .3, P(\ominus | h_3) = 1, P(\oplus | h_3) = 0 \end{aligned} \quad (3.40)$$

therefore

$$\begin{aligned} \sum_{h_i \in H} P(\oplus | h_i) P(h_i | D) &= .4 \\ \sum_{h_i \in H} P(\ominus | h_i) P(h_i | D) &= .6 \end{aligned} \quad (3.41)$$

and

$$\operatorname{argmax}_{v_j \in \{\oplus, \ominus\}} \sum_{h_i \in H} P(v_j | h_i) P(h_i | D) = \ominus \quad (3.42)$$

A Bayes optimal classifier is a system that classifies new cases according to Equation. This strategy increases the likelihood that the new instance will be appropriately classified.

Consider an example for Bayes Optimal Classification,
Let there be 5 hypotheses h_1 through h_5 .

P(hi/D)	P(F/ hi)	P(L/hi)	P(R/hi)
0.4	1	0	0
0.2	0	1	0
0.1	0	0	1
0.1	0	1	0
0.2	0	1	0

Table. 3.1 Hypotheses h_1 .

The MAP theory, therefore, argues that the robot should proceed forward (F). Let's see what the Bayes optimal procedure suggests.

$$\begin{aligned}\sum_{h_i \in H} P(F | h_i)P(h_i | D) &= 0.4 \\ \sum_{h_i \in H} P(L | h_i)P(h_i | D) &= 0.2 + 0.1 + 0.2 = 0.5 \\ \sum_{h_i \in H} P(R | h_i)P(h_i | D) &= 0.1\end{aligned}\tag{3.43}$$

Thus, the Bayes optimal procedure recommends the robot turn left.

Naive Bayes Classifier

The Naive Bayes classifiers, which are a set of classification algorithms, are created using the Bayes' Theorem. 'Each pair of features categorized is independent of the others. Naive Bayes Classifier is a group of algorithms that all work on the above principle.

The naive Bayes classifier is useful for learning tasks in which each instance x is represented by a set of attribute values and the target function $f(x)$ can take any value from a finite set V .

A set of target function training examples is provided, as well as a new instance specified by the tuple of attribute values $(a_1, a_2, \dots, a_n)$.

The learner is given the task of estimating the goal value. The most likely target value V_{MAP} is assigned in the Bayesian strategy to classify the new instance.

$$v_{MAP} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j | a_1, a_2, \dots, a_n)\tag{3.44}$$

We can use Bayes theorem to rewrite this expression as

$$\begin{aligned}v_{MAP} &= \underset{v_j \in V}{\operatorname{argmax}} \frac{P(a_1, a_2, \dots, a_n | v_j)P(v_j)}{P(a_1, a_2, \dots, a_n)} \\ &= \underset{v_j \in V}{\operatorname{argmax}} P(a_1, a_2, \dots, a_n | v_j)P(v_j)\end{aligned}\tag{3.45}$$

Simply count the number of times each target value v_j appears in the training data to estimate each $P(v_j)$. The naive Bayes classifier is based on the simplifying assumption that the attribute values are conditionally independent given the target value.

In other words, the assumption is that given the target value of the instance, the probability of observing the conjunction $a_1, a_2, \dots, a_n$ is just the product of the probabilities for the individual attributes: $P(a_1, a_2, \dots, a_n | v_j) = \prod_i P(a_i | v_j)$. Substituting this Equation, we have the approach used by the naive Bayes classifier.

Naive Bayes classifier:

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_i P(a_i | v_j) \quad (3.46)$$

Where VNB stands for the Naive Bayes classifier's target value.

The naive Bayes learning approach includes a learning stage in which the different $P(v_j)$ and $P(a_i/v_j)$ variables are estimated using the training data's frequency distribution.

The learned hypothesis is represented by the set of these estimations. The basic Naive Bayes **assumption** is that each feature has the following effect: Contribution to the ultimate product that is both independent and equal.

Let's understand the concept of the Naive Bayes classifier better, with the help of an example.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Table. 3.2 Naive Bayes classifier Example.

Table 3.2 above shows 14 training instances of the goal concept PlayTennis, with the characteristics Outlook, Temperature, Humidity, and Wind describing each day. To categorize the following novel instance, we utilize the naive Bayes classifier and the training data from this table:

(Outlook = sunny, Temperature = cool, Humidity = high, Wind = strong)

Our task is to predict the target value (yes or no) of the target concept PlayTennis for this new instance. Instantiating Equation (6.20) to fit the current task, the target value v_{NB} is given by

$$\begin{aligned}
 v_{NB} &= \underset{v_j \in \{yes, no\}}{\operatorname{argmax}} P(v_j) \prod_i P(a_i | v_j) \\
 &= \underset{v_j \in \{yes, no\}}{\operatorname{argmax}} P(v_j) P \left(\begin{array}{l} \text{Outlook} = \text{sunny} | v_j \\ \text{Temperature} = \text{cool} | v_j \\ \text{Humidity} = \text{high} | v_j \\ \text{Wind} = \text{strong} | v_j \end{array} \right)
 \end{aligned} \tag{3.47}$$

First, based on the frequencies of the 14 training instances, the probability of the ' goal values may be easily determined.

$$\begin{aligned}
 P(\text{PlayTennis} = \text{yes}) &= 9/14 = .64 \\
 P(\text{PlayTennis} = \text{no}) &= 5/14 = .36
 \end{aligned}$$

We can also estimate conditional probabilities in the same way. Those for Wind = strong, for example, include

$$\begin{aligned}
 P(\text{Wind} = \text{strong} | \text{PlayTennis} = \text{yes}) &= 3/9 = .33 \\
 P(\text{Wind} = \text{strong} | \text{PlayTennis} = \text{no}) &= 3/5 = .60
 \end{aligned}$$

Using these probability estimates and similar estimates for the remaining attribute values, we calculate v_{NB} according to Equation (6.21) as follows (now omitting attribute names for brevity)

$$\begin{aligned}
 P(\text{yes})P(\text{sunny} | \text{yes})P(\text{cool} | \text{yes})P(\text{high} | \text{yes})P(\text{strong} | \text{yes}) &= .0053 \\
 P(\text{no})P(\text{sunny} | \text{no})P(\text{cool} | \text{no})P(\text{high} | \text{no})P(\text{strong} | \text{no}) &= .0206
 \end{aligned}$$

Based on the probability estimates learned from the training data, the naive Bayes classifier gives the goal value PlayTennis = no to this new occurrence. Furthermore, given the observed attribute values, we can determine the conditional probability that the target value is no by normalizing the above amounts to sum to one.

Types of Naïve Bayes Classifiers

There are three types of classifiers. They are

1. Gaussian Naïve Bayes
2. Multinomial Naïve Bayes
3. Bernoulli Naïve Bayes

Gaussian Naïve Bayes

Gaussian Naïve Bayes is useful when we are working with continuous values whose probabilities can be modeled using a Gaussian distribution.

$$P(x) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (3.48)$$

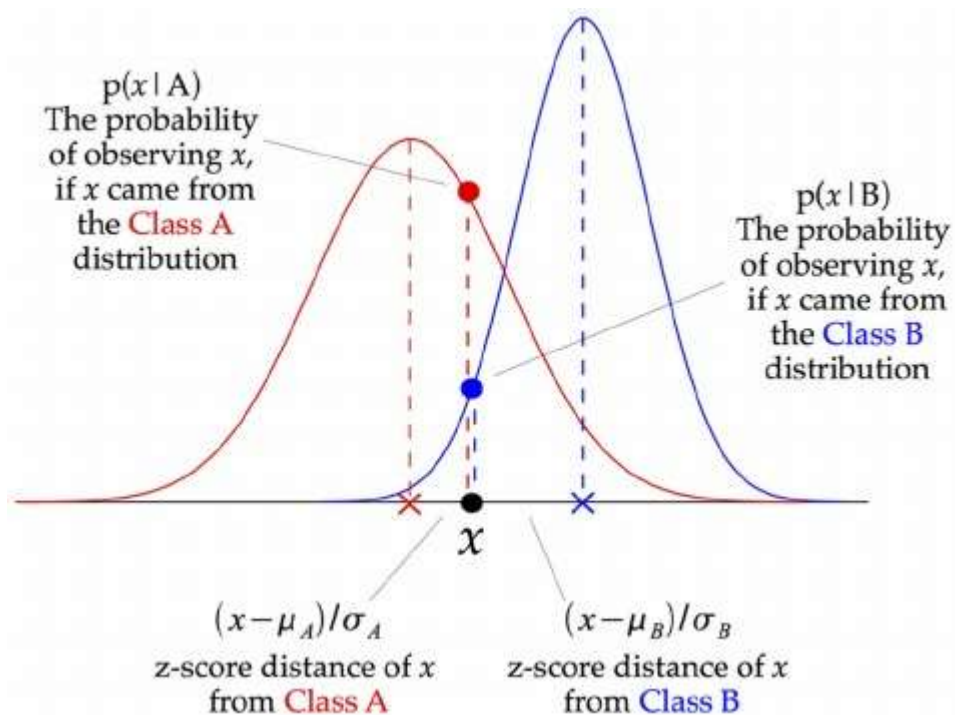


Fig. 3.6 Gaussian Naïve Bayes.

Multinomial naive Bayes

A multinomial distribution is helpful to model feature vectors where each value represents like the number of occurrences of a term or its relative frequency. If the feature vectors have n elements and each element of them can assume k different values with probability p_k , then

$$P(X_1 = x_1 \cap X_2 = x_2 \cap \dots \cap X_k = x_k) = \frac{n!}{\prod_i x_i!} \prod_i p_i^{x_i} \quad (3.49)$$

Bernoulli naive Bayes

If X is a random variable which is Bernoulli-distributed, it assumes only two values and their probability is given as follows

$$P(X) = \begin{cases} p & \text{if } X = 1 \\ q & \text{if } X = 0 \end{cases} \quad (3.50)$$

where $q = 1 - p$ and $0 < p < 1$

3.6 Bayesian Belief Network

Bayesian belief network is key computer technology for dealing with probabilistic events and to solve a problem which has uncertainty. We can define a Bayesian network as:

A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph."

It is also called a **Bayes network**, **belief network**, **decision network**, or **Bayesian model**.

Bayesian networks are probabilistic, because these networks are built from a **probability distribution**, and also use probability theory for prediction and anomaly detection.

Real world applications are probabilistic in nature, and to represent the relationship between multiple events, we need a Bayesian network. It can also be used in various tasks including **prediction**, **anomaly detection**, **diagnostics**, **automated insight**, **reasoning**, **time series prediction**, and **decision making under uncertainty**.

Bayesian Network can be used for building models from data and experts opinions, and it consists of two parts:

- ❖ **Directed Acyclic Graph**
- ❖ **Table of conditional probabilities.**

The generalized form of Bayesian network that represents and solve decision problems under uncertain knowledge is known as an **Influence diagram**.

A Bayesian network graph is made up of nodes and Arcs (directed links), where:

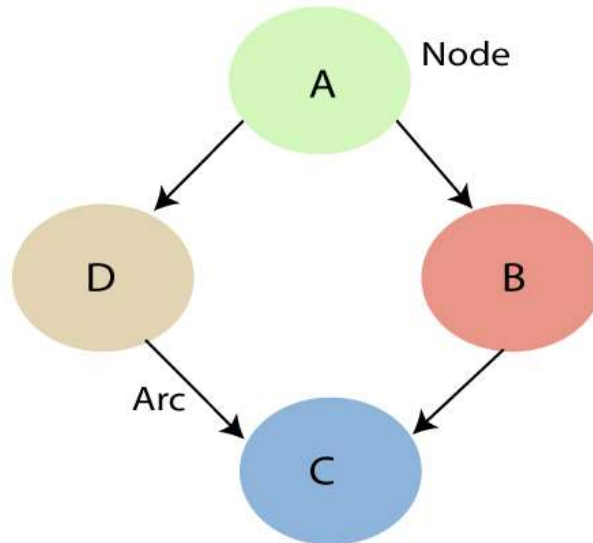


Fig. 3.7 Bayesian network graph.

- ❖ Each **node** corresponds to the random variables, and a variable can be **continuous** or **discrete**.
- ❖ **Arc or directed arrows** represent the causal relationship or conditional probabilities between random variables. These directed links or arrows connect the pair of nodes in the graph.

These links represent that one node directly influence the other node, and if there is no directed link that means that nodes are independent with each other

1. In the above diagram, A, B, C, and D are random variables represented by the nodes of the network graph.
2. If we are considering node B, which is connected with node A by a directed arrow, then node A is called the parent of Node B.
3. Node C is independent of node A.

Note: The Bayesian network graph does not contain any cyclic graph. Hence, it is known as a directed acyclic graph or DAG

The Bayesian network has mainly two components:

- ❖ **Causal Component**
- ❖ **Actual numbers**

Each node in the Bayesian network has condition probability distribution $P(X_i | \text{Parent}(X_i))$, which determines the effect of the parent on that node.

Bayesian network is based on Joint probability distribution and conditional probability. So let's first understand the joint probability distribution:

Joint probability distribution:

If we have variables $x_1, x_2, x_3, \dots, x_n$, then the probabilities of a different combination of $x_1, x_2, x_3 \dots x_n$, are known as Joint probability distribution.

$P[x_1, x_2, x_3, \dots, x_n]$, it can be written as the following way in terms of the joint probability distribution.

$$= P[x_1 | x_2, x_3, \dots, x_n] P[x_2, x_3, \dots, x_n]$$

$$= P[x_1 | x_2, x_3, \dots, x_n] P[x_2 | x_3, \dots, x_n] \dots P[x_{n-1} | x_n] P[x_n].$$

In general for each variable X_i , we can write the equation as:

$$P(X_i | X_{i-1}, \dots, X_1) = P(X_i | \text{Parents}(X_i))$$

How Bayesian Belief Network Applied in Machine Learning?

Machine learning is all about probability, making the Bayesian Belief Network applicable to more than a few aspects of machine learning as a whole. BBN's primary role in machine learning is to visualize the probability model for any given domain, assess and reason probabilities for any given scenario with factual data and evidence, and oversee the connection between the many random variables in any given situation.

These networks offer a visual probability model, which is expressed through a graph that's defined by data nodes and orientated edges. The BBN determines the dependencies and dependencies between selecting given variables in any given sheet of data through this probability model, allowing for clarification, estimations, and decision-making in the machine learning process.

Each domain variable is defined by its node, and each node is defined through arcs. The nodes in question are all quantified by a conditional probability distribution table, which is a prearranged set of integrated random variables.

The BBN models are the defining factor of the theorem itself, and their preparation defines the success rate of the theorem itself. The more data, the better the data, and the more data derivatives, the better the BBN probability event estimation.

The Bayesian Belief Network is instrumental in machine learning, as it substantiates almost every step of the way, which includes data pre-processing, actual learning, and post-processing. The BBN is used in the unsupervised construction of deep neural networks through structure learning, which is directly enabled by applying the generative graph through Bayes' theorem. The algorithmic nature of this application makes it suitable for machine learning – and through the improved structuring of data, BBN plays an even more prominent role in the machine learning process.

Explanation of Bayesian network:

Let's understand the Bayesian network through an example by creating a directed acyclic graph:

Example: Harry installed a new burglar alarm at his home to detect burglary. The alarm reliably responds at detecting a burglary but also responds for minor earthquakes. Harry has two neighbors David and Sophia, who have taken a responsibility to inform Harry at work when they hear the alarm. David always calls Harry when he hears the alarm, but sometimes he got confused with the phone ringing and calls at that time too. On the other hand, Sophia likes to listen to high music, so sometimes she misses to hear the alarm. Here we would like to compute the probability of Burglary Alarm.

Problem:

Calculate the probability that alarm has sounded, but there is neither a burglary, nor an earthquake occurred, and David and Sophia both called the Harry.

Solution:

- ❖ The Bayesian network for the above problem is given below. The network structure is showing that burglary and earthquake is the parent node of the alarm and directly affecting the probability of alarm's going off, but David and Sophia's calls depend on alarm probability.
- ❖ The network is representing that our assumptions do not directly perceive the burglary and also do not notice the minor earthquake, and they also not confer before calling.
- ❖ The conditional distributions for each node are given as conditional probabilities table or CPT.

Mastering machine learning with Python practical with tips and technique

- ❖ Each row in the CPT must be sum to 1 because all the entries in the table represent an exhaustive set of cases for the variable.
- ❖ In CPT, a boolean variable with k boolean parents contains 2^k probabilities. Hence, if there are two parents, then CPT will contain 4 probability values

List of all events occurring in this network:

1. Burglary (B)
2. Earthquake(E)
3. Alarm(A)
4. David Calls(D)
5. Sophia calls(S)

We can write the events of problem statement in the form of probability: $P[D, S, A, B, E]$, can rewrite the above probability statement using joint probability distribution:

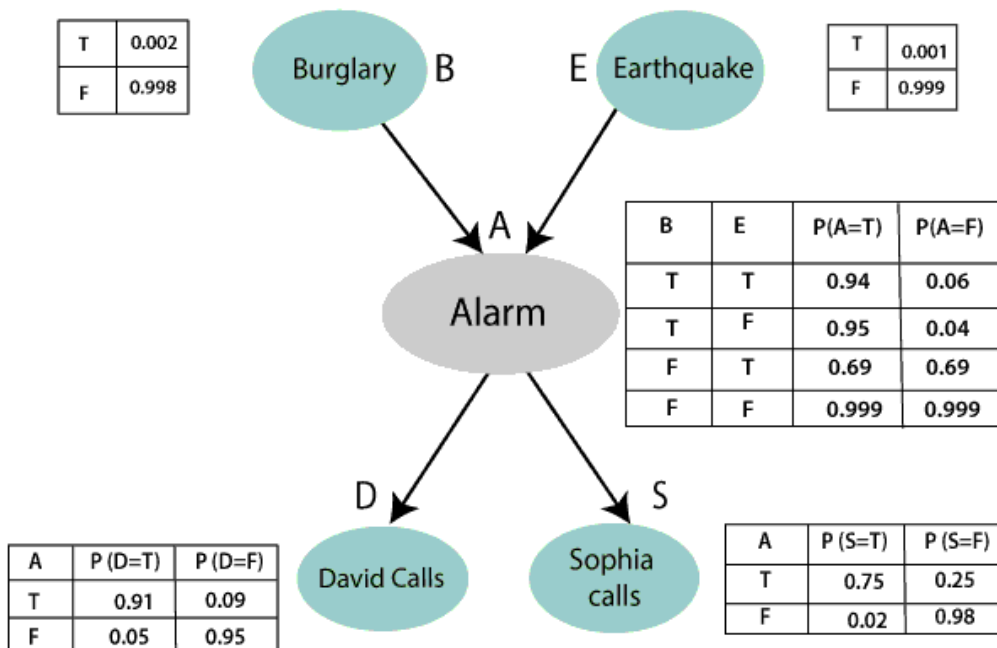
$$P[D, S, A, B, E] = P[D | S, A, B, E]. P[S, A, B, E]$$

$$= P[D | S, A, B, E]. P[S | A, B, E]. P[A, B, E]$$

$$= P[D | A]. P[S | A, B, E]. P[A, B, E]$$

$$= P[D | A]. P[S | A]. P[A | B, E]. P[B, E]$$

$$= P[D | A]. P[S | A]. P[A | B, E]. P[B | E]. P[E]$$



Mastering machine learning with Python practical with tips and technique

Let's take the observed probability for the Burglary and earthquake component:

$P(B = \text{True}) = 0.002$, which is the probability of burglary.

$P(B = \text{False}) = 0.998$, which is the probability of no burglary.

$P(E = \text{True}) = 0.001$, which is the probability of a minor earthquake

$P(E = \text{False}) = 0.999$, Which is the probability that an earthquake not occurred.

We can provide the conditional probabilities as per the below tables:

Conditional probability table for Alarm A:

The Conditional probability of Alarm A depends on Burglar and earthquake:

B	E	P(A)=True	P(A)=False
True	True	0.94	0.06
True	False	0.95	0.04
False	True	0.31	0.69
False	False	0.001	0.999

Conditional probability table for David Calls:

The Conditional probability of David that he will call depends on the probability of Alarm.

A	P(D=True)	P(D=False)
True	0.91	0.09
False	0.05	0.95

Conditional probability table for Sophia Calls:

The Conditional probability of Sophia that she calls is depending on its Parent Node "Alarm."

A	P(S=True)	P(S=False)
True	0.75	0.25
False	0.02	0.98

From the formula of joint distribution, we can write the problem statement in the form of probability distribution:

$$P(S, D, A, \neg B, \neg E) = P(S|A) * P(D|A) * P(A|\neg B \wedge \neg E) * P(\neg B) * P(\neg E).$$

$$= 0.75 * 0.91 * 0.001 * 0.998 * 0.999$$

$$= 0.00068045.$$

Hence, a Bayesian network can answer any query about the domain by using Joint distribution.

The semantics of Bayesian Network:

There are two ways to understand the semantics of the Bayesian network, which is given below:

1. To understand the network as the representation of the Joint probability distribution.

It is helpful to understand how to construct the network.

2. To understand the network as an encoding of a collection of conditional independence statements.

3.7 EM Algorithm

The EM algorithm is considered a latent variable model to find the local maximum likelihood parameters of a statistical model, proposed by Arthur Dempster, Nan Laird, and Donald Rubin in 1977. The EM (Expectation-Maximization) algorithm is one of the most commonly used terms in machine learning to obtain maximum likelihood estimates of variables that are sometimes observable and sometimes not. However, it is also applicable to unobserved data or sometimes called latent. It has various real-world applications in statistics, including obtaining the mode of the posterior marginal distribution of parameters in machine learning and data mining applications.

In most real-life applications of machine learning, it is found that several relevant learning features are available, but very few of them are observable, and the rest are unobservable. If the variables are observable, then it can predict the value using instances. On the other hand, the variables which are latent or directly not observable, for such variables Expectation-Maximization (EM) algorithm plays a vital role to predict the value with the condition that the general form of probability distribution governing those latent variables is known to us. In this topic, we will discuss a basic introduction to the EM algorithm, a flow chart of the EM algorithm, its applications, advantages, and disadvantages of EM algorithm, etc.

What is an EM algorithm?

The Expectation-Maximization (EM) algorithm is defined as the combination of various unsupervised machine learning algorithms, which is used to determine the **local maximum likelihood estimates (MLE)** or **maximum a posteriori estimates (MAP)** for unobservable variables in statistical models. Further, it is a technique to find maximum likelihood estimation when the latent variables are present. It is also referred to as the **latent variable model**.

A latent variable model consists of both observable and unobservable variables where observable can be predicted while unobserved are inferred from the observed variable. These unobservable variables are known as latent variables.

Key Points:

- ❖ It is known as the latent variable model to determine MLE and MAP parameters for latent variables.
- ❖ It is used to predict values of parameters in instances where data is missing or unobservable for learning, and this is done until convergence of the values occurs.

EM Algorithm

The EM algorithm is the combination of various unsupervised ML algorithms, such as the **k-means clustering algorithm**. Being an iterative approach, it consists of two modes. In the first mode, we estimate the missing or latent variables. Hence it is referred to as the **Expectation/estimation step (E-step)**. Further, the other mode is used to optimize the parameters of the models so that it can explain the data more clearly. The second mode is known as the **maximization-step or M-step**.

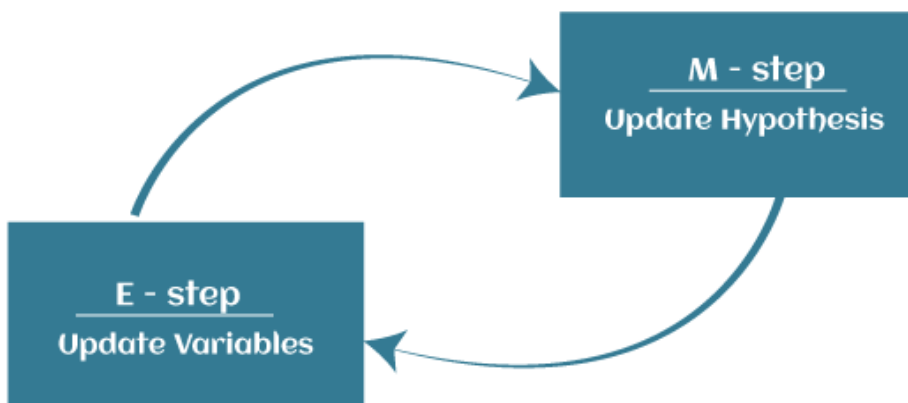


Fig. 3.8 EM Algorithm.

- ❖ **Expectation step (E - step):** It involves the estimation (guess) of all missing values in the dataset so that after completing this step, there should not be any missing value.
- ❖ **Maximization step (M - step):** This step involves the use of estimated data in the E-step and updating the parameters.
- ❖ **Repeat E-step and M-step** until the convergence of the values occurs.

The primary goal of the EM algorithm is to use the available observed data of the dataset to estimate the missing data of the latent variables and then use that data to update the values of the parameters in the M-step.

What is Convergence in the EM algorithm?

Convergence is defined as the specific situation in probability based on intuition, e.g., if there are two random variables that have very less difference in their probability, then they are known as converged. In other words, whenever the values of given variables are matched with each other, it is called convergence.

Steps in EM Algorithm

The EM algorithm is completed mainly in 4 steps, which include **Initialization Step**, **Expectation Step**, **Maximization Step**, and **convergence Step**. These steps are explained as follows:

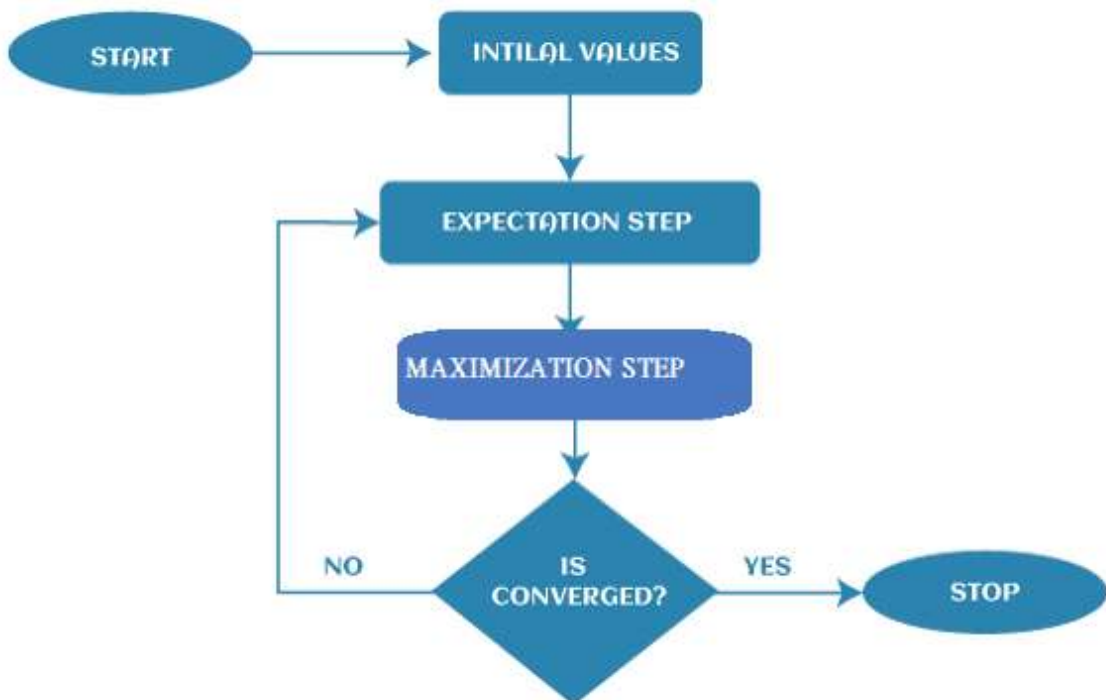


Fig. 3.9 Steps in EM Algorithm.

- ❖ **1st Step:** The very first step is to initialize the parameter values. Further, the system is provided with incomplete observed data with the assumption that data is obtained from a specific model.
- ❖ **2nd Step:** This step is known as Expectation or E-Step, which is used to estimate or guess the values of the missing or incomplete data using the observed data. Further, E-step primarily updates the variables.
- ❖ **3rd Step:** This step is known as Maximization or M-step, where we use complete data obtained from the 2nd step to update the parameter values. Further, M-step primarily updates the hypothesis.
- ❖ **4th step:** The last step is to check if the values of latent variables are converging or not. If it gets "yes", then stop the process; else, repeat the process from step 2 until the convergence occurs.

Gaussian Mixture Model (GMM)

The Gaussian Mixture Model or GMM is defined as a mixture model that has a combination of the unspecified probability distribution function. Further, GMM also requires estimated statistics values such as mean and standard deviation or parameters. It is used to estimate the parameters of the probability distributions to best fit the density of a given training dataset. Although there are plenty of techniques available to estimate the parameter of the Gaussian Mixture Model (GMM), the **Maximum Likelihood Estimation** is one of the most popular techniques among them.

The processes used to generate the data point represent a latent variable or unobservable data. In such cases, the Estimation-Maximization algorithm is one of the best techniques which helps us to estimate the parameters of the gaussian distributions. In the EM algorithm, E-step estimates the expected value for each latent variable, whereas M-step helps in optimizing them significantly using the Maximum Likelihood Estimation (MLE). Further, this process is repeated until a good set of latent values, and a maximum likelihood is achieved that fits the data.

Applications of EM algorithm

The primary aim of the EM algorithm is to estimate the missing data in the latent variables through observed data in datasets. The EM algorithm or latent variable model has a broad range of real-life applications in machine learning. These are as follows:

- ❖ The EM algorithm is applicable in data clustering in machine learning.
- ❖ It is often used in computer vision and NLP (Natural language processing).
- ❖ It is used to estimate the value of the parameter in mixed models such as the **Gaussian Mixture Model** and quantitative genetics.

- ❖ It is also used in psychometrics for estimating item parameters and latent abilities of item response theory models.
- ❖ It is also applicable in the medical and healthcare industry, such as in image reconstruction and structural engineering.
- ❖ It is used to determine the Gaussian density of a function.

Advantages of EM algorithm

- ❖ It is very easy to implement the first two basic steps of the EM algorithm in various machine learning problems, which are E-step and M- step.
- ❖ It is mostly guaranteed that likelihood will enhance after each iteration.
- ❖ It often generates a solution for the M-step in the closed form.

Disadvantages of EM algorithm

- ❖ The convergence of the EM algorithm is very slow.
- ❖ It can make convergence for the local optima only.
- ❖ It takes both forward and backward probability into consideration. It is opposite to that of numerical optimization, which takes only forward probabilities.

3.8 Probabilistic Model

Probabilistic models are fundamental to machine learning, as they provide a statistical framework for capturing uncertainty in data and making predictions about future observations. By learning patterns from historical data, probabilistic models allow us to infer the likelihood of different outcomes, even in situations involving complex relationships and unknown factors. These models are widely used in applications such as image and speech recognition, natural language processing, recommendation systems, and more.

In recent years, advances in computational power and algorithm design have made it possible to develop probabilistic models that efficiently handle large and complex datasets. This progress has further solidified probabilistic models as essential tools for understanding data patterns and making informed decisions under uncertainty.

Categories of Probabilistic Models

Probabilistic models can be broadly classified into three main types:

1. **Generative Models**
2. **Discriminative Models**

3. Graphical Models

Each of these categories has unique characteristics and is suited to specific types of tasks.

1. Generative Models

Generative models aim to learn the **joint distribution** of input and output variables, meaning they model the complete probability distribution over the data. By learning this joint distribution, generative models can be used to generate new data points that resemble the original dataset. This ability to create new, plausible data points makes generative models highly versatile and valuable for tasks like image and speech synthesis, language translation, and text generation.

Popular examples of generative models include:

- ❖ **Gaussian Mixture Models (GMMs):** These models represent data as a mixture of multiple Gaussian distributions, capturing complex patterns in the data.
- ❖ **Variational Autoencoders (VAEs):** These neural networks learn compressed representations of data and can generate new samples by sampling from a learned latent space.
- ❖ **Generative Adversarial Networks (GANs):** GANs consist of two neural networks—a generator and a discriminator—that compete, resulting in realistic generated samples.

Generative models are particularly useful in unsupervised learning tasks and scenarios where generating new data samples is required.

2. Discriminative Models

Discriminative models focus on modeling the **conditional distribution** of the output variable given the input variable. Rather than learning the joint distribution, discriminative models aim to establish a clear boundary or decision rule between different classes, making them well-suited for classification tasks where accurate predictions are the primary goal. Discriminative models do not generate new data; instead, they focus on identifying patterns that distinguish between classes based on input features.

Common examples of discriminative models include:

- ❖ **Logistic Regression:** This model predicts the probability of a binary outcome and is widely used for classification tasks.
- ❖ **Support Vector Machines (SVMs):** SVMs find the optimal hyperplane that separates data points of different classes with maximum margin.

- ❖ **Neural Networks:** In classification tasks, neural networks can be trained as discriminative models, learning complex decision boundaries for high-dimensional data.

Discriminative models are commonly applied in fields such as image recognition, speech recognition, and sentiment analysis, where the primary objective is to classify data points rather than generate new ones.

3. Graphical Models

Graphical models represent probabilistic relationships between variables using graph structures, capturing dependencies and conditional relationships between them. These models allow for a structured approach to represent complex dependencies, making them highly effective for applications involving reasoning, causality, and multi-variable dependencies.

Examples of graphical models include:

- ❖ **Bayesian Networks:** These are directed acyclic graphs where nodes represent variables and edges represent conditional dependencies.
- ❖ **Markov Random Fields (MRFs):** MRFs are undirected graphical models used to capture dependencies between variables without specific directional influences.

Graphical models are widely used in tasks like image recognition, natural language processing, and causal inference, as they can model complex dependencies and provide interpretable results.

Naive Bayes Algorithm in Probabilistic Models

The **Naive Bayes algorithm** is a simple yet powerful probabilistic approach frequently used for classification. It leverages Bayes' theorem to calculate the probability of each class given a set of features, assuming that the features are conditionally independent of each other given the class label. This independence assumption simplifies the computation, making the Naive Bayes algorithm highly efficient.

The Naive Bayes algorithm follows these steps:

1. **Data Collection:** Collect a labeled dataset where each sample includes a set of features and a class label.
2. **Calculate Conditional Probabilities:** For each feature, compute the probability of observing that feature given a particular class. This is achieved by counting occurrences within the class and dividing by the total occurrences of the class.

3. **Calculate Prior Probabilities:** For each class, calculate its prior probability by dividing the number of samples in that class by the total number of samples in the dataset.
4. **Compute Posterior Probabilities:** Given a new sample, use Bayes' theorem to calculate the posterior probability of each class by combining the conditional and prior probabilities.
5. **Class Prediction:** Assign the class with the highest posterior probability to the new sample.

Despite its simplicity, the Naive Bayes algorithm is widely used for applications such as spam filtering, medical diagnosis, and sentiment analysis. Its computational efficiency and effectiveness in high-dimensional datasets make it a popular choice, especially in cases where the independence assumption holds reasonably well.

Probabilistic Models in Deep Learning

In **deep learning**, probabilistic models play a crucial role in optimizing complex neural networks. These models incorporate uncertainty directly into the learning process, allowing deep learning algorithms to better generalize on unseen data. Probabilistic techniques are especially important in deep learning tasks where complex relationships and large amounts of data are involved.

One widely used probabilistic technique in deep learning is **variational inference**. Variational inference provides an efficient way to estimate posterior distributions in models with a large number of parameters, such as deep neural networks. By approximating the posterior distribution, variational inference enables deep learning models to make more reliable predictions and handle uncertainty effectively.

Importance of Probabilistic Models

Probabilistic models provide a systematic way to handle uncertainty in data, making them essential for many machine learning applications. They offer a framework for understanding the likelihood of various outcomes, which is valuable in decision-making processes where certainty cannot be guaranteed. Probabilistic models also support **Bayesian inference**, a powerful tool for updating beliefs about a hypothesis as new data is observed. This is particularly useful when making decisions under uncertainty, such as in finance, medicine, and scientific research.

Advantages of Probabilistic Models

Probabilistic models have several key advantages:

1. **Incorporation of Uncertainty:** By modeling uncertainty explicitly, probabilistic models allow for more accurate predictions and better decision-making, especially in complex or unpredictable scenarios.
2. **Interpretable Insights:** Probabilistic models can reveal how different factors influence outcomes, providing insights into patterns and relationships within data.
3. **Flexibility and Adaptability:** These models can be applied across various fields, such as artificial intelligence, finance, and healthcare, to solve a wide range of problems.

In domains like finance, probabilistic models help assess risks; in healthcare, they aid in predicting patient outcomes, and in AI, they improve system reliability by quantifying the uncertainty of predictions.

Disadvantages of Probabilistic Models

While powerful, probabilistic models also have limitations:

1. **Risk of Overfitting:** Probabilistic models can be prone to overfitting, especially when trained on small datasets. Overfitting occurs when the model becomes too specific to the training data and performs poorly on new data.
2. **Not Universally Applicable:** Some data may not fit well into a probabilistic framework, limiting the effectiveness of probabilistic models in certain applications.
3. **High Computational Cost:** Many probabilistic models require substantial computational resources, particularly for large or complex datasets. Developing and implementing these models can be time-consuming and resource-intensive.

3.9 Sample Complexity

Sample complexity is a key concept in machine learning that determines the number of data samples required for an algorithm to reach a specific level of performance. It is essential for evaluating the efficiency of learning algorithms, as a more efficient algorithm will need fewer samples to achieve reliable learning outcomes. This efficiency reduces the resources needed for data acquisition and storage, which can be a significant consideration in large-scale projects.

In essence, sample complexity addresses the question: "**How much data do I need?**" This requirement can vary greatly based on the complexity of the problem, the diversity within the data, and the precision needed in the model's predictions.

Types of Sample Complexity

Sample complexity is often categorized into two main types:

1. **Worst-case Sample Complexity:** This refers to the maximum number of samples needed to achieve a certain level of learning, regardless of the distribution of data. It represents the upper bound on data requirements, ensuring that the algorithm performs adequately in the most challenging situations.
2. **Average-case Sample Complexity:** This considers the average number of samples required, assuming the data follows a particular distribution. It gives a more typical estimate, based on the likelihood that the data has certain characteristics.

Why Sample Complexity Matters

Understanding sample complexity is valuable for data scientists and machine learning engineers as it helps them assess whether they have sufficient data for effective model training. Sample complexity considerations are important when gathering data, selecting learning algorithms, and evaluating model performance. By understanding the data requirements, engineers can avoid issues related to underfitting or overfitting and ensure that their models have a strong foundation for generalization.

Technical Explanation of Sample Complexity

To grasp sample complexity at a deeper level, one must consider certain concepts from **statistical learning theory**, which provides the mathematical foundation for sample complexity.

VC Dimension

A crucial concept here is the **Vapnik-Chervonenkis (VC) dimension**, which measures a model's capacity or complexity. The VC dimension indicates the maximum extent to which a model can 'memorize' data points, which is closely related to the model's ability to generalize on new data. A high VC dimension suggests that the model is complex, likely requiring a larger sample size to prevent overfitting and ensure effective learning.

PAC Learning Theory

The **Probably Approximately Correct (PAC) learning theory** links VC dimension with sample complexity. PAC learning theory aims to determine the minimum number of samples required to produce a hypothesis that is, with high probability, within a specified error tolerance of the best possible hypothesis.

Essentially, PAC learning seeks to identify the number of samples necessary to create a model that is **probably** (with high confidence) **approximately correct** (within a defined error margin).

The PAC learning bound is typically expressed as follows:

$$N \geq \frac{1}{\epsilon} \cdot \left(\ln |H| + \ln \frac{1}{\delta} \right)$$

(3.51)

where:

- ❖ N represents the sample size,
- ❖ ϵ is the maximum allowable error (representing "approximately correct"),
- ❖ $|H|$ is the size of the hypothesis space, often related to the VC dimension, and
- ❖ δ is the acceptable failure probability (the "probably" component).

This formula shows that sample complexity (N) increases with the complexity of the model (as indicated by $|H|$ or VC dimension) and the desired level of accuracy (smaller ϵ), while it decreases if a higher tolerance for error (larger δ) is acceptable.

Generalization Error

Another concept related to sample complexity is **generalization error**, which measures the difference between a model's performance on the training data and its expected performance on new data. A model with a high generalization error typically has high sample complexity, as it requires more data to accurately learn without overfitting.

In summary, sample complexity depends on the model's complexity (VC dimension), acceptable error tolerance (ϵ), failure probability (δ), and generalization error. These factors collectively shape our understanding of sample complexity in machine learning.

Sample Complexity Across Different Types of Machine Learning

Sample complexity is relevant to all types of machine learning algorithms, but its implications vary across different learning paradigms:

- **Supervised Learning:** In supervised learning, where models learn from labeled data, sample complexity can be reduced by acquiring a diverse and representative set of samples. Representative data ensures the model captures the range of patterns needed for accurate predictions.

- **Unsupervised Learning:** In unsupervised learning, which does not rely on labeled data, the lack of explicit guidance typically results in higher sample complexity. The model often requires a larger sample size to uncover meaningful patterns within the data.
- **Reinforcement Learning:** In reinforcement learning, sample complexity is influenced not only by the number of samples but also by the quality and variety of situations the agent encounters. Effective learning depends on exposure to a range of states and actions, which may require a larger sample size to cover.
- **Semi-Supervised Learning:** In semi-supervised learning, which combines labeled and unlabeled data, sample complexity is often determined by the ratio of labeled to unlabeled data. A larger number of labeled samples generally reduces sample complexity, as labeled data provides clearer guidance for model learning.

Real-World Applications of Sample Complexity Management

In practical applications, understanding and managing sample complexity can significantly impact resource allocation and model performance:

1. **Recommendation Systems (e.g., Netflix):** A company like Netflix uses machine learning models to recommend movies to users. A model with high sample complexity would require vast amounts of viewing data to make accurate recommendations, increasing data storage and processing costs. A model with lower sample complexity could achieve reasonable accuracy with fewer data, optimizing resources.
2. **Medical Diagnostics:** In the medical field, collecting data can be time-consuming and costly. A diagnostic model with lower sample complexity would require fewer patient records to make accurate diagnoses, making it more feasible and efficient in clinical settings.

Estimating Sample Complexity in Practice

Estimating sample complexity for real-world scenarios is often challenging and varies depending on the problem, data, and model type. Here are some general steps for estimating sample complexity:

1. **Assess Problem and Model Complexity:** The complexity of the problem and the chosen model are key factors. For example, complex models like deep neural networks tend to have high VC dimensions, requiring more data to achieve accurate results without overfitting.

2. **Empirical Testing:** Start with a small dataset and gradually increase its size, monitoring the model's performance. The point at which performance improvements plateau can provide an indication of the necessary sample size.
3. **PAC Learning Bounds:** Use PAC learning bounds for a theoretical estimate of sample complexity. Although these bounds can be overly loose for practical use, they provide a rough guideline for data requirements.
4. **Consider Model Complexity:** The complexity of the model (e.g., number of parameters in a neural network, depth of a decision tree) is tied to sample complexity. Higher complexity models typically need more data. Learning curves can help visualize this relationship by showing how performance changes with additional data.
5. **Account for Data Variability:** High variability in the data often necessitates a larger sample size. For instance, in an image recognition model with highly diverse images, more data may be needed to ensure the model learns a generalizable pattern.
6. **Use Bootstrapping Techniques:** Bootstrapping, a resampling technique, can help estimate sample complexity by creating subsets of data and assessing model performance on each subset. This can indicate the amount of data required for effective learning.
7. **Leverage Tools and Libraries:** Libraries like **scikit-learn** provide tools, such as the `learning_curve` function, to visualize performance across different training set sizes, offering insights into sample complexity requirements.

Estimating sample complexity involves balancing the available resources, model complexity, data diversity, and desired performance.

Benefits of Measuring Sample Complexity

Measuring sample complexity offers several advantages:

- ❖ **Guides Data Collection:** It helps estimate the amount of data required, reducing the risk of underfitting or overfitting.
- ❖ **Efficient Resource Allocation:** By understanding sample requirements, unnecessary data collection and storage can be avoided.
- ❖ **Algorithm Selection:** Sample complexity allows for comparisons between different algorithms' data efficiency, aiding in choosing the most suitable algorithm for a given problem.

Challenges in Managing Sample Complexity

Despite its advantages, managing sample complexity poses some challenges:

- ❖ **Estimation Complexity:** Accurately estimating sample complexity requires in-depth knowledge of the problem, model, and data, which may not always be readily available.
- ❖ **Assumption of More Data Equals Better Performance:** More data does not always improve performance, especially if the data is noisy or irrelevant.
- ❖ **Variability Across Algorithms and Data:** Different algorithms and data distributions can drastically affect sample complexity, making it difficult to generalize sample requirements.

Why Machine Learning Engineers Often Overlook Sample Complexity

While sample complexity is crucial for understanding data needs, it is often overlooked by machine learning engineers for several reasons:

1. **Access to Large Datasets:** With large datasets readily available, sample complexity becomes less of a concern as models can be trained on millions of examples to improve performance.
2. **Emphasis on Model Performance:** Engineers often prioritize metrics like accuracy or F1 score over sample complexity, focusing on achieving optimal performance rather than minimizing data usage.
3. **Limited Knowledge:** Some engineers may not have in-depth knowledge of sample complexity and its theoretical foundations.
4. **Availability of Pre-trained Models:** Pre-trained models enable engineers to achieve high performance with relatively small datasets, reducing the need to consider sample complexity in detail.

Although sample complexity is often overlooked, it may gain importance as data becomes scarce or costly in specific fields, and as models continue to grow in size, requiring efficient use of data.

3.10 Mistake Bound Model

The **Mistake Bound (MB) model** is an influential learning framework in machine learning, primarily designed for **analyzing online learning algorithms**. Unlike traditional batch learning methods where an algorithm is trained on a full dataset at once, the MB model is oriented around a **sequential learning approach**. This means that the algorithm receives data examples one at a time, makes predictions, and adjusts based on mistakes. The model's purpose is to quantify how many mistakes an algorithm will make before it successfully learns to predict without errors, focusing on minimizing mistakes during the learning process.

- ❖ Algorithm A has mistake-bound M for learning class C if A makes at most M mistakes on any sequence that is consistent with a function in C .

1. Weighted Majority Algorithm :

- ❖ A classifier combination method
- ❖ Takes a weighted vote among a pool of prediction algorithms, e.g., alternative hypotheses in H , or alternative learning algorithms
- ❖ It begins by weighting each algorithm by 1
- ❖ Whenever an algorithm misclassifies its weight is decreased by β , where $0 < \beta < 1$
- ❖ If A is any set of n prediction algorithms
- ❖ If k is the minimum number of mistakes made by any algorithm in A
- ❖ The number of mistakes made over any training sequence is at most

2. Halving Algorithm :

- ❖ The Halving Algorithm takes the majority vote of consistent experts up to the current round. Formally, denote the set of consistent experts up to round t as C_t ,

$$C_t = \{i: f_i^T = y_r, \forall r = 1, \dots, t-1\} \quad (3.52)$$

- ❖ On round t , the Halving Algorithm makes its prediction as

$$\hat{y}_t = \begin{cases} 1, & \text{if } \left| \{i \in C_t: f_i^t = 1\} \right| \geq \frac{|C_t|}{2}; \\ -1, & \text{otherwise.} \end{cases} \quad (3.53)$$

- ❖ If there exists a perfect expert i such that $f_i^t = y_t$ for all $t = 1, \dots, T$, then $\text{\#mistakes}[\text{Halving}] \leq \log_2 N$
- ❖ The Halving Algorithm works well if there exists a perfect expert, which we call a noise-free setting. What if the best expert makes a few mistakes? This is a noisy setting, and we need a better algorithm than majority vote.

Key Concepts of the Mistake Bound Model

1. **Sequential Learning:** In the MB model, an algorithm encounters examples in a sequence rather than all at once. For each incoming example, it must predict the label (or output) without having previously seen the entire dataset.

2. This sequential prediction aspect is critical for applications where data continuously arrives, such as in real-time systems.
3. **Prediction and Feedback Loop:** When the algorithm makes a prediction, it immediately receives feedback on whether the prediction was correct. If it made a mistake, the algorithm updates its internal model to avoid making the same mistake in the future.
4. **Mistake Count:** Every time the algorithm makes an incorrect prediction, it counts as a "mistake." The **goal is to minimize the total number of mistakes** as the algorithm progresses through the examples. The MB model helps analyze this performance aspect by providing a theoretical **upper bound** on the mistakes, called the **mistake bound**.
5. **Mistake Bound:** The mistake bound is the theoretical maximum number of mistakes an algorithm will make before it fully learns the target concept and no longer makes any mistakes. If an algorithm has a mistake bound of "M," it means it will make, at most, "M" errors before it reaches a consistent prediction accuracy on all subsequent examples.

Importance and Applications of the Mistake Bound Model

The MB model is particularly important in **online learning** and **adversarial learning** settings, where:

- ❖ **Data streams are continuous** and the system must make immediate predictions, such as in stock trading, network security, and adaptive filtering in communication systems.
- ❖ **Adaptation** to changing data patterns is crucial. For instance, in recommendation systems or fraud detection, new patterns emerge, and an algorithm needs to adapt its predictions dynamically.

The model is also valuable for **analyzing learning algorithms' efficiency**. Algorithms with lower mistake bounds are considered more efficient because they reach a state of accurate predictions more quickly. This efficiency measure is essential in applications that prioritize real-time decision-making.

Example: The Perceptron Algorithm in the Mistake Bound Model

The **perceptron algorithm** is a classic example of an algorithm evaluated under the MB model. The perceptron algorithm is used for binary classification and makes predictions based on a linear combination of input features. In each round:

1. The perceptron receives an input example and predicts the class label.
2. If the prediction is incorrect, the perceptron updates its weights based on the error.
3. This update process continues until the algorithm makes no further mistakes.

In the MB model, the perceptron's mistake bound can be analyzed. It is known that, for linearly separable data, the perceptron will make a finite number of mistakes (related to properties of the data such as margin size) before it converges on an accurate linear classifier. This bound provides a theoretical assurance of the perceptron's learning performance over time.

Advantages of the Mistake Bound Model

1. **Efficiency in Real-time Learning:** By focusing on the number of mistakes, the MB model provides insights into how quickly an algorithm can learn without requiring retraining on an entire dataset. This feature is valuable in **real-time applications** where immediate learning from each data point is critical.
2. **Comparison of Algorithms:** The MB model enables comparisons across algorithms based on their mistake bounds. An algorithm with a lower mistake bound is more favorable because it minimizes errors faster, which is beneficial in scenarios with high-stakes predictions or limited computational resources.
3. **Adaptability to Dynamic Environments:** In dynamic settings where data patterns evolve (e.g., changing customer behavior, fluctuating financial markets), the MB model is practical because it promotes incremental learning. This capability helps algorithms adapt efficiently to new data patterns.

Disadvantages and Limitations

1. **Dependence on Data Sequence:** The number of mistakes made by an algorithm can depend on the order in which it receives examples. In some cases, reordering the examples could change the mistake count.
2. **Limited Scope:** The MB model is specifically designed for **binary classification** and certain types of structured prediction problems. It is less applicable to unsupervised learning or settings where the concept to be learned is not well-defined.
3. **Theoretical Nature:** While the mistake bound provides a theoretical upper limit on errors, it does not guarantee that the bound will be the exact count of errors in practical applications.

Mistake Bound Model and Learning Complexity

The MB model also ties into the **complexity theory** of learning, offering a way to categorize problems based on their mistake bounds. Problems that require fewer mistakes to learn are considered easier or more learnable. In contrast, problems with high mistake bounds may require more complex algorithms or longer training periods.

Instant Based Learning

4.1 K-Nearest Neighbour Learning

KNN is one of the most basic yet essential classification algorithms in machine learning. It belongs to the supervised learning domain and finds intense application in pattern recognition, data mining, and intrusion detection.

It is widely disposable in real-life scenarios since it is non-parametric, meaning it does not make any underlying assumptions about the distribution of data (as opposed to other algorithms such as GMM, which assume a Gaussian distribution of the given data). We are given some prior data (also called training data), which classifies coordinates into groups identified by an attribute.

As an example, consider the following table of data points containing two features:

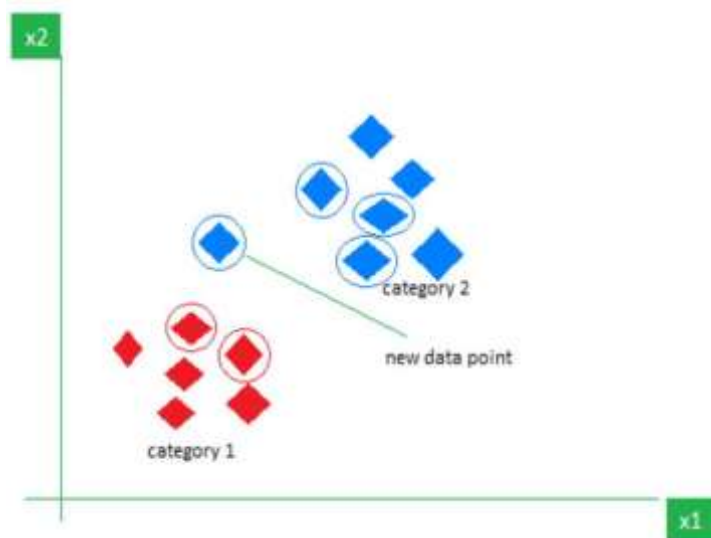


Fig. 4.1 KNN Algorithm working visualization.

Now, given another set of data points (also called testing data), allocate these points to a group by analyzing the training set. Note that the unclassified points are marked as 'White'.

Intuition Behind KNN Algorithm

If we plot these points on a graph, we may be able to locate some clusters or groups. Now, given an unclassified point, we can assign it to a group by observing what group its nearest neighbors belong to. This means a point close to a cluster of points classified as 'Red' has a higher probability of getting classified as 'Red'.

Why do we need a KNN algorithm?

(K-NN) algorithm is a versatile and widely used machine learning algorithm that is primarily used for its simplicity and ease of implementation. It does not require any assumptions about the underlying data distribution. It can also handle both numerical and categorical data, making it a flexible choice for various types of datasets in classification and regression tasks. It is a non-parametric method that makes predictions based on the similarity of data points in a given dataset. K-NN is less sensitive to outliers compared to other algorithms.

The K-NN algorithm works by finding the K nearest neighbors to a given data point based on a distance metric, such as Euclidean distance. The class or value of the data point is then determined by the majority vote or average of the K neighbors. This approach allows the algorithm to adapt to different patterns and make predictions based on the local structure of the data.

Distance Metrics Used in KNN Algorithm

As we know that the KNN algorithm helps us identify the nearest points or the groups for a query point. But to determine the closest groups or the nearest points for a query point we need some metric. For this purpose, we use below distance metrics:

Euclidean Distance

This is nothing but the cartesian distance between the two points which are in the plane/hyperplane. Euclidean distance can also be visualized as the length of the straight line that joins the two points which are into consideration. This metric helps us calculate the net displacement done between the two states of an object.

$$\text{distance}(x, X_i) = \sum_{j=1}^n d(x_j - X_{ij})^2 \quad \text{distance}(x, X_i) = \sum_{j=1}^n d(x_j - X_{ij})^2 \quad (4.1)$$

Manhattan Distance

Manhattan Distance metric is generally used when we are interested in the total distance traveled by the object instead of the displacement. This metric is calculated by summing the absolute difference between the coordinates of the points in n-dimensions.

$$d(x,y) = \sum_{i=1}^n |x_i - y_i| \quad (4.2)$$

Minkowski Distance

We can say that the Euclidean, as well as the Manhattan distance, are special cases of the Minkowski distance.

$$d(x,y) = (\sum_{i=1}^n |x_i - y_i|^p)^{1/p} \quad (4.3)$$

From the formula above we can say that when $p = 2$ then it is the same as the formula for the Euclidean distance and when $p = 1$ then we obtain the formula for the Manhattan distance.

The above-discussed metrics are most common while dealing with a Machine Learning problem but there are other distance metrics as well like Hamming Distance which come in handy while dealing with problems that require overlapping comparisons between two vectors whose contents can be Boolean as well as string values.

How to choose the value of k for KNN Algorithm?

The value of k is very crucial in the KNN algorithm to define the number of neighbors in the algorithm. The value of k in the k -nearest neighbors (k -NN) algorithm should be chosen based on the input data. If the input data has more outliers or noise, a higher value of k would be better. It is recommended to choose an odd value for k to avoid ties in classification. Cross-validation methods can help in selecting the best k value for the given dataset.

Algorithm for K-NN

DistanceToNN=sort(distance from 1st example, distance from kth example) value $i = 1$ to number of training records:

Dist=distance(test example, ith example)

if (Dist<any example in DistanceToNN):

Remove the example from DistanceToNN and value.

Put new example in DistanceToNN and value in sorted order.

Return average of value

Fit using K-NN is more reasonable than 1-NN, K-NN affects very less from noise if dataset is large.

In K-NN algorithm, We can see jump in prediction values due to unit change in input. The reason for this due to change in neighbors. To handles this situation, We can use weighting of neighbors in algorithm. If the distance from neighbor is high, we want less effect from that neighbor. If distance is low, that neighbor should be more effective than others.

Workings of KNN algorithm

The K-Nearest Neighbors (KNN) algorithm operates on the principle of similarity, where it predicts the label or value of a new data point by considering the labels or values of its K nearest neighbors in the training dataset.

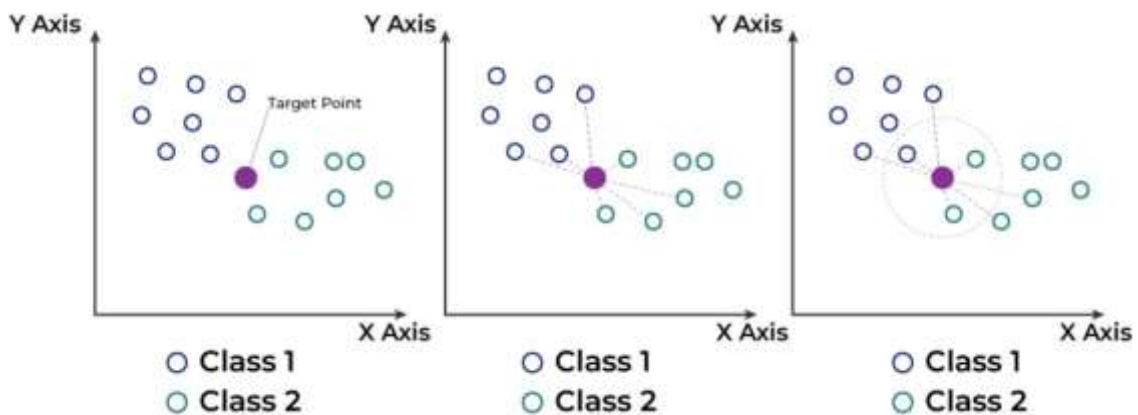


Fig. 4.2 Workings of KNN algorithm.

Step-by-Step explanation of how KNN works is discussed below:

Step 1: Selecting the optimal value of K

- ❖ K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

- ❖ To measure the similarity between target and training data points, Euclidean distance is used. Distance is calculated between each of the data points in the dataset and target point.

Step 3: Finding Nearest Neighbors

- ❖ The k data points with the smallest distances to the target point are the nearest neighbors.

Step 4: Voting for Classification or Taking Average for Regression

- ❖ In the classification problem, the class labels of K -nearest neighbors are determined by performing majority voting. The class with the most occurrences among the neighbors becomes the predicted class for the target data point.
- ❖ In the regression problem, the class label is calculated by taking average of the target values of K nearest neighbors. The calculated average value becomes the predicted output for the target data point.

Let X be the training dataset with n data points, where each data point is represented by a d -dimensional feature vector X_i and Y be the corresponding labels or values for each data point in X . Given a new data point x , the algorithm calculates the distance between x and each data point X_i in X using a distance metric, such as Euclidean distance: $\text{distance}(x, X_i) = \sqrt{\sum_{j=1}^d (x_j - X_{ij})^2}$

The algorithm selects the K data points from X that have the shortest distances to x . For classification tasks, the algorithm assigns the label y that is most frequent among the K nearest neighbors to x . For regression tasks, the algorithm calculates the average or weighted average of the values y of the K nearest neighbors and assigns it as the predicted value for x .

Advantages of the KNN Algorithm

- ❖ Easy to implement as the complexity of the algorithm is not that high.

- ❖ **Adapts Easily** - As per the working of the KNN algorithm it stores all the data in memory storage and hence whenever a new example or data point is added then the algorithm adjusts itself as per that new example and has its contribution to the future predictions as well.
- ❖ **Few Hyperparameters** - The only parameters which are required in the training of a KNN algorithm are the value of k and the choice of the distance metric which we would like to choose from our evaluation metric.

Disadvantages of the KNN Algorithm

- ❖ **Does not scale** - As we have heard about this that the KNN algorithm is also considered a Lazy Algorithm. The main significance of this term is that this takes lots of computing power as well as data storage. This makes this algorithm both time-consuming and resource exhausting.
- ❖ **Curse of Dimensionality** - There is a term known as the peaking phenomenon according to this the KNN algorithm is affected by the curse of dimensionality which implies the algorithm faces a hard time classifying the data points properly when the dimensionality is too high.
- ❖ **Prone to Overfitting** - As the algorithm is affected due to the curse of dimensionality it is prone to the problem of overfitting as well. Hence generally feature selection as well as dimensionality reduction techniques are applied to deal with this problem.

4.2 Locally Weighted Linear Regression

Within the field of machine learning and regression analysis, Locally Weighted Linear Regression (LWLR) emerges as a notable approach that bolsters predictive accuracy through the integration of local adaptation. In contrast to conventional linear regression models, which presume a universal correlation among variables, LWLR acknowledges the significance of localized patterns and relationships present in the data. In the subsequent discourse, we embark on an exploration of the fundamental principles, diverse applications, and inherent advantages offered by Locally Weighted Linear Regression. Our aim is to shed light on its exceptional capacity to amplify predictive prowess and furnish intricate understandings of intricate datasets.

Fundamentally, LWLR manifests as a non-parametric regression algorithm that discerns the connection between a dependent variable and several independent variables.

Notably, LWLR's distinctiveness emanates from its dynamic adaptability, which empowers it to bestow distinct weights upon individual data points contingent on their proximity to the target point under prediction. In essence, this algorithm accords greater significance to proximate data points, deeming them as more influential contributors in the prediction process.

Principles of Locally Weighted Linear Regression

LWLR functions on the premise that the association between the dependent and independent variables adheres to linearity; however, this relationship is allowed to exhibit variability across distinct sections within the dataset. This is achieved by employing an individual linear regression model for each prediction, employing a weighted least squares technique. The determination of weights is carried out through a kernel function, which bestows elevated weights upon data points in close proximity to the target point and diminishes the weights for those that are farther away.

Applications of Locally Weighted Linear Regression

- ❖ **Time Series Analysis:** LWLR is particularly useful in time series analysis, where the relationship between variables may change over time. By adapting to the local patterns and trends, LWLR can capture the dynamics of time-varying data and make accurate predictions.
- ❖ **Anomaly Detection:** LWLR can be employed for anomaly detection in various domains, such as fraud detection or network intrusion detection. By identifying deviations from the expected patterns in a localized manner, LWLR helps detect abnormal behavior that may go unnoticed using traditional regression models.
- ❖ **Robotics and Control Systems:** In robotics and control systems, LWLR can be utilized to model and predict the behavior of complex systems. By adapting to local conditions and variations, LWLR enables precise control and decision-making in dynamic environments.

Benefits of Locally Weighted Linear Regression

- ❖ **Improved Predictive Accuracy:** By considering local patterns and relationships, LWLR can capture subtle nuances in the data that might be overlooked by global regression models. This results in more accurate predictions and better model performance.
- ❖ **Flexibility and Adaptability:** LWLR can adapt to different regions of the dataset, making it suitable for complex and non-linear relationships. It offers flexibility in capturing local variations, allowing for more nuanced analysis and insights.

- ❖ **Interpretable Results:** Despite its adaptive nature, LWLR still provides interpretable results. The localized models offer insights into the relationships between variables within specific regions of the data, aiding in the understanding of complex phenomena.

Code:

For the sake of understanding, we will implement it into code.

```
# Locally weighted linear regression (LWR)
k <- 5
k_folds <- createFolds(data$No, k)
tau <- 1000
MSE <- sapply(names(k_folds), function(fold) {
  train <- data[!No %in% k_folds[[fold]]]
  test <- data[No %in% k_folds[[fold]]]
  predictions <- sapply(seq.int(nrow(test)), function(observation) {
    X <- c(test[observation, X6_longitude], test[observation, X5_latitude])
    halversine <- sapply(seq.int(nrow(train)), function(obs) {
      X_i <- c(train[obs, X6_longitude], train[obs, X5_latitude])
      distm(X_i, X, fun = distHaversine)[1, 1]
    })
    weights <- exp(-(halversine^2 / (2 * tau^2)))
    model <- lm(Adjusted_house_price_per_unit_area
      ~ X2_house_age
      + X3_distance_to_the_nearest_MRT_station
      + X4_number_of_convenience_stores,
      data = train,
      weights = weights)
    predict(model, test[observation])
  })
  MSE <- sum((test$Adjusted_house_price_per_unit_area - predictions)^2)
  MSE / nrow(test)
})
LWR <- mean(MSE)

# Linear regression
MSE <- sapply(names(k_folds), function(fold) {
  train <- data[!No %in% k_folds[[fold]]]
  test <- data[No %in% k_folds[[fold]]]
  model <- lm(Adjusted_house_price_per_unit_area
    ~ X2_house_age
```

```
+ X3_distance_to_the_nearest_MRT_station
+ X4_number_of_convenience_stores,
data = train)
predictions <- sapply(seq.int(nrow(test)), function(observation) {
  predict(model, test[observation])
})
MSE <- sum((test$Adjusted_house_price_per_unit_area - predictions)^2)
MSE / nrow(test)
})
LM <- mean(MSE)

# Display results
data.table(Method = c("LWR", "Linear regression"), MSE = c(LWR, LM))
```

Output:

Method	MSE
<chr>	<dbl>
LWR	94.57899
Linear regression	119.17100

In terms of Mean Squared Error (MSE), Locally Weighted Regression (LWR) outperforms traditional Linear Regression. With an MSE of 94.57899, LWR demonstrates superior predictive accuracy compared to Linear Regression, which has an MSE of 119.17100. This indicates that LWR provides more precise predictions and exhibits a better fit for the data. The improved performance of LWR can be attributed to its ability to incorporate local adaptation and consider the localized patterns and relationships within the dataset.

4.3 Radial Basis Function (RBF)

A Radial Basis Function (RBF) is a type of mathematical function that processes real-valued inputs and produces real-valued outputs. It operates by calculating the distance between the input value and a specified fixed point in space, often called a center or prototype. This fixed point is positioned at an imaginary or pre-defined location within the spatial context of the data.

The output of the RBF is highest when the input is close to this center, gradually decreasing as the distance grows. RBFs are commonly utilized in machine learning, particularly in algorithms such as Support Vector Machines (SVM) and Artificial Neural Networks (ANN), and play a significant role in interpolation, approximation, and pattern recognition tasks. In these applications, RBFs can transform data into higher-dimensional space, enabling the capture of complex, non-linear relationships within the data.

Common Types of Radial Basis Functions

Several types of Radial Basis Functions are used, each defined by unique characteristics and mathematical expressions. Here are a few key examples:

1. **Gaussian Radial Basis Function:** The most widely used RBF, the Gaussian RBF, has a smooth, bell-shaped curve. It is particularly effective in various applications due to its simplicity and versatility. The Gaussian RBF can be expressed as

$$\phi(r) = \exp\left\{-\frac{r^2}{2\sigma^2}\right\} \quad \phi(r) = \exp\left(-\frac{r^2}{2\sigma^2}\right) \quad (4.4)$$

where r represents the Euclidean distance between the input and the center, and σ is a parameter that controls the width of the curve.

2. **Multiquadric Radial Basis Function:** This RBF is often applied in interpolation and meshless methods, producing smooth transitions between points. The Multiquadric RBF is defined by:

$$\phi(r) = r^2 + c^2 \quad \phi(r) = \sqrt{r^2 + c^2} \quad \phi(r) = r^2 + c^2 \quad (4.5)$$

where c is a constant.

3. **Inverse Multiquadric Radial Basis Function:** Similar to the multiquadric RBF but with an inverse in the denominator, this function has a different shape and is represented as:

$$\phi(r) = \frac{1}{r^2 + c^2} \quad \phi(r) = \frac{1}{\sqrt{r^2 + c^2}} \quad \phi(r) = r^2 + c^2 \quad (4.6)$$

4. **Thin Plate Spline Radial Basis Function:** Commonly used in surface interpolation and deformation tasks, the Thin Plate Spline RBF is defined as:

$$\phi(r) = r^2 \log^{-1} f_0(r) \quad \phi(r) = r^2 \log(r) \quad \phi(r) = r^2 \log(r) \quad (4.7)$$

5. **Cubic Radial Basis Function:** Defined by a cubic polynomial behavior, the Cubic RBF is expressed as:

$$\phi(r) = r^3 \quad \phi(r) = r^3 \quad \phi(r) = r^3 \quad (4.8)$$

This function is sometimes used for interpolation purposes.

Example of RBF in Option Pricing

Consider using an RBF approach to improve option pricing, which requires capturing non-linear dependencies between factors like stock price (S), time to expiration (T), and implied volatility (σ). Traditional linear models struggle with these non-linear relationships, especially in fluctuating markets. By associating each combination of these factors with a radial basis function, an RBF-based model adjusts dynamically. The RBF is higher when the current input is close to the RBF center (representing a specific combination of S , T , and σ) and diminishes as the distance grows. This adaptability enables the model to handle the market's complexities, allowing for more precise option pricing predictions.

Why Use Radial Basis Functions?

One of the primary advantages of RBFs is their ability to effectively capture non-linear relationships in data, which are often challenging for simpler linear models. By transforming input data into higher-dimensional space, RBFs reveal hidden patterns, making them ideal for applications such as pattern recognition, interpolation, and regression tasks.

In fields like machine learning, signal processing, and numerical analysis, RBFs are highly valued for their unique mathematical properties and adaptability. They efficiently handle intricate dependencies within data, making them suitable for tasks with complex patterns, such as approximation and pattern recognition.

How Radial Basis Functions Work

In Radial Basis Function Networks (RBFNs), RBFs are used in a hidden layer to evaluate the proximity between input vectors and predetermined centers in the input space. Each RBF in the network calculates the Euclidean distance between the input vector and its center. It then squares this distance, divides it by $2\sigma^2$, and applies an exponential function, resulting in a weighted output that reflects the input's proximity to the center. RBFNs leverage multiple RBFs with distinct centers, combining their outputs to generate the network's final prediction. The network adjusts parameters, like the centers and widths of the RBFs, during training, often using techniques such as k-means clustering or other optimization methods.

RBFNs' ability to flexibly evaluate similarities between patterns makes them especially useful for tasks such as pattern recognition, regression, and interpolation.

An RBF Network (RBFN) typically has three layers:

1. **Input Layer:** Nodes in this layer represent the features or dimensions of the input data, where each node corresponds to an element in the input vector.
2. **Hidden Layer:** This layer contains nodes connected to radial basis functions. Each hidden node functions as a center or prototype for an RBF.

3. **Output Layer:** The output layer provides the network's final prediction, usually calculated as a linear combination of the outputs from the hidden layer.

Advantages of Radial Basis Functions

RBFs provide several advantages:

- ❖ **Resilience to Noise:** RBFs are generally resistant to noise in input data, making them suitable for tasks where input data may contain noise or inconsistencies.
- ❖ **Effective in Non-linear Distributions:** RBFs excel at modeling complex data distributions, such as logarithmic or trigonometric patterns, by transforming input data into a high-dimensional space.
- ❖ **Ease of Interpretation:** The structure of an RBF network is often easier to interpret than other neural networks. Each node in the hidden layer represents a specific region in the data space, allowing a clearer understanding of its role in the model.
- ❖ **Simplified Hidden Layer Management:** RBFNs typically involve only one hidden layer, which is computationally efficient and relatively straightforward to manage compared to other deep architectures.

By leveraging RBFs, models can generalize complex patterns effectively, uncovering hidden patterns in non-linear distributions. The structure of RBFNs also allows easier handling and interpretation, making them valuable for diverse applications across machine learning, signal processing, and beyond.

4.4 Case based Learning

Case-Based Learning (CBL) in machine learning is a methodology derived from Case-Based Reasoning (CBR) in Artificial Intelligence. It involves solving new problems by recalling and adapting solutions from previous, similar cases, making it particularly effective for domains where unique situations require tailored responses. Here is an in-depth look at this approach, its mechanisms, applications, benefits, and challenges.

Case-Based Reasoning (CBR) is an AI method that solves new problems by referring to previously encountered cases, reusing past solutions as templates for new issues. Unlike rule-based systems that rely on generalizations, CBR focuses on individual instances. By referencing past cases, CBR systems adapt solutions to meet the specific needs of each new case, making it highly relevant in fields where one-size-fits-all solutions may not apply.

Key Components of Case-Based Learning in Machine Learning

1. **Case Retrieval:** When a new problem arises, the system searches its case library for instances that closely resemble the current situation. This similarity can be measured through various metrics, such as Euclidean distance, cosine similarity, or custom-defined measures specific to the domain. The goal is to retrieve cases that provide the most relevant information for crafting a solution to the current problem.
2. **Case Reuse:** After identifying a similar case, the system reuses the solution from that case, adapting it as necessary to address the nuances of the new situation. This step often involves adjusting parameters or modifying certain actions to make the solution applicable.
3. **Case Revision:** Following the initial application of the adapted solution, the system evaluates its effectiveness. If modifications are needed, the solution undergoes a revision process to optimize its performance. This revision is crucial for enhancing accuracy and ensuring that the solution meets the demands of the new problem.
4. **Case Retention:** Once the solution is finalized, the new problem and its adapted solution are stored as a new case in the case library. This allows the system to learn over time, building a richer repository of cases that enhance its problem-solving capabilities.

Practical Applications of Case-Based Learning in Machine Learning

Case-Based Learning is especially valuable in fields that require adaptability, interpretability, and experience-based reasoning. Some prominent applications include:

- ❖ **Healthcare and Medical Diagnosis:** In medical fields, CBL systems can assist in diagnosing illnesses and recommending treatments by referencing historical patient data and applying solutions from similar cases. This allows the system to consider the unique conditions of each patient while leveraging past medical insights.
- ❖ **Personalized Recommendation Systems:** In e-commerce or streaming platforms, CBL can be used to make personalized recommendations based on user behaviors similar to those of previous users. Rather than relying solely on generalized models, the system can draw on cases with similar user interactions to suggest tailored products or content.
- ❖ **Legal Assistance and Analysis:** Legal research tools can leverage CBL to retrieve past cases with similar legal arguments or precedents, helping attorneys develop legal strategies based on analogous cases.
- ❖ **Customer Support and Troubleshooting:** Many technical support systems employ CBL by referencing solutions from similar past cases to address customer issues. By recalling solutions from cases that match the current issue, support agents can offer more accurate assistance.

Advantages of Case-Based Learning

- ❖ **High Interpretability:** Because CBL relies on previous cases, the reasoning process is more transparent and interpretable than with many black-box machine learning models. The decision-making process is easily explainable since it's grounded in concrete, real-life examples, which is crucial for fields where justification of decisions is essential, such as healthcare and law.
- ❖ **Adaptability to New Scenarios:** Unlike rigid rule-based systems that can struggle with novel problems, CBL can adjust past solutions to new circumstances, making it more flexible in the face of unique cases.
- ❖ **Incremental Learning Capability:** CBL is inherently incremental, meaning the system can evolve with each new case. Unlike traditional ML models that require retraining with new data, CBL systems can continually update their case library, improving their problem-solving skills without needing full retraining.

Challenges of Case-Based Learning

- ❖ **Computational Cost:** Searching through extensive case databases to find similar cases can be computationally demanding, especially in large-scale applications where the similarity assessment process can be complex.
- ❖ **Case Maintenance and Storage:** As the system accumulates more cases, it becomes essential to manage, organize, and sometimes prune the case library to maintain efficiency and relevance. Over time, outdated or irrelevant cases may need to be removed to prevent redundancy and ensure that only useful cases are retained.
- ❖ **Quality of Cases:** The success of CBL hinges on the quality of the stored cases. If the case library contains biased, inaccurate, or poorly documented cases, the system's recommendations may be unreliable or inappropriate, which is especially problematic in high-stakes fields like healthcare.

Techniques Supporting Case-Based Learning

- ❖ **Feature Engineering:** Crafting relevant features is essential in CBL to ensure that similar cases are retrieved effectively. In many applications, domain experts play a key role in designing features that accurately capture the important aspects of cases for similarity assessment.
- ❖ **Similarity Metrics:** Choosing the right similarity measure is critical for effective case retrieval. Simple distance metrics (such as Euclidean distance) may be adequate in some situations, while others may require more sophisticated metrics, such as Mahalanobis distance or learned similarity functions.

- ❖ **Deep Learning and Embeddings:** In some CBL systems, embeddings from neural networks are used to represent cases in a lower-dimensional space, where similar cases are closer together. This approach enables efficient retrieval of similar cases, even in complex domains where feature spaces are large and high-dimensional.

Case-Based Learning in Educational Contexts

Case-Based Learning is also an effective pedagogical approach, especially in machine learning education. By presenting students with real-world cases, educators can help them understand the practical applications of machine learning techniques. Instead of focusing purely on abstract theory, CBL allows students to work through cases, giving them hands-on experience with data, problem-solving, and model evaluation.

Advanced Learning

5.1 Learning Sets of Rules

Learning sets of rules is a powerful approach in machine learning, enabling systems to make decisions or classify data by following interpretable, human-readable rules. There are multiple approaches to learning rule sets, each with unique strengths and methods of representation.

1. **From Decision Trees to Rule Sets:** One approach to learning rules is to first construct a decision tree. Each path from the root to a leaf node in the tree represents a sequence of decisions that lead to a specific outcome. By traversing each path, we can translate the decision tree into a corresponding set of rules one rule for each leaf node. This allows us to represent the decision tree's structure in an equivalent rule-based form, which is often more readable and easier to modify.
2. **Genetic Algorithms for Rule Learning:** Another method leverages genetic algorithms (GAs) to directly evolve rule sets. In this approach, each rule set is encoded as a bit string, representing possible hypotheses. Genetic operators like crossover, mutation, and selection are used to search through the hypothesis space, aiming to optimize the rule set to perform effectively on the training data. This approach allows for a more flexible and exploratory search compared to greedy decision tree algorithms, potentially discovering novel rule combinations that enhance performance.
3. **Production Rules (If-Then Rules):** One of the most expressive and human-readable ways to represent learned hypotheses is through production rules, often formatted as if-then statements. These rules can capture complex relationships in data and are easily interpretable by humans. Production rules are widely used in expert systems and knowledge-based systems due to their readability and logical structure.
4. **Direct Rule Learning vs. Transformation from Other Models:** Rules can be directly learned from data or derived from other representations, such as decision trees. In direct rule learning, algorithms attempt to learn rules without first constructing an intermediary model, focusing instead on maximizing predictive accuracy and interpretability from the start. Direct learning methods are often computationally intensive but yield high-quality rule sets that are optimized for both performance and readability.
5. **Advantages of First-Order Rules:** Direct rule-learning algorithms can also learn first-order rules, which support variables and quantifiers, enabling greater expressiveness. First-order rules (as opposed to propositional or zero-order rules) have enhanced

representational power, allowing them to capture more intricate relationships in data. This capability is particularly valuable in domains that require relational knowledge or logical inferences.

6. **Automatic Inference of PROLOG-like Programs:** Learning first-order rules can be viewed as an automatic process of inferring PROLOG programs from examples. This interpretation aligns rule learning with logic programming, where the goal is to generate logic-based programs that generalize from examples to perform reasoning tasks. This approach is especially relevant in fields that require symbolic reasoning and relational knowledge representation, such as natural language processing, bioinformatics, and knowledge representation.

5.1.1 Advanced Concepts in Rule Learning

1. **Hybrid Models Combining Rule Learning and Neural Networks:** In recent research, hybrid models have emerged that combine rule-based learning with neural networks. These models aim to leverage the interpretability of rules while benefiting from the high-dimensional pattern recognition capabilities of neural networks. For instance, neural networks can be used to identify patterns in data, and then rule extraction techniques are applied to derive interpretable rules from the trained models.
2. **Multi-Objective Rule Learning:** Many applications require balancing multiple objectives, such as accuracy, interpretability, and computational efficiency. Multi-objective rule-learning algorithms, including Pareto-based methods, optimize these objectives simultaneously, resulting in rule sets that are both effective and human-readable. These approaches are widely used in fields like healthcare and finance, where interpretability is critical.
3. **Rule Pruning and Simplification:** Direct rule-learning algorithms often produce complex rule sets, which may include redundant or irrelevant conditions. Pruning techniques, such as Reduced Error Pruning and Minimum Description Length (MDL), are used to simplify rule sets, enhancing both interpretability and generalization. Pruning reduces over-fitting by removing overly specific rules or rule components.
4. **Sequential Covering Algorithms:** Sequential covering is a popular approach in rule learning, where rules are learned one at a time in a sequential manner, covering portions of the training data iteratively. Each rule is designed to capture a specific subset of the data, and once a rule is created, instances covered by that rule are removed from the dataset. This process continues until most of the data is covered by a comprehensive set of rules.
5. **Advanced Applications of Rule Learning in Explainable AI:** Rule learning has become increasingly significant in the field of Explainable AI (XAI), especially in domains where understanding model decisions is critical.

6. Rule-based explanations are highly valued in regulatory environments and industries such as healthcare, where transparency and accountability are essential. By providing a clear rationale for predictions, rule-based systems increase user trust and facilitate compliance with ethical and legal standards.

5.2 Sequential Covering Algorithms

Sequential covering algorithms are widely used in rule-based learning, particularly in the context of propositional learning systems. The goal of a sequential covering algorithm is to develop a set of rules (or a "cover") that can classify all positive examples while avoiding negative examples. This approach iteratively learns one rule at a time, removes the examples covered by that rule, and repeats this process until all positive examples are covered.

Sequential covering is often referred to as a "separate-and-conquer" strategy because it separates instances that a rule covers from those it does not, allowing the algorithm to conquer or classify each instance sequentially. This method is foundational to many rule-learning algorithms, making it a key approach in constructing interpretable models.

5.2.1 Steps in Sequential Covering

1. **Initialize an Empty Rule Set:** Start with no rules.
2. **Learn a Single Rule:** Use a rule-learning function (e.g., Learn-One-Rule) to find the best rule that covers the largest possible number of positive examples without covering any negative examples.
3. **Check Rule Threshold:** If the learned rule meets a threshold for accuracy or other metrics, add it to the rule set.
4. **Remove Covered Examples:** Remove all instances (data points) covered by the rule from the dataset.
5. **Repeat:** Continue the process by learning additional rules on the remaining examples until all positive examples are covered or a stopping condition is reached.
6. **Finalize Rule Ordering:** Order the rules according to performance or precedence.

Example Visualization: Sequential covering is illustrated through steps in a 2D data plot. In each iteration, the algorithm identifies a rule that best separates a subset of positive examples, removes those examples, and repeats until most or all positive examples are covered (**Fig. 5.1**).

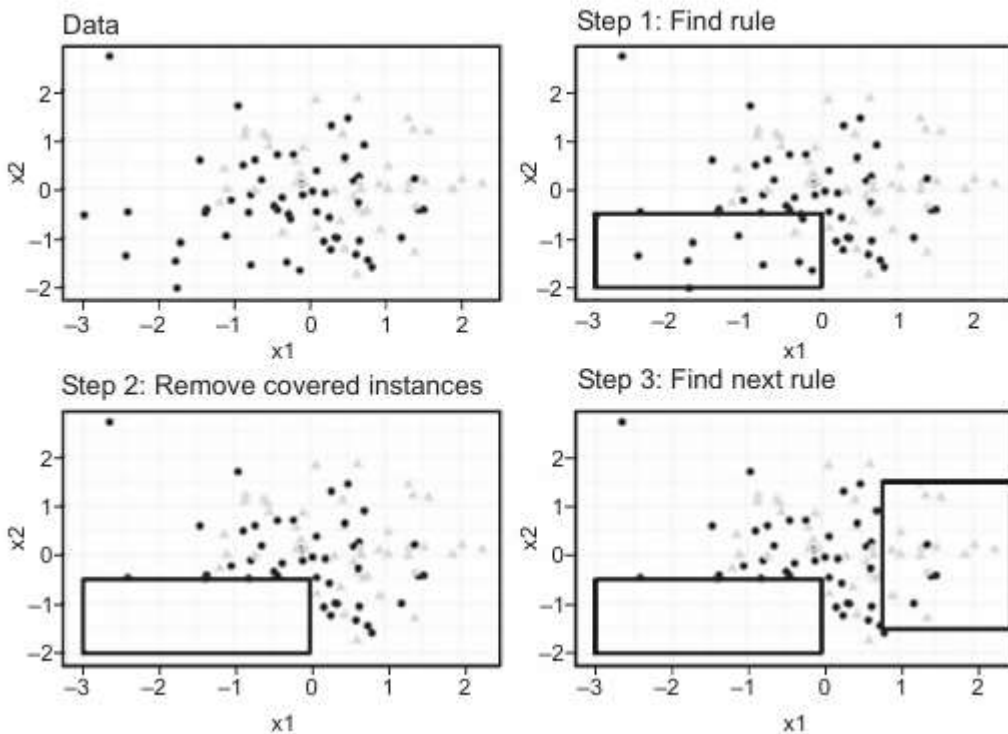


Fig. 5.1 Example for visualization of sequential covering.

5.2.2 Characteristics and Limitations of Sequential Covering

- **High Accuracy Requirement:** Sequential covering algorithms often aim for high accuracy in each learned rule, ensuring reliable predictions.
- **Low Coverage Potential:** Sometimes, sequential covering may miss examples due to overly specific rules, leaving certain cases unclassified.
- **Multi-Class Adaptation:** In multi-class settings, the algorithm is adapted to handle classes in order of least prevalence, starting with the least common class and progressing to more prevalent ones. This is known as a "one-versus-all" strategy, where rules are successively generated for each class against the rest.

5.2.3 Beam Search for Learning Single Rules

When the objective is to learn a single rule, beam search provides an efficient search method for finding optimal paths through a decision tree. A straightforward algorithm like OneR (One Rule) would be ineffective here, as it covers the entire feature space with only a single rule. Beam search, however, allows more flexibility by limiting the breadth of search to the most promising paths, reducing computational cost while aiming to find the best rules.

Steps in Beam Search for Rule Learning

1. **Decision Tree Generation:** Construct a decision tree (e.g., using CART or another algorithm) to find a rule that separates positive from negative examples.
2. **Select the Purest Node:** At each level of the tree, select the node with the lowest misclassification rate as the candidate rule.
3. **Backtracking for Rule Derivation:** The terminal node with the highest accuracy is used as the rule, while the path leading to this node forms the conditions of the rule.

Example Visualization: In a decision tree, beam search follows a path from the root to a specific terminal node with the purest subset, using attributes (like size = big) to form rules as shown in **Fig. 5.2**.

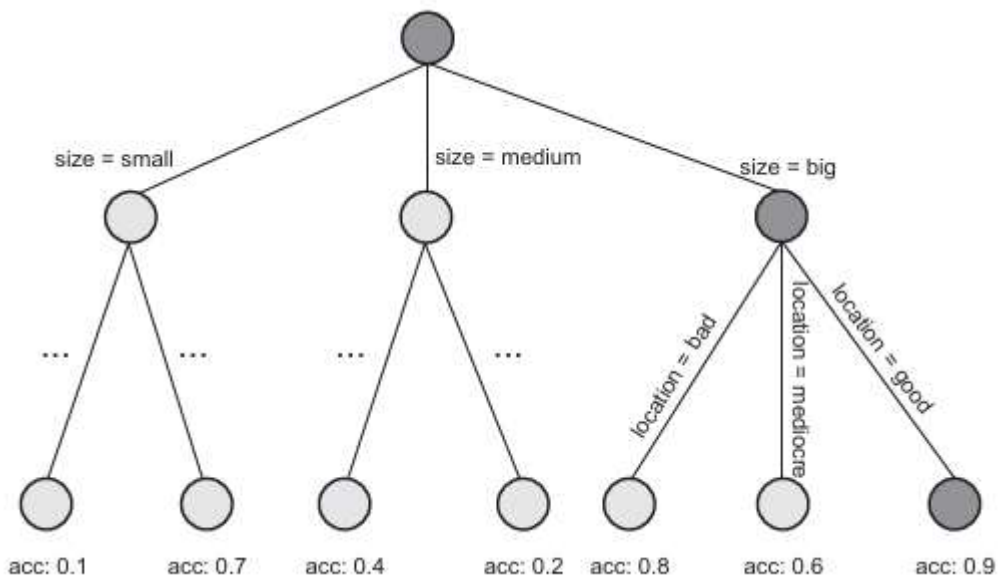


Fig. 5.2 Learning a rule by searching a path through a decision tree.

Search Strategies in Beam Search

Beam search in rule learning explores a limited subset of paths, balancing depth and breadth. There are various search strategies that can be employed in rule learning to enhance effectiveness:

- ❖ **Hill-Climbing:** Greedily pursues the best local solution but may get stuck in local optima.

- ❖ **Best-First Search:** Expands the most promising nodes first based on heuristic evaluation.
- ❖ **Stochastic Search:** Introduces randomness to escape local optima, exploring more diverse rules.

Beam search, by controlling the beam width, provides a controlled trade-off between thorough exploration and computational efficiency.

5.2.4 Variations in Rule Learning Algorithms

In rule learning, different algorithms use unique approaches to refine or optimize the process based on specific needs. Some common variations are:

1. **Single Rule Learning (e.g., AQ):** Algorithms like AQ focus on learning a single rule at a time, specifically covering only positive examples. Once a rule is established, new positive examples are explored iteratively to create additional rules.
2. **Simultaneous Covering:** Unlike sequential covering, where rules are created and refined one by one, some algorithms, such as CN2, generate multiple rules simultaneously. This approach often uses a general-to-specific search strategy, allowing it to maintain multiple hypotheses and explore rule sets in parallel.
3. **Direction of Search:**
 - ❖ **General-to-Specific (Top-Down):** Starts with a broad rule, narrowing down by adding conditions until a satisfactory rule is found. This approach ensures that the model explores general patterns before diving into specifics.
 - ❖ **Specific-to-General (Bottom-Up):** Begins with highly specific rules and generalizes by removing conditions. This approach is typically used when starting from examples and is effective in noisy data environments.
4. **Generate-Then-Test vs. Example-Driven:**
 - ❖ **Generate-Then-Test:** Involves generating candidate rules and testing them on the dataset to refine hypotheses. This approach, used in CN2 and similar algorithms, is more computationally intensive but can yield robust results.
 - ❖ **Example-Driven:** Algorithms like AQ evaluate each example iteratively, building rules around specific instances and expanding coverage as needed. Example-driven methods are generally more efficient in processing but may be less exhaustive.

5. **Performance Metrics and Rule Evaluation:** Rule-learning systems can prioritize different performance metrics, such as accuracy, precision, recall, and coverage, based on application needs. Multi-objective optimization can be employed to balance these factors, especially in applications like medical diagnosis, where precision and interpretability are equally crucial.
6. **Handling Noise and Overfitting:** Many rule-learning algorithms incorporate techniques to handle noise, such as pruning low-accuracy rules or using Minimum Description Length (MDL) to penalize overly complex rule sets. Regularization techniques can be applied in conjunction with rule learning to prevent overfitting, ensuring generalization to unseen data.

5.2.5 Advanced Applications of Rule Learning Variations

- ❖ **Explainable AI (XAI):** Rule-learning variations are widely used in XAI, providing interpretable models that can explain decision-making processes. Algorithms with high interpretability, like CN2 and sequential covering, are increasingly relevant in fields where transparency is paramount.
- ❖ **Medical Diagnosis and Fraud Detection:** These fields benefit from multi-objective rule learning, which allows algorithms to optimize for both sensitivity (recall) and specificity (precision), thereby achieving reliable and interpretable results.

5.3 First Order Rules

5.3.1 Overview of Propositional vs. First-Order Logic

In logic, propositional logic allows for the representation of individual propositions and their combinations based on truth-functional operators. For example, propositions such as "Tom is a man" or "All men are mortal" can be represented by proposition letters like P or Q. In propositional logic, complex statements are formed using logical connectives such as:

1. **AND** ($\wedge$)
2. **OR** ($\vee$)
3. **NOT** ($\neg$)
4. **IMPLIES** ($\rightarrow$)

Inference rules, such as $P \rightarrow Q$, operate on these propositional forms. However, propositional logic is limited because it only allows statements about specific, fixed propositions and cannot express general relationships or quantify over individuals. For example, in propositional logic, the inference that "Tom is mortal" from "Tom is a man" and "All men are mortal" cannot be directly represented.

5.3.2 Introduction to First-Order Logic

First-order logic (FOL) extends propositional logic by allowing statements to include variables and quantifiers, enabling reasoning over sets of individuals or classes. This added expressiveness allows us to construct rules involving general relationships, such as assertions about categories of entities (e.g., “all men are mortal”). In FOL, statements can be expressed using predicates and quantifiers like:

- ❖ **Universal quantifier** ($\forall$): Represents statements about all members of a set (e.g., $\forall x(\text{man}(x) \rightarrow \text{mortal}(x))$)
- ❖ **Existential quantifier** ($\exists$): Represents statements about the existence of at least one member satisfying a condition

For instance, in FOL:

- ❖ “Tom is a man” can be represented as $\text{man}(\text{Tom})$
- ❖ “All men are mortal” can be represented as $\forall x(\text{man}(x) \rightarrow \text{mortal}(x))$
- ❖ These expressions enable us to draw inferences about specific or general cases, such as concluding that $\text{mortal}(\text{Tom})$.

Expressiveness and Applications in Machine Learning

First-order logic's expressiveness allows for a finer-grained specification when representing knowledge, which is particularly valuable in machine learning contexts. It enables the representation of relational knowledge, where entities and their relationships are explicitly defined. For instance, learning complex relational concepts, such as familial relationships, requires the ability to reason over pairs or groups of individuals.

Example of First-Order Rules in Relational Learning

Consider a relational learning task where the goal is to learn the concept of a "daughter" based on attributes of persons. The following properties might be used:

- ❖ **Attributes:** <Name, Mother, Father, Male, Female>
- ❖ **Relation to Learn:** $\text{Daughter}(x, y)$, where x is a daughter of y if x is female and y is the father of x .

In this setup, the training data might consist of pairs of individuals, where each person is represented with relevant attributes. For example:

- ❖ **Person 1:** Name = Ann, Mother = Sue, Father = Bob, Male = F, Female = T
- ❖ **Person 2:** Name = Bob, Mother = Gill, Father = Joe, Male = T, Female = F

Using such examples, we can infer a rule in FOL for identifying a daughter relationship:

- Rule: IF Father₁ = Bob $\wedge$ Name₂ = Ann $\wedge$ Female₁ = T, THEN Daughter_{1,2} = T

These rules are more flexible than propositional rules and allow us to represent and reason about relational structures and attributes within complex datasets.

5.3.3 Advanced Concepts in First-Order Logic for Machine Learning

1. **Inductive Logic Programming (ILP):** ILP is a subfield of machine learning that uses first-order logic to learn models from examples. ILP systems, like FOIL (First Order Inductive Learner), learn logical rules by constructing hypotheses that explain positive examples and exclude negative ones. ILP is especially useful in domains like bioinformatics, natural language processing, and knowledge discovery, where relationships among entities are crucial.
2. **Probabilistic First-Order Logic:** Traditional FOL assumes deterministic relationships. However, in real-world applications, uncertainty often needs to be accounted for. Probabilistic FOL combines logical structures with probabilistic reasoning, enabling models to handle uncertainty in relational data. This approach is useful in fields like computer vision, where relationships between entities may have probabilistic dependencies.
3. **Constraint Logic Programming (CLP):** CLP extends FOL by integrating constraints (e.g., numeric constraints) into the logical framework, enabling more complex and practical rule-based reasoning. CLP is particularly useful in applications requiring optimization or decision-making, such as scheduling, resource allocation, and automated planning.
4. **Neuro-Symbolic Integration:** Recent advances involve integrating first-order rules with neural networks to create neuro-symbolic systems. In these systems, neural networks handle high-dimensional data, while symbolic rules (in FOL) are used for logical reasoning. This hybrid approach combines the pattern recognition capabilities of deep learning with the interpretability of rule-based reasoning, making it suitable for applications requiring both complex perception and logical inference.
5. **Automated Knowledge Base Construction:** First-order logic forms the basis of many automated reasoning systems that construct knowledge bases from unstructured data. For instance, ontologies in semantic web applications use FOL-based structures to represent hierarchical relationships.

6. Rule-learning algorithms in FOL are employed to discover logical rules in these knowledge bases, facilitating automated reasoning in domains like healthcare, finance, and legal systems.

5.4 Sets of First-Order Rules: FOIL

FOIL (First-Order Inductive Learner) is an extension of propositional rule learning algorithms, such as Sequential Covering and Learn-One-Rule, to first-order rule learning. Unlike propositional rule learners, FOIL is designed to work with relational data and can create rules involving variables. FOIL generates sets of first-order rules, similar to Horn clauses, with a few key differences to optimize expressiveness and reduce complexity:

1. **No Function Symbols:** FOIL restricts literals to avoid function symbols, simplifying the hypothesis space.
2. **Negation in Rule Bodies:** FOIL allows negation of literals within rule bodies, making it more expressive than basic Horn clauses.

5.4.1 Learning Process in FOIL

Like Sequential Covering, FOIL learns one rule at a time and removes positive examples covered by each learned rule before moving on to generate additional rules. However, FOIL specifically focuses on learning rules that predict when the target literal (or predicate) is true, rather than predicting both true and false cases.

FOIL employs a hill-climbing search strategy (using a beam width of one) to navigate through the hypothesis space. It uses two nested loops to build hypotheses:

1. **Outer Loop:** At each iteration, the outer loop adds a new rule to an overarching disjunctive hypothesis. This loop progresses in a specific-to-general fashion, beginning with an empty hypothesis and adding rules until all positive instances are covered.
2. **Inner Loop:** Within each specific rule, the inner loop adds conjunctive constraints to refine the rule. This loop follows a general-to-specific approach, adding conditions until the rule excludes all negative examples.

5.4.2 FOIL Algorithm Steps

The basic FOIL algorithm can be broken down as follows:

1. **Initialization:** Define positive examples Pos and negative examples Neg.
2. **Learning Rules:**

- ❖ Start with an empty set of learned rules.
- ❖ While positive examples remain uncovered, create a new rule to cover a subset of these examples.
- ❖ Each rule is refined iteratively by adding literals that maximize a performance metric (FOIL-GAIN) until it sufficiently excludes negative examples.

3. **FOIL-GAIN:** FOIL uses a specialized metric, FOIL-GAIN, which evaluates each candidate literal by accounting for variable bindings and selecting the one that best improves the rule's performance.

FOIL Algorithm Pseudocode

FOIL(Target_predicate, Predicates, Examples)

Pos $\leftarrow$ positive Examples

Neg $\leftarrow$ negative Examples

Learned_rules $\leftarrow$ {}

while Pos is not empty do

 NewRule $\leftarrow$ most general rule possible (no conditions)

 NewRuleNeg $\leftarrow$ Neg

 while NewRuleNeg is not empty do

 Candidate_literals $\leftarrow$ generate literals from Predicates

 Best_literal $\leftarrow$ argmax(FOIL_Gain(Literal, NewRule))

 add Best_literal to NewRule

 NewRuleNeg $\leftarrow$ subset of NewRuleNeg that satisfies NewRule conditions

 Learned_rules $\leftarrow$ Learned_rules + NewRule

 Pos $\leftarrow$ Pos - {examples covered by NewRule}

Return Learned_rules

5.4.3 FOIL Features and Characteristics

FOIL is similar to propositional rule learners in structure but with significant differences:

1. **First-Order Rule Learning:** FOIL can learn first-order rules, accommodating variables within rule preconditions to represent relationships.
2. **FOIL-GAIN Metric:** A performance measure that adjusts for variable bindings, selecting literals that maximize coverage of positive examples while excluding negatives.
3. **Focus on Positive Prediction:** FOIL learns rules that predict the target literal when it is true, focusing on covering positive examples only.
4. **Hill-Climbing Search:** FOIL employs a simple hill-climbing search, refining rules iteratively with each literal addition.

5.4.4 Generating Candidate Specializations

When refining a rule, FOIL generates candidate literals based on predicates. These candidates are created with specific constraints:

- ❖ **Predicates with Variables:** Literals take the form $Q(v_1, \dots, v_r)$, where Q is a predicate from the set of provided predicates, and v_i are variables (new or existing). At least one variable in the literal must already exist in the rule to ensure relevance.
- ❖ **Equality Constraints:** Literals can specify equality between two variables already in the rule (e.g., $x_j = x_k$).
- ❖ **Negation:** FOIL also allows negated literals as specializations, increasing the expressiveness of rules.

5.4.5 Limitations of FOIL

FOIL, while powerful, has several limitations:

1. **Complexity of Literal Search Space:** The search space for potential literals can become intractable, especially in large datasets with numerous predicates and variables.
2. **Need for Extensive Background Knowledge:** FOIL often requires extensive background information and predicates, which must be provided manually.
3. **Local Optima in Hill-Climbing:** Hill-climbing search can get stuck in local optima, sometimes failing to find consistent rules covering all positive examples.
4. **Requirement for Complete Examples:** FOIL needs comprehensive examples for learning effective rules, limiting its performance in domains with sparse data.

5. **Handling Closed-World Negatives:** FOIL assumes a closed-world model for negatives, where any example not known to be true is assumed to be false, which may not always be accurate.
6. **Difficulty with Numeric Attributes:** FOIL struggles with handling numeric data without additional preprocessing or feature engineering.
7. **Inability to Learn Recursive Definitions:** FOIL is limited in expressing recursive definitions, which restricts its application in problems requiring self-referential rules.

5.4.6 Advanced Extensions and Applications of FOIL

FOIL has inspired a variety of advanced approaches to address its limitations and expand its applications:

1. **FOIL Variants for Noise Tolerance:** Variants of FOIL, such as CN2, incorporate noise-handling mechanisms to increase robustness in noisy datasets. These adaptations add noise thresholds or allow probabilistic rules that do not strictly satisfy all examples.
2. **Integration with Inductive Logic Programming (ILP):** FOIL's approach to first-order rule learning is foundational to ILP, which combines background knowledge with examples to induce logical rules. ILP systems such as Progol and Aleph build on FOIL's principles, allowing more expressive hypothesis spaces and better handling of recursive definitions.
3. **Probabilistic Extensions:** Probabilistic logic programming extends FOIL by associating probabilities with learned rules, allowing it to handle uncertainty in real-world applications, such as diagnosis and recommendation systems.
4. **FOIL in Neuro-Symbolic Systems:** Recent developments in neuro-symbolic AI incorporate FOIL-inspired logic with neural networks. This approach allows for high-level symbolic reasoning alongside deep learning, making FOIL suitable for explainable AI tasks in complex domains like healthcare.
5. **Applications in Knowledge Discovery:** FOIL is commonly applied in knowledge discovery tasks in domains like bioinformatics, where relational data structures benefit from FOIL's rule-based, first-order representation. For example, FOIL is used to identify relationships between proteins or genetic markers.

5.5 Induction on Inverted Deduction

Induction is the process of generating generalizations from specific examples, while deduction involves deriving specific conclusions based on general principles. In this context, **Induction on Inverted Deduction (IoID)** is an approach where learning is conceptualized as the inverse of deduction.

This technique is especially useful in fields like Inductive Logic Programming (ILP), where learning involves discovering hypotheses that explain observed data based on given background knowledge.

In IoID, given a dataset D and some background knowledge B , the task is to find a hypothesis h that, when combined with B , sufficiently explains D . This process is particularly valuable for learning relational patterns, where additional domain knowledge can assist in constraining the search space.

Formulation of the Problem

Suppose the training data consists of examples in the form $\langle x_i, f(x_i) \rangle$, where x_i denotes the i^{th} example, and $f(x_i)$ represents the target concept or value. The goal of learning is to discover a hypothesis h such that:

$$\forall \langle x_i, f(x_i) \rangle \in D, (B \wedge h \wedge x_i) \vdash f(x_i)$$

This expression means that, for each example in D , h combined with the background knowledge B logically entails the target outcome $f(x_i)$.

Example

Consider the target concept **Child(u, v)**, where we want to learn the relationship between two individuals, u and v , indicating that u is a child of v .

- **Example Data:** A positive example could be **Child(Bob, Sharon)**, where the instance is described by attributes such as **Male(Bob)**, **Female(Sharon)**, and **Father(Sharon, Bob)**.
- **Background Knowledge:** We might know that **Parent(u, v)** if **Father(v, u)**.

To learn the concept **Child**, we could generate hypotheses based on the background knowledge:

- **Hypothesis 1:** $h_1 : \text{Child}(u, v) \leftarrow \text{Father}(v, u)$
- **Hypothesis 2:** $h_2 : \text{Child}(u, v) \leftarrow \text{Parent}(v, u)$

In this case, IoID leverages background knowledge B to guide the discovery of h allowing us to select hypotheses that logically explain the observed relationships.

5.5.1 Key Features of Induction on Inverted Deduction

- a. **Flexible Learning Definition:** IoID generalizes the concept of learning as the discovery of a hypothesis that explains observed data, potentially without the need for explicit background knowledge. This aligns with common definitions of machine learning but emphasizes the role of structured background information.
- b. **Incorporation of Background Knowledge:** By including background knowledge, IoID enables richer and more accurate hypothesis formation. This approach allows for domain-specific information to shape the learning process, enhancing the relevance and utility of generated hypotheses.
- c. **Guided Hypothesis Search:** Background knowledge constrains the search for hypotheses, focusing the learning process on hypotheses that are logically coherent with BBB. This reduces the search space and makes the learning process more efficient.

5.5.2 Challenges in Induction on Inverted Deduction

- ❖ **Handling Noisy Data:** IoID relies on precise logical formulations, which can be challenging when data is noisy. Noisy examples may not perfectly fit any logical rule, requiring additional mechanisms (like probabilistic reasoning or noise-tolerant methods) to handle inconsistencies.
- ❖ **Expressiveness of First-Order Logic:** First-order logic is highly expressive, which can make hypothesis search computationally intensive. Searching the vast space of first-order expressions is often intractable, especially in large datasets with complex relationships.
- ❖ **Complexity of Background Knowledge:** While background knowledge constrains the hypothesis search, it can also add complexity. Representing background information accurately and concisely is essential, as overly detailed knowledge can increase the hypothesis space and slow down learning.

5.5.3 Advanced Techniques in IoID

1. **Relational Reinforcement Learning:** IoID principles are increasingly applied in relational reinforcement learning, where agents use background knowledge to inform learning within relational environments. By integrating IoID, agents can discover policies based on relationships among entities, enhancing decision-making in dynamic settings.

- 2. **Probabilistic Logic Learning:** To address the limitation of noisy data, IoID can be extended to probabilistic logic learning, which incorporates probabilities into logic-based hypotheses. This allows for handling uncertainty in relational data and improves robustness in applications like natural language processing and knowledge base construction.
- 3. **Meta-Interpretive Learning (MIL):** MIL is an advanced technique in ILP that extends IoID by using higher-order logic to learn more abstract hypotheses. MIL enables the learning of recursive definitions and complex relationships, facilitating the discovery of patterns that require self-referential logic, such as genealogies or hierarchical structures.
- 4. **Hybrid Symbolic-Connectionist Models:** Recent advancements integrate IoID with neural networks, creating hybrid models that combine symbolic reasoning with neural representations. These models use IoID to interpret high-level relationships while leveraging neural networks for pattern recognition, enabling applications in explainable AI.
- 5. **Automated Reasoning in Knowledge Graphs:** IoID is increasingly used in reasoning over knowledge graphs, where the goal is to derive new relationships or infer missing links based on existing structured data. By using background knowledge encoded in graph form, IoID-based algorithms can infer logical relationships, assisting in tasks like recommendation systems and semantic search.

5.6 Inverting Resolution

The resolution rule is a fundamental and powerful inference rule used in first-order logic for deductive reasoning. It is both sound (meaning it only derives true statements if the premises are true) and complete (meaning it can derive any true statement expressible in the language). Resolution is widely used in automated theorem proving and logic-based systems.

Let L represent an arbitrary propositional literal, and let P and R be arbitrary propositional clauses. The resolution rule operates as follows:

$$\begin{array}{ccccc} & P & & V & & L \\ \neg & L & & v & & R \\ \hline & P & & V & & R \end{array}$$

Given the two clauses above the line, the resolution rule allows us to conclude the clause below the line. Intuitively, this makes sense: if $P \vee L$ and $\neg L \vee R$ are both true, then either P or R must also be true. This is because L and $\neg L$ cannot both be true simultaneously, so one of the other components (P or R) in the clauses must be true to satisfy the statements.

Thus, the resolution rule allows us to derive $P \vee R$, simplifying the logical structure and aiding in the deduction process.

5.6.1 Application of the Resolution Rule in Cigol

The resolution rule is a critical operator in **Cigol**, an inversion-based reasoning system. Cigol employs resolution with sequential covering but extends it to handle first-order representations, making it suitable for complex logic applications.

Consider two clauses:

- $C = A \vee B$
- $C_2 = B \vee D$

The resolution rule in Cigol applies as follows:

1. Any literal present in C but absent in C_1 must appear in C_2 .
2. For example:
 - $C_2 = A \vee \neg D$ or $C_2 = A \vee \neg D \vee B$

The literal occurring in C_1 but not in C is removed by the resolution rule, and its negation appears in C_2 .

Cigol utilizes inverting resolution with sequential covering, incorporating first-order representations to expand its capabilities in deductive inference. This enables Cigol to handle complex logical structures and to work effectively with both propositional and relational data.

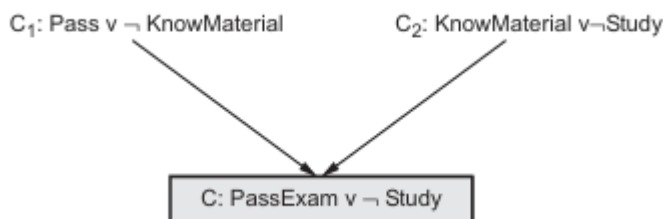


Fig. 5.3 Resolution.

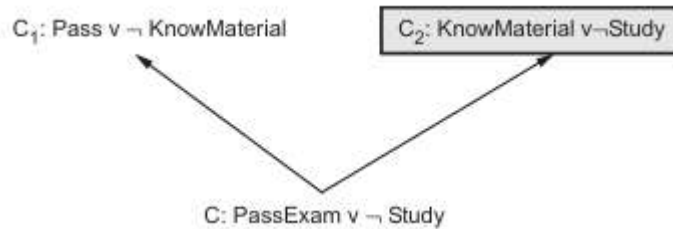


Fig. 5.4 Inverse resolution.

5.6.2 Advanced Concepts and Applications of Resolution in Automated Reasoning

1. **Resolution in First-Order Logic and Automated Theorem Proving:** The resolution rule serves as a cornerstone in automated theorem proving, particularly in systems that perform proofs in first-order logic. First-order resolution involves more complex structures, such as unification, where variable substitutions are made to match terms across clauses. This is essential for automated reasoning systems that require the manipulation of relational data and quantified statements.
2. **Use in Knowledge Representation and Reasoning:** Resolution-based reasoning is commonly applied in knowledge representation systems, where logical rules and facts are used to infer new information. For example, in artificial intelligence systems like expert systems, resolution helps derive conclusions based on known facts, enabling complex inferencing over large knowledge bases.
3. **Resolution with Constraints:** Advanced resolution methods integrate constraints to enhance logical reasoning in specific domains. For instance, **constrained resolution** restricts the application of the resolution rule to clauses satisfying certain conditions, improving efficiency by pruning unnecessary parts of the search space. This is particularly valuable in applications like constraint logic programming, where resource allocation and optimization problems are solved through logical deduction.
4. **Resolution in Non-Monotonic Reasoning:** Traditional logic systems assume monotonic reasoning, where adding new information never invalidates existing conclusions. However, many real-world problems involve non-monotonic reasoning, where new information may override previous conclusions. **Non-monotonic resolution** extends traditional resolution techniques to handle such cases, supporting applications in dynamic environments like belief revision, where agents need to adapt their knowledge based on changing information.
5. **Probabilistic Resolution and Fuzzy Logic:** In some applications, the strict true/false dichotomy of classical logic is insufficient, as uncertainty or ambiguity may be present in the data. Probabilistic resolution introduces probabilities into logical clauses, allowing for inferencing in uncertain environments.

- 6. **Fuzzy resolution** further extends this by incorporating degrees of truth, enabling reasoning over imprecise or gradational data, which is essential in fields like natural language processing and robotics.
- 7. **Parallel and Distributed Resolution Algorithms:** Given the computational intensity of resolution in large-scale logic systems, parallel and distributed algorithms have been developed to enhance performance. These algorithms distribute the resolution process across multiple processors, allowing for concurrent exploration of the logical search space. This is particularly useful in high-performance computing applications, such as verifying the correctness of complex software systems and large-scale scientific simulations.

5.7 Analytical Learning

In analytical learning, the input to the learner includes a hypothesis space H , training examples D , and an additional domain theory B , which consists of background knowledge used to explain the observed training examples. The goal of analytical learning is to derive a hypothesis h from H that is consistent with both the training data D and the domain theory B . Analytical learning leverages domain knowledge to enhance the learning process, making it especially useful when data is scarce or incomplete.

In contrast, inductive learning relies solely on training examples to generate a hypothesis. In inductive learning, the learner is provided with a hypothesis space H and a set of training examples $D = \{(x_1, f(x_1)), \dots, (x_n, f(x_n))\}$, where $f(x_i)$ denotes the target value for each instance x_i . The goal is to select a hypothesis h from H that fits the data, often without access to additional background knowledge.

Key Differences between Inductive and Analytical Learning

Parameters	Inductive Learning	Analytical Learning
Goal	Hypothesis fits data	Hypothesis fits domain theory
Justification	Statistical inference	Deductive inference
Merit	Requires little prior knowledge	Learns effectively from scarce data
Demerit	Scarcity of data, incorrect bias	Imperfect or incomplete domain theory

Table. 5.1 Difference between inductive and analytical learning.

5.7.1 Properties of a Learning Method

To evaluate the effectiveness of a learning method, several properties are considered:

- a. **No Domain Theory:** In the absence of domain theory, the method should perform as effectively as a purely inductive approach, relying on data alone.
- b. **Perfect Domain Theory:** When a perfect domain theory is available, the method should perform as effectively as a purely analytical approach, utilizing domain knowledge to derive accurate hypotheses.
- c. **Imperfect Domain Theory:** If the domain theory is incomplete or imperfect, the method should combine both inductive and analytical techniques, allowing it to outperform purely inductive or analytical approaches by compensating for deficiencies in data or knowledge.
- d. **Unknown Error Level:** The method should be adaptable to unknown levels of error within the training data, ensuring robustness against noisy or inconsistent data.
- e. **Unknown Error in Domain Theory:** The method should also handle unknown levels of error in the domain theory, adapting to inaccuracies in background knowledge.

5.7.2 Advanced Concepts in Analytical Learning

1. **Explanation-Based Learning (EBL):** EBL is a specialized form of analytical learning that derives generalizable rules from a single training example by using domain knowledge to explain why the example is an instance of a target concept. EBL is particularly useful in domains with complex or hierarchical structures, such as chess or medical diagnosis, where the learner needs to understand underlying principles rather than patterns.
2. **Combining Inductive and Analytical Learning (Hybrid Approaches):** Hybrid learning approaches combine inductive and analytical techniques to leverage the strengths of both. For example, **Inductive Logic Programming (ILP)** often uses background knowledge in a deductive manner while learning from examples inductively. This is especially effective in tasks that involve relational or structured data, like bioinformatics or knowledge graph completion.
3. **Abductive Learning:** Abductive reasoning, another form of inference, is often integrated with analytical learning to hypothesize explanations for observed phenomena. Abductive learning, used in diagnostic applications, generates the most plausible explanation for given data by constructing hypotheses that best explain the observations in conjunction with the domain theory.

4. **Meta-Learning:** Meta-learning, or "learning to learn," can extend analytical learning by adapting the learning process based on domain knowledge about learning itself. Meta-learning approaches, which include algorithm selection and hyperparameter tuning, optimize the learning process, particularly when transferring knowledge across related tasks or domains.
5. **Theory Revision and Refinement:** In practical applications, domain theories are often incomplete or inaccurate. Theory revision techniques allow analytical learning systems to refine domain knowledge based on new evidence from training data. This iterative process adjusts the domain theory to improve compatibility with observed data, facilitating more accurate predictions.
6. **Applications in Explainable AI (XAI):** Analytical learning aligns closely with goals in explainable AI, as it provides interpretable models grounded in domain knowledge. By generating hypotheses that are consistent with both data and prior knowledge, analytical learning models are inherently more interpretable, which is crucial in fields like healthcare, finance, and law, where transparency is essential.

5.8 Perfect Domain Theories

A **domain theory** is considered **correct** if each of its assertions accurately represents truths about the world. It is **complete** with respect to a given target concept and instance space if it fully accounts for every positive example in that space. In learning systems, a perfect domain theory is one that is both correct and complete, enabling the generation of consistent and accurate hypotheses.

5.8.1 Prolog and Logic Programming

Prolog is a logic programming language rooted in first-order logic, used extensively in artificial intelligence and computational linguistics. Unlike procedural languages, Prolog is declarative, meaning that programs consist of facts and rules, and computations are carried out by querying these relationships. Prolog's formalism in first-order logic enables it to represent complex relationships and rules in a structured and logical manner, making it suitable for tasks requiring deductive reasoning.

5.8.2 PROLOG-EBG (Explanation-Based Generalization)

PROLOG-EBG is an explanation-based learning system that uses a **sequential covering algorithm** to generalize rules based on a domain theory. It computes the most general rule that can justify the target concept by determining the **weakest pre-image** of an explanation, allowing it to create hypotheses that are just broad enough to cover the positive examples without being overly specific.

Key characteristics of PROLOG-EBG:

- It constructs **intermediate features** after analyzing examples, facilitating the formation of generalized rules.
- As a **deductive learning system**, it assumes the domain knowledge provided is both correct and complete. If this assumption is violated, the resulting learned concepts may be inaccurate or incomplete.
- PROLOG-EBG produces general hypotheses, represented as Horn clauses, that are logically justified based on prior knowledge.

In its pure form, PROLOG-EBG operates deductively rather than inductively. It derives hypotheses directly from the domain theory B without relying on statistical inference. By calculating the weakest pre-image of the explanation, it generates a hypothesis h that logically follows from B and covers the training data D .

Example of PROLOG-EBG Algorithm

The basic structure of the PROLOG-EBG algorithm is as follows:

PROLOG-EBG(TargetConcept, TrainingExamples, DomainTheory)

LearnedRules $\leftarrow \{\}$

Pos $\leftarrow$ positive examples from TrainingExamples

for each PositiveExample in Pos that is not covered by LearnedRules do

1. Explain: Explanation $\leftarrow$ explanation (proof) using DomainTheory that demonstrates how PositiveExample satisfies TargetConcept

2. Analyze: SufficientConditions $\leftarrow$ most general conditions of PositiveExample that satisfy TargetConcept according to Explanation

3. Refine: LearnedRules $\leftarrow$ LearnedRules + NewHornClause

where NewHornClause is of the form TargetConcept $\leftarrow$ SufficientConditions

Return LearnedRules

5.8.3 Steps in the PROLOG-EBG Algorithm

1. **Explain:** For each positive example not already covered by existing rules, generate an explanation (or proof) within the terms of the domain theory. This explanation shows how the positive example satisfies the target concept.
2. **Analyze:** Identify the **most general set of sufficient conditions** for the example that are required to meet the target concept according to the explanation. This step helps in determining which features of the example are essential to classify it positively without unnecessary specifics.
3. **Refine:** Add a new Horn clause to the set of learned rules, where the clause represents the target concept derived from the sufficient conditions.

The result is a set of generalized rules that logically follow from the domain theory and sufficiently cover the positive examples in the training data.

5.8.4 Properties of PROLOG-EBG

- ❖ **Justified Generalization:** By using a domain theory to explain individual examples, PROLOG-EBG generates hypotheses that are logically sound and consistent with background knowledge.
- ❖ **Dependency on Domain Theory:** PROLOG-EBG assumes a correct and complete domain theory. If the domain theory is inaccurate or incomplete, the learned concepts may also be erroneous or insufficient.
- ❖ **Sequential Covering Approach:** It covers one positive example at a time, refining the hypothesis space iteratively to cover all instances.

5.8.5 Limitations and Challenges of PROLOG-EBG

1. **Reliance on Perfect Domain Theory:** PROLOG-EBG's effectiveness is highly dependent on the accuracy and completeness of the domain theory. In cases where background knowledge is flawed or incomplete, the learned rules may fail to generalize correctly.
2. **Limited Flexibility in Handling Noisy Data:** Since PROLOG-EBG is a deductive learning system, it does not handle noisy or inconsistent data well. In real-world applications with data imperfections, pure deductive approaches may produce overly rigid hypotheses.
3. **Complexity and Computational Cost:** Generating explanations and analyzing the most general conditions can be computationally intensive, particularly in domains with large and complex datasets.

5.8.6 Advanced Applications and Extensions of PROLOG-EBG

1. **Integrating Inductive and Deductive Learning:** To overcome limitations in handling noisy data, hybrid approaches combine inductive learning with PROLOG-EBG. By leveraging statistical inference alongside deductive reasoning, these systems can achieve robust generalization even with imperfect domain theories.
2. **Meta-Interpretive Learning (MIL):** MIL extends the capabilities of PROLOG-EBG by allowing the creation of recursive rules and higher-order generalizations. This approach is especially useful in complex domains, such as natural language processing, where hierarchical or recursive patterns are prevalent.
3. **Applications in Knowledge-Based Systems:** PROLOG-EBG is widely used in knowledge-based AI systems, particularly in expert systems that require precise reasoning within well-defined domains, like medical diagnosis or legal reasoning. By grounding hypotheses in a domain theory, PROLOG-EBG produces interpretable models that align closely with expert knowledge.
4. **Explainable AI (XAI):** As AI applications increasingly require transparency, the PROLOG-EBG approach's reliance on domain theory makes it inherently interpretable. This interpretability is valuable in fields where decision explanations are critical, such as finance, healthcare, and policy-making.
5. **Extensions to Incomplete Theories:** Recent work on adaptive learning systems seeks to relax the assumption of a perfect domain theory. These systems iteratively refine and update the domain theory as new information becomes available, making PROLOG-EBG more applicable in dynamic, real-world environments.

5.9 Explanation-Based Learning (EBL)

Explanation-Based Learning (EBL) is a type of analytical learning where the learner processes each novel example by creating an explanation of how the observed target value for this example aligns with the domain theory. By examining this explanation, EBL identifies the general conditions that apply to the concept and refines its hypothesis to incorporate these generalized conditions.

In an EBL system:

- ❖ The system accepts a training example and uses background knowledge (domain theory) to explain why this example meets the target concept.
- ❖ It extracts only the relevant aspects of the example, filtering out extraneous details and forming a generalized explanation that can be applied to similar future problems.
- ❖ EBL transforms this explanation into operational rules or control knowledge, which guides the learning process and helps solve related problems effectively.

5.9.1 Elements of EBL

1. **Domain Theory:** Background information that describes the problem space, including relevant rules, definitions, and relationships.
2. **Goal Concept:** The specific concept or target the learner seeks to understand or classify, which guides the formulation of operational rules.
3. **Example:** A positive instance of the concept, illustrating the conditions under which the target concept is valid.
4. **New Domain Theory:** The modified theory, incorporating the learned generalization to define the target concept in a more robust way.

5.9.2 Phases of Explanation-Based Learning

1. Problem Solving

- ❖ The system uses the domain theory and a given example to solve a problem, generating a solution that justifies the concept.
- ❖ Actions taken to derive the solution are recorded as a trace, which serves as a basis for generalization.

2. Resolution Trace Analysis and Filtering

- ❖ The trace is analyzed to identify primitive and relevant actions, isolating only the essential steps required to solve the problem.
- ❖ Filtering criteria, based on domain knowledge, help remove non-essential parts of the trace, leaving a focused explanation.

3. Generalization of the Explanation

- ❖ EBL generalizes the explanation by substituting constants with variables, creating a broader rule applicable to similar situations.
- ❖ A regression algorithm (often goal regression) performs this substitution, ensuring that the generalized rule maintains the original meaning of the explanation.

4. Building New Knowledge

- ❖ The system translates the generalized explanation into a form that aligns with the domain's formal representation.
- ❖ This new knowledge may define predicates or control rules that guide future problem-solving actions.

5. Incorporating New Knowledge

- ❖ The new knowledge is integrated into the domain theory. If not frequently used, it may be periodically evaluated and discarded to maintain an efficient and relevant knowledge base.

5.9.3 Features and Characteristics of EBL

- ❖ **Feature Selection:** Choosing the right features is crucial, as irrelevant features can lead to inefficient generalizations.
- ❖ **Goal-Driven Learning:** EBL is goal-driven, focusing on generalizing only those aspects of examples that contribute to achieving the target concept.
- ❖ **Logical Proofs as Explanations:** In classical EBL, explanations are structured as logical proofs, providing a rigorous basis for the learned knowledge.
- ❖ **Control Knowledge Generation:** EBL produces control rules or operational knowledge, which guide the system's learning process by constraining future searches.

EBL is dynamic, incorporating domain knowledge through example-based explanations. Unlike inductive learning, which generalizes from data patterns, EBL generates hypotheses grounded in prior knowledge, making it suitable for tasks that require interpretability.

5.9.4 Knowledge Level Learning

Knowledge Level Learning (KLL) represents a high-level abstraction in computer systems, focusing on the use of domain knowledge to drive decision-making processes. In KLL, knowledge is described in terms of **determinations**—assertions that a particular attribute of an instance is defined by other attributes without specifying exact dependencies.

Example of Knowledge-Level Analytical Learning

Consider a target concept, “people who speak Hindi,” with the domain theory containing the determination: “The language a person speaks is determined by their nationality.”

- ❖ This theory alone does not provide sufficient information to classify instances as positive or negative.
- ❖ However, if given an example such as “Jon, a 23-year-old left-handed individual from the U.S., speaks Hindi,” the system can derive a broader rule (e.g., “all U.S. people speak Hindi”) using both the example and the initial domain theory.

Key Features of Knowledge Level Learning

1. **Abstract Reasoning:** KLL enables learning at a high level of abstraction, focusing on the relationships and dependencies among attributes rather than specific instance details.
2. **Implicit Determination Rules:** The learning process is guided by implicit rules, or determinations, which specify the attributes that define other attributes without requiring explicit dependencies.
3. **Broad Applicability:** KLL is useful in domains where complete causal information is unavailable but general rules can still be inferred from observations.

5.9.5 Advanced Concepts in Explanation-Based Learning and Knowledge-Level Learning

1. **Hybrid EBL-ILP Models:** Combining EBL with Inductive Logic Programming (ILP) enables systems to use both example-driven generalization and background knowledge to derive rules. This hybrid approach is especially effective in relational learning tasks and knowledge discovery in fields like biology and medicine.
2. **Meta-Explanation-Based Learning:** Meta-EBL extends EBL by focusing not only on explaining individual instances but also on the structure of the explanations themselves. Meta-EBL examines patterns in explanations, aiming to improve the efficiency of the learning process by generalizing over explanation structures.
3. **Incremental Learning in EBL:** Incremental EBL continuously integrates new information, allowing the system to refine hypotheses over time. This approach is valuable in dynamic domains where the system's knowledge needs to be updated regularly, such as in adaptive learning systems.
4. **Probabilistic Knowledge-Level Learning:** To handle uncertainty, probabilistic KLL integrates probabilistic inference into the knowledge-level framework. This allows for knowledge-level reasoning in domains where attributes may not have deterministic dependencies, enabling applications in areas like finance and risk assessment.
5. **Applications in Explainable AI (XAI):** Both EBL and KLL are valuable for explainable AI, as they produce hypotheses and control rules that are grounded in domain knowledge, making them inherently interpretable. These approaches are essential in fields where transparency is critical, such as healthcare and legal reasoning.

5.10 FOCL Algorithm

The **FOCL (First Order Combined Learner) Algorithm** is an extension of explanation-based and inductive learning methods that uses domain theory to enhance hypothesis generation. By leveraging background knowledge, FOCL expands the range of candidate specializations at each step, allowing it to refine hypotheses for more precise learning.

FOCL constructs hypotheses by iteratively refining its current hypothesis h through two primary operations:

1. **Adding Individual Literals:** For each operational literal (a statement grounded in the domain theory) that is not already part of h , FOCL creates a specialization by adding this literal to the preconditions of h . This single-literal addition refines the hypothesis step-by-step, gradually narrowing it to match the target concept.
2. **Adding Conjunctions of Literals:** FOCL can also add a logically sufficient set of operational conditions that directly satisfy the target concept according to the domain theory. By including a conjunction of literals, FOCL can quickly create a hypothesis that covers a broader range of cases consistent with the domain.

5.10.1 Steps in FOCL's Operation

- ❖ **Selecting Domain Theory Clauses:** FOCL begins by selecting a clause from the domain theory whose head (conclusion) aligns with the target concept.
- ❖ **Evaluating Information Gain:** If multiple domain theory clauses match the target concept, FOCL selects the clause with the highest information gain relative to the training examples. This prioritization ensures that the selected clause maximizes relevance to the target concept.

5.10.2 Form of FOCL Hypothesis

FOCL learns Horn clauses with the general structure:

$$c \leftarrow O_i \wedge O_b \wedge O_f$$

Where,

- ❖ c represents the target concept,
- ❖ O_i is an initial conjunction of operational literals, added one at a time by the syntactic operator,

- ❖ O_b is a conjunction of operational literals added in a single step based on the domain theory, and
- ❖ O_f is a final conjunction of operational literals added incrementally.

Note: Any of these sets (O_i, O_b, O_f) may be empty, allowing FOCL flexibility in constructing hypothesis that balance domain theory guidance and empirical data.

5.10.3 Specialization Generation in FOCL

FOCL generates candidate specializations using two complementary strategies:

1. **Syntactic Generation:** Generates candidate hypotheses based purely on syntactic rules, adding one literal at a time to refine the hypothesis incrementally.
2. **Domain-Theoretic Generation:** Uses the domain theory to propose broader candidate specializations, incorporating conjunctions of literals that are logically sufficient for the target concept.

By combining these strategies, FOCL strikes a balance between inductive data-driven learning and analytical learning guided by domain knowledge.

5.10.4 Hypothesis Selection in FOCL

FOCL's final hypothesis selection process considers both the accuracy of each candidate hypothesis on training data and the domain theory's influence. It follows a two-step evaluation:

1. **Syntactic Refinement Based on Performance:** Hypotheses are syntactically refined by adding literals to improve alignment with the target concept, based on training data performance.
2. **Domain Theory Conformance:** The domain theory's guidance ensures that the hypothesis remains logically consistent with background knowledge, enhancing interpretability.

5.10.5 Advanced Features of FOCL

1. **Hybrid of Inductive and Analytical Learning:** FOCL is a hybrid learning method, combining inductive methods with explanation-based (analytical) learning. This hybrid approach enables FOCL to leverage domain knowledge while maintaining adaptability to empirical data, making it more versatile than purely inductive or deductive algorithms.

2. **Optimization with Information Gain:** FOCL's use of information gain to prioritize domain theory clauses ensures that it selects the most relevant clauses for hypothesis refinement. This optimization is particularly effective in large hypothesis spaces, where choosing the most informative path reduces computational complexity.
3. **Handling Uncertainty in Domain Knowledge:** FOCL's reliance on domain theory is flexible, allowing it to adapt to scenarios where the domain theory is incomplete or imperfect. By balancing syntactic generation with domain-theoretic guidance, FOCL can still produce accurate hypotheses even when domain knowledge is partially incorrect.
4. **Applications in Relational and Symbolic Learning:** FOCL is well-suited for tasks involving relational data or complex symbolic reasoning, as its ability to incorporate domain theory allows it to capture intricate relationships that may be missed by purely data-driven methods. Fields like bioinformatics, legal reasoning, and automated theorem proving benefit from FOCL's ability to combine logical and data-driven insights.

5.11 Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning where an **agent** learns how to make decisions by interacting with an **environment**. Unlike supervised learning, where labeled data provides immediate feedback, RL agents receive feedback in the form of **rewards** or **penalties** based on the actions they take. The goal of the agent is to **maximize cumulative rewards** over time by learning a policy that maps states of the environment to actions.

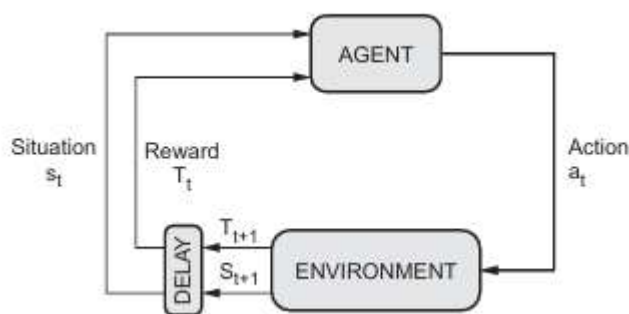


Fig. 5.5 Reinforcement learning.

5.11.1 Key Components of Reinforcement Learning

1. **Agent:** The learner or decision-maker in the system. The agent takes actions based on its policy to maximize rewards.

2. **Environment:** Everything outside the agent that the agent interacts with. The environment responds to the agent's actions by transitioning to new states and providing rewards.
3. **State:** A representation of the current situation of the environment. The state gives the agent information about the environment at any given time.
4. **Action:** Choices available to the agent. The action impacts the environment and causes a transition to a new state.
5. **Reward:** Feedback from the environment after an action is taken. Rewards guide the agent toward its goal, and the agent aims to maximize the cumulative reward over time.
6. **Policy:** A strategy that defines the agent's actions based on the current state. It can be deterministic (specific actions in specific states) or stochastic (probabilistic actions based on state).
7. **Value Function:** A function that estimates the expected cumulative reward for each state, helping the agent evaluate the long-term benefits of actions rather than just immediate rewards.
8. **Model of the Environment:** In some cases, the agent has a model that predicts the environment's responses to actions, which it can use for planning.

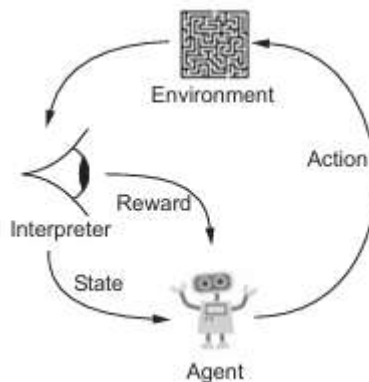


Fig. 5.6 Key components of RL.

5.11.2 Reinforcement Learning Process

1. **Initialization:** The agent starts with no prior knowledge, often with a random or exploratory policy.
2. **Interaction:** The agent interacts with the environment by observing states, taking actions, receiving rewards, and observing new states.
3. **Learning:** Based on the observed rewards and state transitions, the agent updates its policy or value functions to improve its future decision-making.

4. **Evaluation and Adjustment:** Over time, the agent evaluates its accumulated experience to adjust its policy. This process continues until the agent's policy converges to an optimal or satisfactory level, maximizing cumulative rewards.

5.11.3 Types of Reinforcement Learning

- ❖ **Model-Free RL:** The agent learns purely from experience without a model of the environment. Algorithms like Q-learning and SARSA are common in model-free RL.
- ❖ **Model-Based RL:** The agent builds a model of the environment's dynamics and uses it to plan future actions. Model-based approaches are efficient but may be computationally intensive or require complex models.
- ❖ **Value-Based RL:** Focuses on learning the value function, which estimates the long-term reward of each state (e.g., Q-learning).
- ❖ **Policy-Based RL:** Learns a direct mapping from states to actions (policy), optimizing it to maximize rewards (e.g., REINFORCE).

Example: Navigating a Maze

Consider a scenario where an agent needs to navigate through a maze filled with obstacles to reach a reward. The objective for the agent is to find the most efficient path that avoids hurdles and leads to the reward. This example illustrates the problem simply: the agent must explore different paths and learn the optimal route to reach the goal while overcoming obstacles along the way.

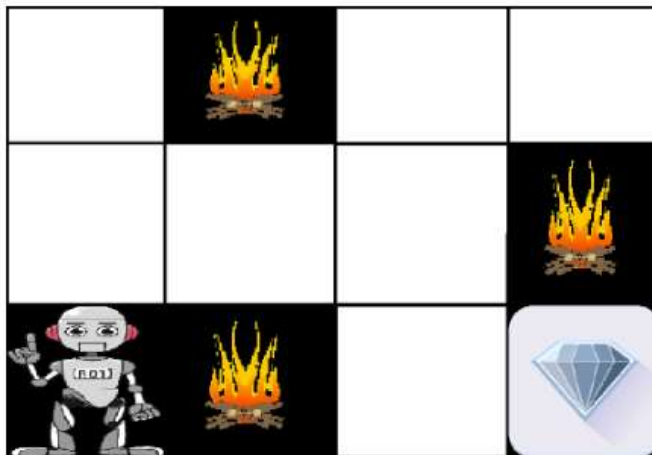


Fig. 5. Navigating a Maze

In the image above, we see a robot, a diamond, and fire. The robot's objective is to reach the diamond, which represents the reward, while avoiding obstacles such as fire.

The robot learns by exploring different paths, eventually identifying the one that leads to the diamond with the fewest obstacles. Each correct move earns the robot a reward, while each incorrect move reduces its reward. The total reward is calculated once the robot successfully reaches the diamond.

Key Points in Reinforcement Learning

- **Input:** The initial state from which the model begins.
- **Output:** There are multiple possible outputs, as different solutions may exist for the problem.
- **Training:** Training depends on the input. The model produces a state, and the user provides feedback by rewarding or penalizing the model based on its actions.
- **Learning Process:** The model continuously learns through this feedback.
- **Optimal Solution:** The best solution is the one that maximizes the total reward.

Example: CartPole Environment in OpenAI Gym

The CartPole environment is a well-known reinforcement learning challenge in which the objective is to keep a pole balanced on a moving cart by applying forces either to the left or right.

```
import gym

import numpy as np

import warnings

# Suppress specific deprecation warnings

warnings.filterwarnings("ignore", category=DeprecationWarning)

# Load the environment with render mode specified

env = gym.make('CartPole-v1', render_mode="human")

# Initialize the environment to get the initial state

state = env.reset()
```

Print the state space and action space

```
print("State space:", env.observation_space)
```

```
print("Action space:", env.action_space)
```

Run a few steps in the environment with random actions

```
for _ in range(10):
```

```
    env.render() # Render the environment for visualization
```

```
    action = env.action_space.sample() # Take a random action
```

Take a step in the environment

```
    step_result = env.step(action)
```

Check the number of values returned and unpack accordingly

```
    if len(step_result) == 4:
```

```
        next_state, reward, done, info = step_result
```

```
        terminated = False
```

```
    else:
```

```
        next_state, reward, done, truncated, info = step_result
```

```
        terminated = done or truncated
```

```
    print(f'Action: {action}, Reward: {reward}, Next State: {next_state}, Done: {done}, Info: {info}')
```

```
    if terminated:
```

```
        state = env.reset() # Reset the environment if the episode is finished
```

```
env.close() # Close the environment when done
```

Output

State space: Box([-4.8000002e+00 -3.4028235e+38 -4.1887903e-01 -3.4028235e+38], [4.8000002e+00 3.4028235e+38 4.1887903e-01 3.4028235e+38], (4,), float32)

Action space: Discrete(2)

Action: 0, Reward: 1.0, Next State: [0.00661033 -0.21114323 -0.00940697 0.30795237], Done: False, Info: {}

Action: 0, Reward: 1.0, Next State: [0.00238747 -0.40612987 -0.00324792 0.5976538], Done: False, Info: {}

Action: 0, Reward: 1.0, Next State: [-0.00573513 -0.60120624 0.00870516 0.8893119], Done: False, Info: {}

Action: 0, Reward: 1.0, Next State: [-0.01775926 -0.79644525 0.02649139 1.1847186], Done: False, Info: {}

Action: 1, Reward: 1.0, Next State: [-0.03368816 -0.60167676 0.05018577 0.90045595], Done: False, Info: {}

Action: 0, Reward: 1.0, Next State: [-0.04572169 -0.7974415 0.06819488 1.2084818], Done: False, Info: {}

Action: 1, Reward: 1.0, Next State: [-0.06167053 -0.60326344 0.09236452 0.9379254], Done: False, Info: {}

Action: 1, Reward: 1.0, Next State: [-0.0737358 -0.40950003 0.11112303 0.6756358], Done: False, Info: {}

Action: 0, Reward: 1.0, Next State: [-0.08192579 -0.6059764 0.12463574 1.0011356], Done: False, Info: {}

Action: 1, Reward: 1.0, Next State: [-0.09404532 -0.4127204 0.14465846 0.75004834], Done: False, Info: {}

Explanation:

1. **Environment Setup:** We initialize the CartPole environment using `gym.make`, setting `render_mode="human"` to enable visual representation of the environment.
2. **State and Action Spaces:** The state space and action space are displayed to understand the environment's dimensions and the types of actions the agent can take.
3. **Random Actions:** The agent executes random actions for several steps, with each step displaying the resulting state transitions, rewards, and additional information.

4. **Termination Handling:** If the episode ends (for example, if the pole falls), the environment resets to start a new episode.

5.11.4 Applications of Reinforcement Learning

Reinforcement learning has a wide range of applications across various fields:

- ❖ **Robotics:** RL enables robots to learn motor control, navigate environments, and adapt to changing conditions.
- ❖ **Finance:** RL models are used for portfolio optimization, automated trading, and risk management.
- ❖ **Healthcare:** RL helps in personalized treatment planning, medical diagnosis, and optimizing resource allocation in healthcare systems.
- ❖ **Gaming:** DRL has achieved human-level performance in games like Go, chess, and video games, demonstrating the potential of RL in solving complex decision-making tasks.
- ❖ **Autonomous Systems:** RL is foundational in self-driving cars, where agents learn to navigate traffic and make decisions based on sensor data.

5.12 Markov Decision Process

Reinforcement Learning (RL) is a branch of Machine Learning in which machines or software agents learn to identify the ideal actions within a specific context to maximize their performance. This learning process requires a reward feedback mechanism, known as the reinforcement signal, which enables the agent to adjust its behavior.

RL is characterized by a specific problem type, and all approaches to solving it are categorized as Reinforcement Learning algorithms. The core of the problem involves an agent choosing the optimal action based on its current state. When this decision-making process is iterative, the problem is described as a Markov Decision Process (MDP).

An MDP model consists of:

- ❖ A set of possible states, S .
- ❖ A set of models.
- ❖ A set of possible actions, A .
- ❖ A reward function, $R(s, a)$, which assigns a real value based on actions in given states. A policy, which guides the agent's actions, serves as the solution to a Markov Decision Process.

States: $\mathbf{S}$

Model: $\mathbf{T(S, a, S')} \sim \mathbf{P(S'}$
 $\quad \quad \quad \quad \quad | \mathbf{S, a)}$

Actions: $\mathbf{A(S), A}$

Reward: $\mathbf{R(S), R(S, a), R(S, a, S')}$

 $\Pi(\mathbf{S}) \rightarrow \mathbf{a}$

Policy: Π^*

State: A state represents a collection of tokens that define every possible situation an agent can be in.

Model: Also known as the Transition Model, a model defines the effect of an action within a state.

Specifically, $T(S, a, S')$ represents a transition, where being in state S and performing an action a leads to state S' . For stochastic (uncertain or probabilistic) actions, we use $P(S' | S, a)$, the probability of reaching state S' if action a is taken in state S . The Markov property states that the outcome of an action depends only on the current state, not on the sequence of events that preceded it.

Actions: Actions, denoted by A , encompass all possible actions an agent can take. $A(s)$ specifies the set of actions available when in state S .

Reward: A reward is a real-valued function that indicates the benefit of a particular state or action. $R(s)$ defines the reward for simply being in state S . $R(S, a)$ denotes the reward for taking action a while in state S . $R(S, a, S')$ indicates the reward for being in state S , performing action a , and arriving in state S' .

Policy: A policy provides a solution to the Markov Decision Process (MDP). It is a mapping from states S to actions a , specifying the optimal action a to take while in each state S .

Let's consider an example of a **Markov Decision Process (MDP)** in a different setting: a **delivery robot** navigating a warehouse to deliver packages.

Scenario: Warehouse Delivery Robot

Imagine a robot operating in a small warehouse with aisles and storage areas. The warehouse is organized in a grid layout with some obstacles (walls) that the robot needs to avoid. The robot's goal is to deliver packages from the **starting location** to a designated **delivery point** while minimizing time and avoiding any obstacles.

1. States (S)

- ❖ Each cell in the warehouse grid represents a unique **state** that the robot can be in.
- ❖ Let's label the warehouse cells as follows:

(0,0) (0,1) (0,2) (0,3)

(1,0) (1,1) (1,2) (1,3)

(2,0) (2,1) (2,2) (2,3)

(3,0) (3,1) (3,2) (3,3)

- The **starting state** is (0,0), where the robot starts with the package.
- The **delivery point** is at (3,3).
- Obstacles or walls are placed at certain cells, such as (1,1) and (2,2), which the robot cannot pass through.

2. Actions (A)

- ❖ The robot has four possible actions in any state:
 - **Up:** Moves the robot one cell up.
 - **Down:** Moves the robot one cell down.
 - **Left:** Moves the robot one cell left.
 - **Right:** Moves the robot one cell right.
- ❖ If the robot tries to move into an obstacle or beyond the grid boundary, it remains in the same cell.

3. Transition Model (T)

- ❖ The transition model defines how the robot moves between states.
- ❖ In a deterministic model, the robot always moves to the intended cell unless blocked by an obstacle or boundary.
- ❖ In a stochastic model, the robot might end up in an unintended cell due to minor positioning errors or random disturbances. For example:
 - **80% chance** of moving in the intended direction.
 - **10% chance** of veering one cell to the left of the intended direction.
 - **10% chance** of veering one cell to the right.

If the robot is in cell (0,2) and chooses to move **Down**, there's an 80% chance it will move to (1,2), a 10% chance it will end up in (0,1), and a 10% chance it will end up in (0,3).

4. Rewards (R)

- ❖ The reward function assigns values to different states and actions:
 - **Delivery point (3,3)**: +20 reward. This motivates the robot to reach the delivery location.
 - **Obstacle cells (1,1) and (2,2)**: -10 penalty if the robot attempts to move into them, as it represents wasted time or risk.
 - **All other cells**: -1 reward to penalize each move slightly, encouraging the robot to find the shortest path to the goal.

5. Policy (π)

- ❖ A **policy** maps each state to an action that maximizes the expected reward.
- ❖ For example, a policy for the robot might be:
 - Move **Right** from (0,0) to (0,1).
 - Continue moving **Down** or **Right** to navigate around obstacles and eventually reach (3,3).
- ❖ An optimal policy will direct the robot to avoid cells with obstacles while moving efficiently toward the delivery point.

Example Execution

Let's see how the robot might navigate the warehouse under a deterministic policy:

1. **Starting at (0,0):**

- The policy suggests the robot **Move Right** to (0,1).

2. **State (0,1):**

- The policy suggests the robot **Move Right** again to (0,2).

3. **State (0,2):**

- The policy suggests the robot **Move Down** to (1,2), avoiding the obstacle at (1,1).

4. **State (1,2):**

- The policy suggests the robot **Move Down** to (2,2), but (2,2) is an obstacle. The policy instead directs the robot to **Move Right** to (1,3).

5. **State (1,3):**

- The policy suggests the robot **Move Down** to (2,3).

6. **State (2,3):**

- The policy suggests the robot **Move Down** again to (3,3), reaching the **delivery point**.

In a **stochastic environment**, the robot may not move exactly as planned due to potential errors in movement. For example, if the robot tries to move **Right** from (0,0) and veers to (0,1) by chance, the policy would adjust accordingly. Optimal policies in such settings aim to minimize risk by finding the safest routes that avoid high-penalty obstacles.

5.13 Q-Learning

Q-learning is a model-free reinforcement learning algorithm that enables an agent to learn the optimal action-selection policy by updating Q-values, which indicate the expected rewards for actions taken in specific states.

Reinforcement Learning is a learning paradigm in which an agent learns to behave optimally within an environment by continuously interacting with it. Throughout the learning process, the agent encounters various situations, known as states, within this environment. While in a particular state, the agent can choose from a set of permissible actions, each of which yields different rewards or penalties. Over time, the agent learns to maximize these rewards to act optimally in any given state. Q-learning is a foundational form of reinforcement learning that leverages Q-values (also referred to as action values) to iteratively refine the agent's behavior.

This example provides a clearer understanding of reinforcement learning.

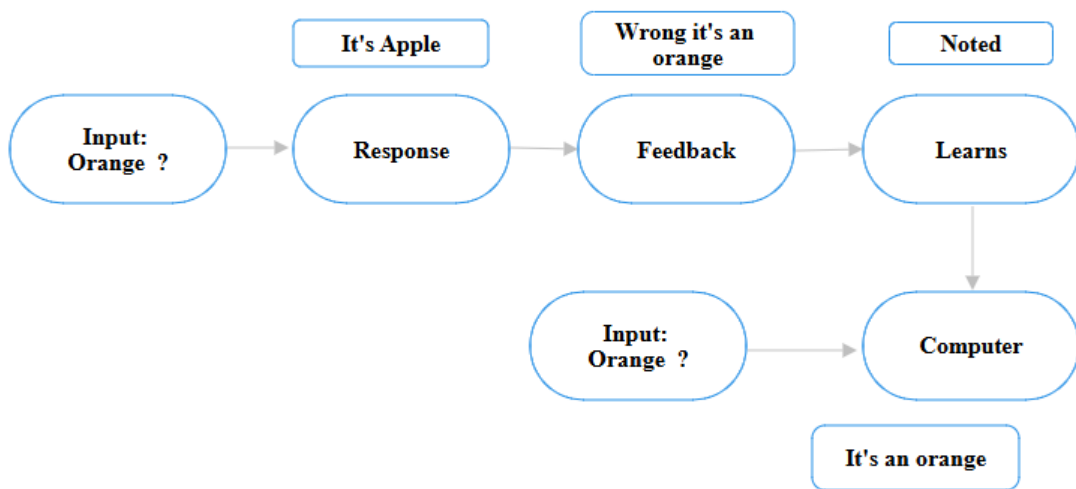


Fig. 5.7 Q-Learning.

Q-learning is a well-known model-free reinforcement learning algorithm widely applied in machine learning and artificial intelligence. It falls under temporal difference learning techniques, where an agent learns by observing outcomes, interacting with its environment, and receiving feedback in the form of rewards.

5.13.1 Key Components of Q-learning

1. Q-Values or Action-Values

Q-values are estimates for specific state-action pairs, denoted as $Q(S,A)$, which indicate the value of taking action A in state S . These values are updated iteratively using a Temporal Difference (TD) update rule.

2. Rewards and Episodes

An agent begins in a starting state and makes transitions between states based on chosen actions and the environment. At each transition, the agent receives a reward and moves to the next state. An episode ends when the agent reaches a terminal state, with no further transitions possible.

3. Temporal Difference (TD) Update

The TD update rule can be defined as:

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$$

- ❖ S: Current state
- ❖ A: Current action, chosen according to a policy
- ❖ S': Next state
- ❖ A': Next action with the maximum Q-value in state S'
- ❖ R: Reward from the environment for action A in state S
- ❖ γ : Discount factor for future rewards, $0 < \gamma \leq 1$
- ❖ α : Learning rate for updating Q-values.

4. Choosing Actions with the ϵ -Greedy Policy

In the ϵ -greedy strategy:

- ❖ With probability $1 - \epsilon$, the agent exploits by choosing the action with the highest Q-value.
- ❖ With probability ϵ , it explores by randomly selecting an action to discover potentially beneficial actions.

5.13.2 How Q-Learning Works

Q-learning operates iteratively, with an agent exploring the environment and continuously updating Q-values based on feedback. Essential elements include:

- ❖ **Agent:** Interacts with the environment, making decisions.
- ❖ **States:** Represent the agent's position in the environment.
- ❖ **Actions:** Choices the agent can make in any given state.
- ❖ **Rewards:** Feedback based on the agent's actions.

- ❖ **Episodes:** Sequences of states and actions that conclude when the agent reaches a terminal state.
- ❖ **Q-values:** Metrics that guide the agent in choosing optimal actions in specific states.

5.13.3 Methods for Determining Q-Values

1. Temporal Difference

Q-values are updated by comparing current and previous state-action values.

2. Bellman's Equation

This recursive formula, created by Richard Bellman in 1957, calculates the value of a state-action pair:

$$Q(s, a) = R(s, a) + \gamma \max_a Q(s', a)$$

- ❖ $Q(s, a)$: Q-value for state-action pair
- ❖ $R(s, a)$: Immediate reward for action a in state s
- ❖ γ : Discount factor
- ❖ $\max_a Q(s', a)$: Maximum Q-value for the next state s' .

Q-table

The Q-table records expected rewards for actions in each state. As the agent learns, the Q-table updates to guide it in selecting actions that maximize rewards.

Implementing Q-Learning

Here's a Python implementation using Q-learning:

```
import numpy as np
```

```
# Environment setup
```

```
n_states = 16 # Total states
```

```
n_actions = 4 # Actions (up, down, left, right)
```

```
goal_state = 15 # Goal state
```

Initialize Q-table with zeros

```
Q_table = np.zeros((n_states, n_actions))
```

Hyperparameters

```
learning_rate = 0.8
```

```
discount_factor = 0.95
```

```
exploration_prob = 0.2
```

```
epochs = 1000
```

Q-learning algorithm

```
for epoch in range(epochs):
```

```
    current_state = np.random.randint(0, n_states) # Start randomly
```

```
    while current_state != goal_state:
```

```
        # Action selection (epsilon-greedy)
```

```
        if np.random.rand() < exploration_prob:
```

```
            action = np.random.randint(0, n_actions) # Explore
```

```
        else:
```

```
            action = np.argmax(Q_table[current_state]) # Exploit
```

Move to the next state

```
    next_state = (current_state + 1) % n_states
```

Reward function

```
    reward = 1 if next_state == goal_state else 0
```

Update Q-value

```
Q_table[current_state, action] += learning_rate * (
    reward + discount_factor * np.max(Q_table[next_state]) - Q_table[current_state,
action]
)

current_state = next_state # Move to next state
```

Print learned Q-table

```
print("Learned Q-table:")
```

```
print(Q_table)
```

5.13.4 Advantages of Q-Learning

- Effective for achieving long-term goals, often mirroring human learning patterns.
- Corrects mistakes during training and prevents their recurrence.
- Provides optimal solutions for specific problems.

5.13.5 Disadvantages of Q-Learning

- ❖ Gathering real-world samples can be costly, especially for physical systems like robotics.
- ❖ Combining Q-learning with other techniques, like deep learning, can help address some limitations.

5.13.6 Applications of Q-learning

Q-learning has many applications, including:

- ❖ **Atari Games:** Used with Deep Q Networks (DQN) for superhuman performance in games like Space Invaders.
- ❖ **Robot Control:** Helps robots navigate, avoid obstacles, and optimize movements.
- ❖ **Traffic Management:** Optimizes traffic signals and route planning to reduce congestion.
- ❖ **Algorithmic Trading:** Enables automated trading systems to adapt strategies based on market conditions.

- ❖ **Personalized Treatment Plans:** Recommends personalized medical interventions by analyzing patient data and response.

5.14 Temporal Difference (TD) learning

Temporal Difference (TD) learning is a fundamental method in reinforcement learning (RL) where an agent learns by iteratively updating its predictions based on the difference between observed and expected outcomes over time. It combines ideas from dynamic programming and Monte Carlo methods, making it both powerful and efficient in environments where learning can be done from incomplete information.

5.14.1 Key Concepts of Temporal Difference Learning

1. TD Error:

The core idea in TD learning is the temporal difference error (TD error), which quantifies the difference between the predicted and observed rewards over one step. It is calculated as:

$$\text{TD Error} = R + \gamma \cdot V(S') - V(S)$$

Where,

- R is the reward received after taking an action in the current state S .
- γ is the discount factor ($0 < \gamma \leq 1$), representing the value of future rewards.
- $V(S')$ is the estimated value of the next state, and $V(S)$ is the current state's estimated value.

2. TD Update Rule:

The TD learning update rule modifies the value of the current state by incorporating the TD error. For value function V , the update is:

$$V(S) \leftarrow V(S) + \alpha \cdot (R + \gamma \cdot V(S') - V(S))$$

Where,

- α is the learning rate, determining the size of the update.
- $V(S)$ is the current value estimate of state S , updated incrementally based on the TD error.

3. Bootstrapping:

TD methods use bootstrapping, meaning that the value of the current state is updated based on estimates of future values, rather than waiting for a final outcome as in Monte Carlo methods. This allows the agent to learn from each step without having to wait for the episode's end.

4. On-Policy vs. Off-Policy TD Learning:

- **On-Policy TD (e.g., SARSA):** Learns the value of the policy the agent is following. The update uses the action chosen by the agent's policy.
- **Off-Policy TD (e.g., Q-Learning):** Learns the optimal policy independently of the agent's actions, using the best possible action from the current Q-value estimates.

5.14.2 Types of TD Learning Methods

1. SARSA (State-Action-Reward-State-Action):

An on-policy TD algorithm where the update is based on the action taken by the current policy, often using an ϵ -greedy strategy.

2. Q-Learning:

An off-policy TD algorithm where the update uses the maximum Q-value of the next state, which aims to learn an optimal policy by always considering the best possible future action.

3. TD(λ):

A more generalized form of TD learning that uses a decay parameter, λ , to blend learning from multiple steps in the future, enhancing learning speed and convergence.

5.14.3 Advantages of TD Learning

- ❖ **Incremental Updates:** Allows for real-time learning, which is beneficial in continuous or large environments.
- ❖ **Efficiency:** Balances between Monte Carlo (full episode) methods and dynamic programming (complete model-based) methods, making it suitable for many practical RL tasks.
- ❖ **Applicability:** Works well in both model-free and model-based contexts, allowing it to adapt across different types of RL problems.

5.14.4 Applications of Temporal Difference Learning

TD learning is widely used in:

- ❖ **Game AI:** For strategy games, TD learning helps models update values and policies in real-time as they play.
- ❖ **Robotics:** Enables real-time learning from continuous interactions in dynamic environments.
- ❖ **Finance:** Assists in sequential decision-making tasks like portfolio optimization.
- ❖ **Healthcare:** Useful for learning optimal policies in treatment planning based on patient feedback.

Applications

6.1 Image Recognition

Image recognition is one of the foundational applications of machine learning, allowing systems to identify objects, people, places, and other elements within digital images. One common use of image recognition technology is in automatic friend tagging on social media platforms. For example, Facebook uses machine learning for its auto-tagging suggestion feature, where it identifies friends in uploaded photos and suggests their names for tagging. This technology is powered by face detection and recognition algorithms developed under Facebook's "DeepFace" project, a deep learning-based model designed to recognize faces and identify people in photos accurately. By leveraging deep learning, DeepFace can analyze facial features and patterns, providing accurate suggestions for tagging users in photos. give a example coding for image recognition.

Implementation Code:

```
# Import necessary libraries

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

from tensorflow.keras.datasets import cifar10

from tensorflow.keras.utils import to_categorical

import matplotlib.pyplot as plt

import numpy as np


# Load and preprocess the CIFAR-10 dataset

(x_train, y_train), (x_test, y_test) = cifar10.load_data()


# Normalize pixel values to be between 0 and 1

x_train, x_test = x_train / 255.0, x_test / 255.0
```

```
# Convert class vectors to binary class matrices (one-hot encoding)

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)


# Define class names for CIFAR-10

class_names = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse",
               "ship", "truck"]


# Build the CNN model

model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])


# Compile the model

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

```
# Train the model

history      =      model.fit(x_train,      y_train,      epochs=10,      batch_size=64,
validation_data=(x_test, y_test))

# Evaluate the model on test data

test_loss, test_accuracy = model.evaluate(x_test, y_test)

print(f"Test accuracy: {test_accuracy * 100:.2f}%")

# Function to make predictions and visualize

def plot_predictions(index):

    plt.figure(figsize=(2, 2))

    plt.imshow(x_test[index])

    plt.axis('off')

    pred = model.predict(np.expand_dims(x_test[index], axis=0))

    predicted_class = class_names[np.argmax(pred)]

    true_class = class_names[np.argmax(y_test[index])]

    plt.title(f"Predicted: {predicted_class}\nTrue: {true_class}")

    plt.show()

# Test the model with a few examples

for i in range(5):

    plot_predictions(i)
```

Output image:



Fig. 6.1 Image Recognition.

6.2 Speech Recognition

Speech recognition is a process that enables computers to interpret and convert spoken language into text. This is the underlying technology for features like **Google's "Search by Voice"** option. It falls under **speech-to-text** or **computer speech recognition** applications, where spoken words are translated into text that machines can process.

Various virtual assistants such as Google Assistant, Apple's Siri, Microsoft's Cortana, and Amazon's Alexa utilize speech recognition technology to execute voice-based commands. Machine learning algorithms are central to these systems, as they help the assistant learn from interactions and improve accuracy over time. Speech recognition is widely used in areas like virtual assistants, automated transcription, and voice command interfaces in devices.

Implementation Code:

```
import speech_recognition as sr

from pydub import AudioSegment

import matplotlib.pyplot as plt

import numpy as np


# Initialize recognizer class (for recognizing the speech)
recognizer = sr.Recognizer()


# Record audio from the microphone
def record_audio():

    with sr.Microphone() as source:

        print("Recording audio...")

        recognizer.adjust_for_ambient_noise(source) # Adjust for ambient noise

        audio = recognizer.listen(source)

        print("Recording complete.")

    return audio


# Function for feature extraction (dummy example with wave visualization)
def feature_extraction(audio_data):

    # Convert audio to numpy array for visualization (this is just a basic example)
    audio_samples = np.frombuffer(audio_data.get_wav_data(), np.int16)

    plt.figure(figsize=(10, 4))

    plt.plot(audio_samples)

    plt.title("Waveform of Recorded Audio")

    plt.xlabel("Sample Index")

    plt.ylabel("Amplitude")
```

```
plt.show()
```

```
# Normally, feature extraction would involve extracting MFCCs, but this is for illustration.
```

```
return audio_samples
```

```
# Function to recognize speech using Google Web Speech API
```

```
def recognize_speech(audio):
```

```
    try:
```

```
        # Recognize speech using Google Web Speech API
```

```
        print("Transcribing audio...")
```

```
        text = recognizer.recognize_google(audio)
```

```
        print("Transcription complete.")
```

```
        return text
```

```
    except sr.UnknownValueError:
```

```
        print("Speech Recognition could not understand the audio")
```

```
        return None
```

```
    except sr.RequestError as e:
```

```
        print(f"Could not request results from Google Speech Recognition service; {e}")
```

```
        return None
```

```
# Putting it all together
```

```
def speech_recognition_pipeline():
```

```
    # Step 1: Acoustic Signal Processing (Recording Audio)
```

```
    audio = record_audio()
```

```
    # Step 2: Feature Extraction
```

```
    features = feature_extraction(audio)
```

```
# Step 3: Acoustic Model and Language Model (Handled by recognizer in backend)

# Step 4: Decoding

recognized_text = recognize_speech(audio)

if recognized_text:

    print(f"Recognized Text: {recognized_text}")

else:

    print("No transcription available.")

# Run the pipeline

speech_recognition_pipeline()
```

Output image:

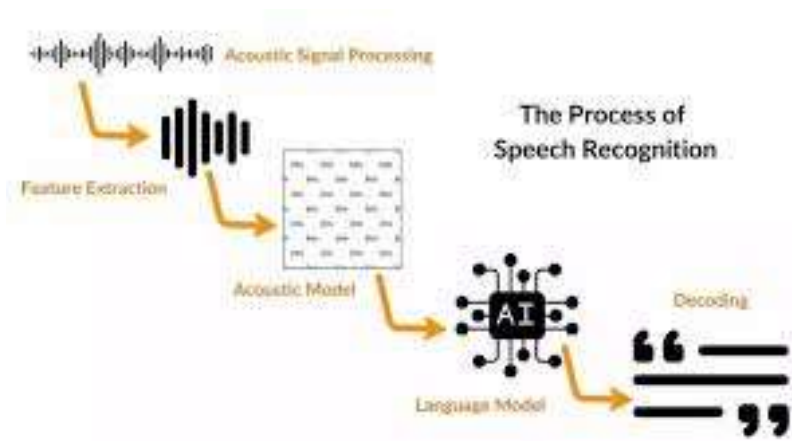


Fig. 6.2 Speech Recognition.

6.3 Traffic Prediction

Traffic prediction, as seen in tools like **Google Maps**, uses machine learning to provide accurate route information, identify the shortest paths, and predict traffic conditions (clear, slow, or congested). Traffic predictions are generated based on two key factors:

- ❖ **Real-time vehicle location data:** Google Maps gathers real-time location data from app users, creating a live network of traffic conditions.

- ❖ **Historical traffic patterns:** Past data on traffic conditions at specific times and days are also used to predict likely traffic patterns.

This system continuously improves as more users contribute data, making traffic predictions more reliable and accurate over time.

Implementation

```
# Step 1: Import Necessary Libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Step 2: Simulate Data or Load Your Own Data
```

```
# Uncomment the following lines if you want to upload your own CSV file
```

```
# from google.colab import files
```

```
# uploaded = files.upload() # Upload your file
```

```
# Load the data into a DataFrame (replace 'traffic_data.csv' with your file name)
```

```
# df = pd.read_csv('traffic_data.csv', parse_dates=['Date'])
```

```
# Generate sample data for illustration purposes if you don't have your own file
```

```
date_range = pd.date_range(start='2015-11-01', end='2017-07-01', freq='D')
```

```
data = {
```

```
    'Date': np.tile(date_range, 4), # Duplicate date range for 4 junctions
```

```
    'Vehicles': np.random.poisson(lam=30, size=len(date_range) * 4), # Random vehicle counts
```

```
    'Junction': np.repeat([1, 2, 3, 4], len(date_range)) # Four junctions
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# Step 3: Prepare the Data for Plotting
```

```
# Ensure 'Date' column is in datetime format
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Pivot data to have each junction in a separate column
```

```
pivot_df = df.pivot(index='Date', columns='Junction', values='Vehicles')
```

```
# Step 4: Plot the Data
```

```
# Plot the stacked area chart
```

```
plt.figure(figsize=(20, 5))
```

```
pivot_df.plot.area(stacked=True, alpha=0.5, colormap='Set1')
```

```
plt.title("Years of Traffic at Junctions")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Vehicles in Number")
```

```
plt.legend(title="Junction", loc="upper right")
```

```
plt.show()
```

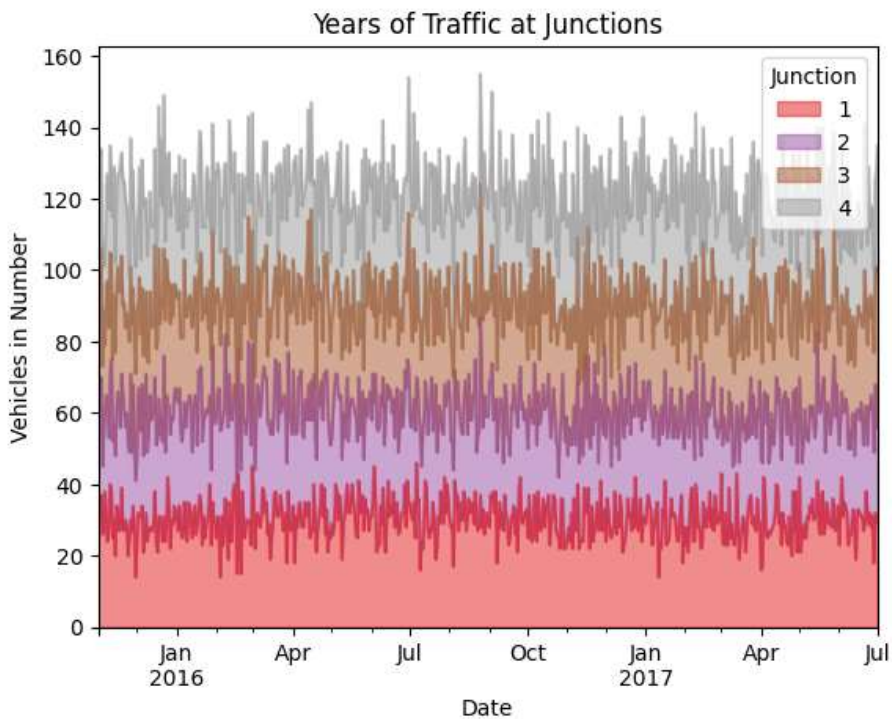


Fig. 6.3 Year of Traffic at junctions.

6.4 Product Recommendations

Machine learning is widely utilized by e-commerce platforms (like Amazon) and streaming services (such as Netflix) for **product and content recommendations**. When users search for products or watch shows, these platforms analyze their browsing history and preferences to make personalized suggestions.

For instance, after a user views a product on Amazon, they may begin seeing ads for similar products across other websites. This is powered by machine learning algorithms that track user behavior and determine preferences. Similarly, Netflix provides personalized recommendations for shows and movies based on a user's viewing history, helping enhance the user experience by suggesting relevant content.

6.5 Self-Driving Cars

Self-driving or autonomous cars represent a cutting-edge application of machine learning. Companies like **Tesla** use machine learning, particularly unsupervised learning techniques, to train car models that can identify objects, people, and obstacles on the road.

The autonomous driving system collects data from various sensors, cameras, and radar systems, then uses deep learning algorithms to interpret the environment, detect lanes, recognize traffic signs, and make driving decisions. This technology allows the car to navigate safely while reducing the need for human intervention.



Fig. 6.4 Self-driving cars.

6.6 Email Spam and Malware Filtering

Machine learning algorithms play a crucial role in **spam and malware detection** within email systems. When an email arrives, it is automatically classified as important, normal, or spam. This classification process relies on several types of filters:

- ❖ **Content Filter:** Analyzes the content of the email for common spam characteristics.
- ❖ **Header Filter:** Examines the email header information for patterns often associated with spam.
- ❖ **General Blacklists:** Blocks known spam senders based on a blacklist.
- ❖ **Rules-Based Filters:** Filters emails based on specific rules set by the user.
- ❖ **Permission Filters:** Allows only emails from pre-approved senders.

Gmail, for example, uses machine learning algorithms such as Naive Bayes, Decision Trees, and Multi-Layer Perceptrons to improve its spam detection and protect users from malware and phishing attacks.

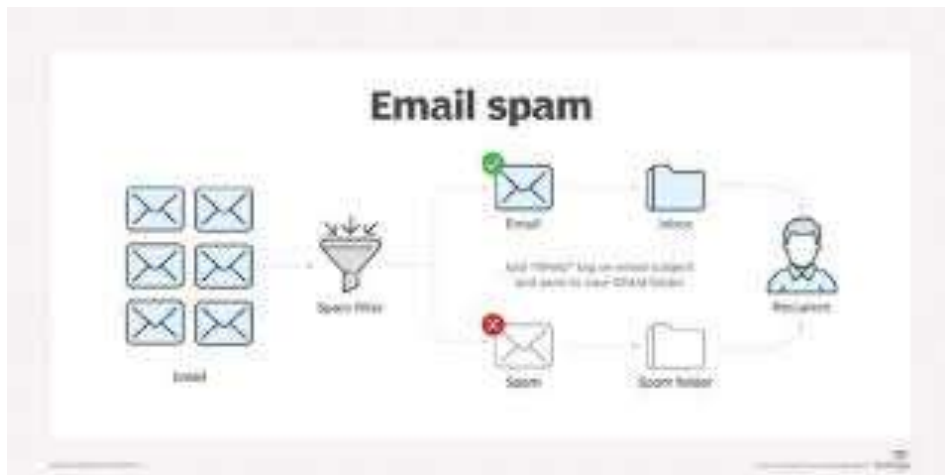


Fig. 6.5 Email Spam.

6.7 Virtual Personal Assistants

Virtual assistants like **Google Assistant, Siri, Alexa, and Cortana** are some of the most widely used applications of machine learning. These assistants can perform various tasks based on voice commands, such as playing music, setting reminders, calling someone, or scheduling appointments.

Machine learning enables these assistants to understand voice commands, analyze them, and respond accurately. Voice instructions are sent to the cloud, where machine learning algorithms process and decode the data to produce a relevant response. Over time, the assistants learn from user interactions and improve their performance by adapting to individual preferences.

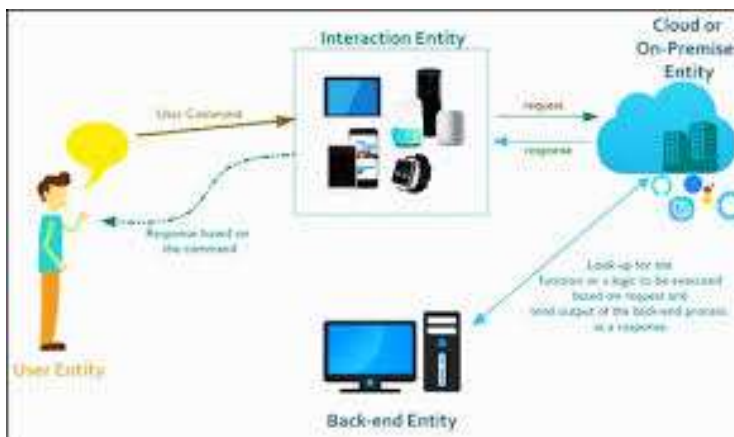


Fig. 6.6 Virtual Personal Assistants.

6.8 Online Fraud Detection

Machine learning is essential for ensuring the security of online transactions by detecting potential fraud. Financial institutions use machine learning algorithms, such as **Feed-Forward Neural Networks**, to identify suspicious transaction patterns and flag them as potentially fraudulent.

For each transaction, the algorithm generates a unique pattern. In genuine transactions, a specific pattern is observed, but for fraudulent transactions, the pattern deviates significantly. By recognizing these deviations, machine learning models help detect and prevent fraud in real time, ensuring safer online transactions.



Fig. 6.7 Fraud Detection.

6.9 Stock Market Trading

Machine learning has become increasingly popular in stock market trading, where models are used to predict stock price movements. Algorithms like **Long Short-Term Memory (LSTM) neural networks** are particularly useful in analyzing historical stock data and identifying trends.

LSTM networks are well-suited for time series prediction, enabling traders to make informed decisions based on forecasted trends. While stock trading remains a risky venture, machine learning algorithms provide insights that help traders reduce risks by identifying potential market movements.



Fig. 6.8 Stock Market Trading.

6.10 Medical Diagnosis

In the field of medical science, machine learning aids in diagnosing diseases by analyzing medical data. For instance, machine learning models can create 3D visualizations to locate lesions in the brain, aiding in the diagnosis of brain tumors and other conditions.

Machine learning algorithms in medical imaging can detect patterns that indicate specific diseases, including cancers, cardiovascular conditions, and neurological disorders. These systems support doctors in early diagnosis and can improve patient outcomes by enabling faster treatment.

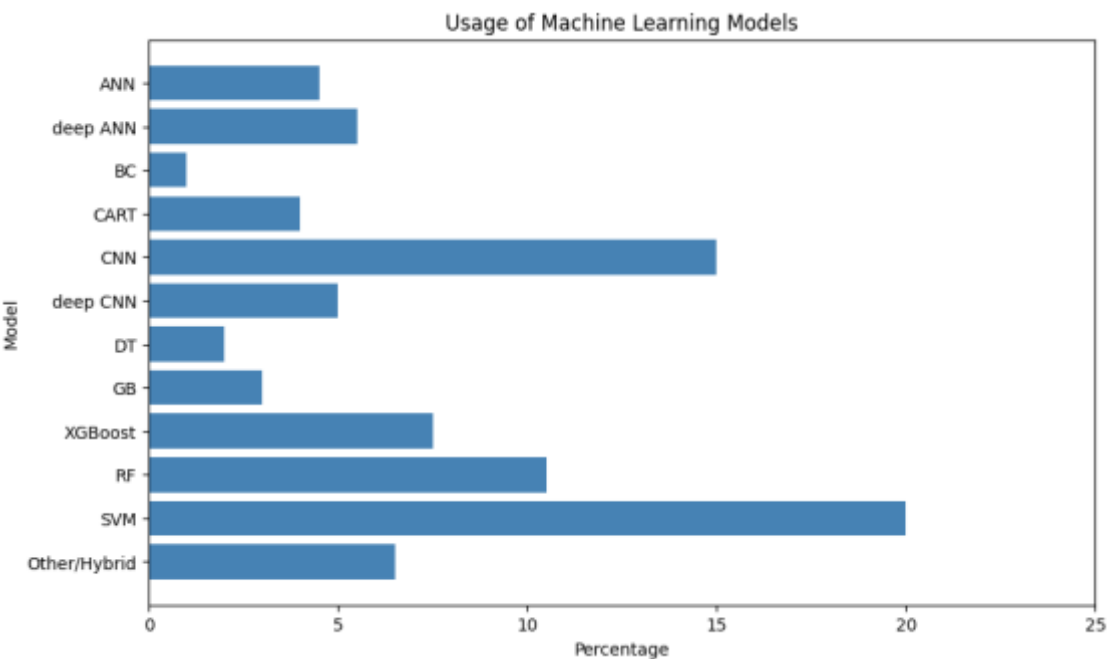


Fig. 6.9 Medical Diagnosis.

6.11 Automatic Language Translation

Machine learning also supports automatic language translation, making it easier for people to communicate across language barriers. **Google Neural Machine Translation (GNMT)** is a notable example that translates text from one language to another using machine learning techniques.

GNMT uses a **sequence-to-sequence learning algorithm** combined with image recognition for translating text in images. For instance, if a tourist in a foreign country encounters a sign in an unfamiliar language, GNMT can translate it into their native language. This technology has broad applications in travel, global communication, and multilingual content management.

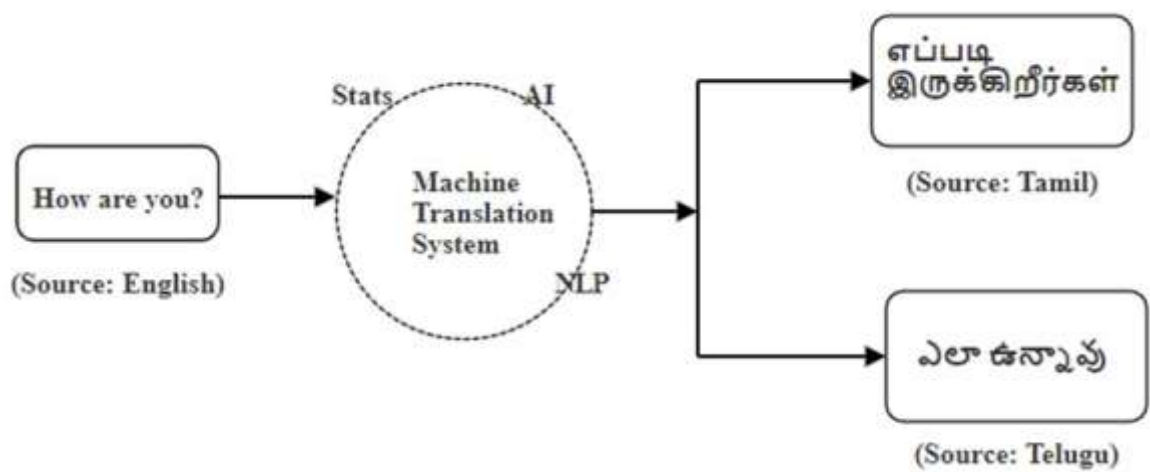


Fig. 6.10 Language Translation.